Architecture Manual

RSL Fulfillment Hub — Colorado Co., Ltd.

01418321 System Analysis and Design — Final Project

คู่มือสถาปัตยกรรมของเว็บ 01418321-final-project

frontend • backend • flow ทั้งระบบ • การทดสอบ • สิ่งที่ต้องแก้ • เตรียมตอบคำถาม

จัดทำโดย **xbpkdi-nov-2026**

อ่านก่อน

- เขียนให้คนที่**ไม่เคยอ่านโค้ด**อ่านเข้าใจได้ ทุกบทเริ่มด้วยภาษาคน แล้วค่อยตามด้วยชื่อไฟล์/ฟังก์ชันสำหรับใช้อ้างอิง
- ถ้าเจอศัพท์ที่ไม่รู้ ให้เปิดบทที่ 1 “ศัพท์ที่ต้องรู้”
- อ้างอิงโค้ด branch `dev` ที่ commit `8c4c7ef` (Merge PR #29, 7 พ.ย. 2569)
- เลขบรรทัดในโค้ดอาจเลื่อนถ้ามี commit ใหม่ ให้ค้นด้วยชื่อฟังก์ชันแทน
- ไฟล์นี้อยู่ใน `321-sa-proj/backend-guide/` ซึ่งอยู่ใน `.gitignore` จึง**ไม่ถูก push**
- สัญลักษณ์ในเล่ม
  - `สถานะ order` = สถานะที่เห็นบนหน้าเว็บ
  - **UC 2A** = รหัส Use Case ตามรายงาน
  - ชื่อไฟล์/ฟังก์ชัน = ชื่อในโค้ด ใช้ค้นหา

สรุป 5 ข้อที่สำคัญที่สุด

1. ฐานข้อมูลจริงมี **17 ตาราง** (รายงานรอบแรกออกแบบ 15 ตาราง ฉบับสุดท้ายแก้เป็น 17 แล้ว)
2. ในโค้ด**ไม่มี** Controller, Repository หรือ Scheduler แบบใน sequence diagram
3. งานอัตโนมัติ (2S, 3S, 6S, 7S) **ไม่ได้รันตามเวลา** แต่รันต่อทันทีหลัง Admin กดปุ่ม
4. ระบบภายนอก (Rakuten, RSL, ขนส่ง, Supplier, เครื่องพิมพ์) ยังเป็น**ของจำลอง (mock)** ทั้งหมด
5. **เกือบไม่มี automated test** มีแค่ตัวเดียว ที่เหลือทดสอบด้วยมือ

สารบัญ

1 ศัพท์ที่ต้องรู้ (ภาษาคน) 4

2 ภาพรวมสถาปัตยกรรม 5

2.1 ชั้นของระบบ (layers) . . . 5
2.2 การอ่านข้อมูล vs การเขียนข้อมูล . . . 5
2.3 สิ่งที่ ไม่มี ในระบบ . . . 6
2.4 เทียบ object ใน sequence/collaboration diagram กับโค้ดจริง . . . 6

3 Frontend 7

3.1 เครื่องมือที่ใช้ (Stack) . . . 7
3.2 โครงโฟลเดอร์ฝั่งหน้าจอ . . . 7
3.3 โครงหน้า (App shell) . . . 7
3.4 ทุกหน้าทำงานแบบเดียวกัน . . . 8
3.5 สิ่งที่เก็บในเบราว์เซอร์ . . . 8
3.6 ไฟล์ช่วยฝั่งหน้าจอ . . . 9
3.7 หน้าตอนพิเศษ . . . 9
3.8 Design system . . . 9
3.9 ถ้าจะเพิ่มหรือแก้ฟีเจอร์ . . . 9

4 รันระบบในเครื่องตัวเอง 10

4.1 สิ่งที่ตั้งไว้ให้แล้วในเครื่องนี้ . . . 10
4.2 เปิดเว็บ (ทำตามลำดับ) . . . 10
4.3 คำสั่งอื่น . . . 10
4.4 หน้า /dev (เครื่องมือทดสอบ) . . . 10

5 ฐานข้อมูล 12

5.1 ER แบบย่อ . . . 12
5.2 แต่ละตารางเก็บอะไร . . . 13
5.3 ค่าที่ต้องรู้ . . . 13
5.4 สร้างตารางและใส่ข้อมูลตัวอย่าง . . . 14

6 Login และการกันหน้า (UC 1A) 15

6.1 ความปลอดภัย . . . 16

7 งานอัตโนมัติ (System auto) ทำงานอย่างไร 18

7.1 ปุ่มไหนพางานอัตโนมัติอะไรตามมา . . . 18
7.2 เส้นทางสถานะของ order (ตามโค้ดจริง) . . . 19

8 Flow ราย Use Case 21

8.1 Flow A: order ปกติ ตั้งแต่ดึงเข้าจนส่งถึงลูกค้า . . . 21
8.2 Flow B: สต๊อกไม่พอ → ตัดสินใจ → สั่งซื้อ . . . 28
8.3 UC 3S ตรวจสอบและปรับสต๊อกด้วยมือ . . . 29
8.4 Flow C: UC 4S คำนวณต้นทุนต่อหน่วย . . . 30
8.5 Flow D: UC 7A ยกเลิกคำสั่งซื้อ . . . 31

8.6 Flow E: UC 10A ลบข้อมูลเก่า . . . 32
8.7 Flow F: UC 3A ตั้งกฎ SKU / ข้อมูลสินค้า . . . 33

9 ระบบภายนอก (Adapters และ Mock) 34
9.1 ทุก adapter ตอบกลับแบบเดียวกัน . . . 34
9.2 adapter แต่ละตัว . . . 34
9.3 ถ้าจะต่อระบบจริง . . . 34

10 การทดสอบ (Testing) 35
10.1 Automated test ตัวเดียวที่มี . . . 35
10.2 เครื่องมือตรวจโค้ด . . . 35
10.3 ทดสอบด้วยมือ (quickstart) . . . 35
10.4 ทำไปแล้วแค่ไหน . . . 36
10.5 ไฟล์ข้อมูลทดสอบ . . . 36
10.6 ส่วนที่ไม่มี test เลย . . . 36

11 จุดอ่อนของ Backend 37

12 ข้อเสนอแนะ: ต้องแก้อะไรต่อ 38
12.1 (ก) รายงาน: ตารางที่ต่างจากโค้ด . . . 38
12.2 (ข) สไลด์ สคริปต์ และ Q&A . . . 39
12.3 (ค) Backend ที่ควรแก้ (เรียงความสำคัญ) . . . 39

13 เตรียมตอบคำถาม: Code 40
13.1 สถาปัตยกรรม . . . 40
13.2 ความถูกต้องของข้อมูล . . . 40
13.3 ความปลอดภัย . . . 41
13.4 งานอัตโนมัติและระบบภายนอก . . . 41
13.5 การทดสอบ . . . 42

14 เตรียมพรี: สิ่งที่อาจารย์ตรวจจริง 43
14.1 เช็กลิสต์เทียบกับงานเรา . . . 43
14.2 MVC ใน Sequence: เทียบกับโค้ดจริง . . . 44

15 ทัวร์หน้าเว็บทีละหน้า 45
15.1 ภาพรวมทุกหน้า . . . 45
15.2 หน้า Login (`/login`) . . . 46
15.3 ภาพรวมระบบ (`/dashboard`) . . . 47
15.4 ตรวจสอบคำสั่งซื้อ (`/orders/verify`) . . . 48
15.5 จับคู่ Order กับ RSL (`/orders/rsl-match`) . . . 49
15.6 ยกเลิก Order (`/orders/cancel`) . . . 50
15.7 ตั้งกฎ SKU / ข้อมูลสินค้า (`/products`) . . . 52
15.8 ตรวจสอบสต๊อก (`/products/stock`) . . . 53
15.9 คำนวณต้นทุนต่อหน่วย (`/products/cost`) . . . 54
15.10 ตัดสินใจสั่งซื้อเพิ่ม (`/products/reorder`) . . . 55
15.11 จัดส่งสินค้าให้ลูกค้า (`/shipping`) . . . 57
15.12 พิมพ์ใบปะสินค้า (`/shipping/label`) . . . 58

15.13 ลบข้อมูลเก่า (/system/cleanup) . . . 59
15.14 หน้ารายงาน (/reports/label/<order>, /reports/unit-cost/<sku>) . . . 60
15.15 เครื่องมือทดสอบ (/dev) . . . 62
15.16 ผังโครงสร้างหน้าจอ (/sitemap) . . . 63

16 เดโม: เล่นโปรแกรม + SELECT ให้ดู **64**
16.1 เปิด DB . . . 64
16.2 ชุด SELECT สำหรับเดโม . . . 64
16.3 ลำดับเดโมแนะนำ (ประมาณ 5 นาที) . . . 65

17 เตรียมตอบคำถาม: Diagram **67**
17.1 Business Process (As-Is / To-Be / Pain point) . . . 67
17.2 Use Case Diagram และ Use Case Description . . . 67
17.3 Sequence และ Collaboration Diagram . . . 67
17.4 CRUD Matrix และ Query Table . . . 68
17.5 ER Diagram และ Table Structure . . . 68
17.6 Class Diagram และ Component Diagram . . . 68
17.7 State Diagram . . . 68

18 ภาคผนวก: แผนที่ไฟล์ **70**

1 ศัพท์ที่ต้องรู้ (ภาษาคน)

เปรียบระบบเป็นร้านอาหาร

- **หน้าเว็บ (frontend)** = หน้าร้าน ลูกค้าเห็นเมนูและสั่งอาหาร
- **Server Action** = พนักงานรับออร์เดอร์ รับคำสั่งจากหน้าร้านไปส่งครัว
- **Use-case function** = พ่อครัว รู้สูตรว่าต้องทำอะไรก่อนหลัง
- **Query / SQL** = การหยิบหรือเก็บของในตู้เย็น
- **ฐานข้อมูล (PostgreSQL)** = ตู้เย็นและชั้นเก็บของ
- **Adapter** = คนติดต่อร้านอื่น (Rakuten, ขนส่ง, supplier) ตอนนี้เป็นคนแสดงบทบาทสมมติ

ศัพท์	ความหมายแบบง่าย
Frontend	ส่วนที่ผู้ใช้เห็นและกดได้ในเบราว์เซอร์
Backend	ส่วนที่ทำงานอยู่บนเครื่อง server ผู้ใช้มองไม่เห็น เช่น ตรวจรหัสผ่าน บันทึกข้อมูล
Next.js	framework ที่ทำให้เขียน frontend กับ backend ไว้ในโปรเจกต์เดียวกันได้
Page (`page.tsx`)	ไฟล์ของหน้าเว็บหนึ่งหน้า ชื่อโฟลเดอร์คือ URL เช่น `orders/verify/page.tsx` = หน้า `/orders/verify`
Layout	กรอบที่ครอบทุกหน้า เช่น เมนูด้านซ้าย และเป็นจุดที่ตรวจว่า login หรือยัง
Server Action	ฟังก์ชันที่หน้าเว็บเรียกได้ตรง ๆ แต่ไปทำงานบน server เหมือนกดปุ่มแล้วมีคนหลังร้านทำต่อ
Use-case function	ฟังก์ชันที่เก็บกฎธุรกิจของ UC หนึ่งตัว เช่น “ตรวจสอบ order ได้เฉพาะสถานะรอตรวจสอบ”
SQL / Query	คำสั่งที่ใช้อ่านหรือเขียนฐานข้อมูล เช่น `SELECT`, `UPDATE`, `INSERT`
ตาราง (table)	ที่เก็บข้อมูลแบบตาราง Excel มีคอลัมน์และแถว
Primary key (PK)	คอลัมน์ที่ไม่ซ้ำกันเลย ใช้ระบุแถว เช่น `order_id`
Foreign key (FK)	คอลัมน์ที่ชี้ไปหาแถวในตารางอื่น เช่น `order_items.order_id` ชี้ไป `orders`
Migration	ไฟล์สร้าง/แก้โครงตาราง รันตามลำดับเลข 001, 002, ...
Seed	ข้อมูลตัวอย่างที่ใส่ลงฐานข้อมูลเพื่อทดสอบ
Transaction	การทำหลายคำสั่งเป็นก้อนเดียว “สำเร็จทั้งหมด หรือไม่เกิดอะไรเลย” เหมือนโอนเงินที่ต้องหักบัญชีหนึ่งและเพิ่มอีกบัญชีพร้อมกัน
Commit / Rollback	commit = ยืนยันบันทึกทั้งก้อน, rollback = ยกเลิกทั้งก้อนกลับไปเหมือนเดิม
Row lock (`FOR UPDATE`)	จองแถวข้อมูลไว้ คนอื่นต้องรอจนเราทำเสร็จ กันสองคนแก้ของชิ้นเดียวกันพร้อมกัน
Idempotent	ทำซ้ำกี่ครั้งผลก็เหมือนทำครั้งเดียว เช่น กดส่งซ้ำแล้วไม่ตัดสต๊อกซ้ำ
Adapter	ตัวกลางคุยกับระบบภายนอก เปลี่ยนจากของจำลองเป็นของจริงได้โดยไม่ต้องแก้ส่วนอื่น
Mock	ของจำลอง ตอบกลับเหมือนระบบจริงแต่ไม่ได้ต่อจริง
Session / Cookie	บัตรผ่านหลัง login เบราว์เซอร์ถือ cookie ไว้ server ตรวจทุกครั้งที่เปิดหน้า
Hash (scrypt, sha256)	การแปลงข้อความเป็นรหัสที่ย้อนกลับไม่ได้ ใช้เก็บรหัสผ่านแทนตัวจริง
Snapshot	ภาพถ่ายข้อมูลทั้งหมด ณ ตอนเปิดหน้า หน้าเว็บอ่านจากภาพนี้
Revalidate	สั่งให้ถ่าย snapshot ใหม่หลังมีการบันทึก หน้าเว็บจึงอัปเดตเอง
Lint / Type check / Build	เครื่องมือตรวจโค้ดอัตโนมัติ: เขียนผิดรูปแบบ / ชนิดข้อมูลไม่ตรง / สร้างเว็บจริงไม่ผ่าน
Automated test	โปรแกรมที่ทดสอบระบบให้เองโดยไม่ต้องกดด้วยมือ

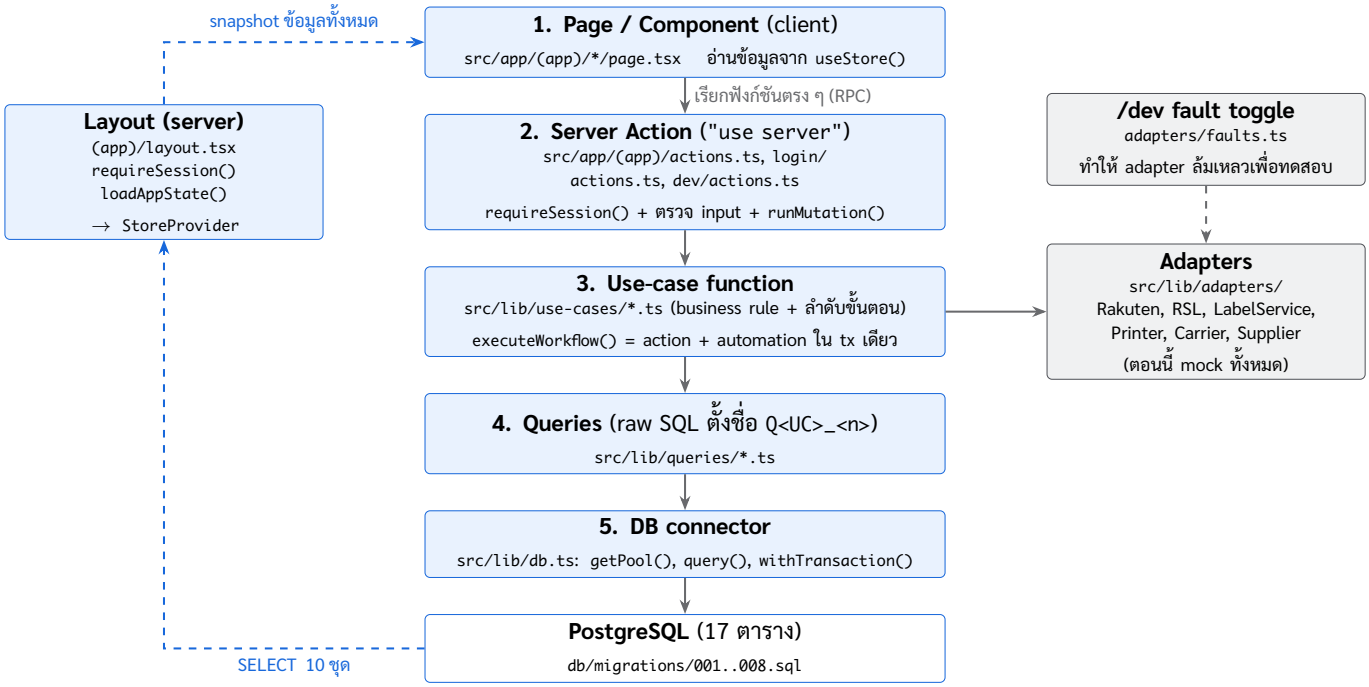
2 ภาพรวมสถาปัตยกรรม

อธิบายแบบภาษาคน

1. ผู้ใช้กดปุ่มบนหน้าเว็บ
2. หน้าเว็บส่งคำขอไปที่ Server Action
3. Server Action ตรวจว่า login แล้ว และข้อมูลที่ส่งมาถูกรูปแบบ
4. Server Action ส่งต่อให้ use-case function ซึ่งรู้กฎของงานนั้น
5. use-case ใช้ query อ่าน/เขียนฐานข้อมูล และถ้าต้องคุยกับระบบภายนอกก็เรียก adapter
6. บันทึกเสร็จ ระบบถ่าย snapshot ใหม่ หน้าเว็บอัปเดตเอง

2.1 ชั้นของระบบ (layers)

- เว็บสร้างด้วย **Next.js 16 App Router**
- หน้าจอกับ backend อยู่ในโปรเจกต์เดียวกัน
- โค้ดฝั่ง server อยู่ใน `src/lib/` และไฟล์ `actions.ts`



2.2 การอ่านข้อมูล vs การเขียนข้อมูล

อ่าน (เปิดหน้าเว็บ)

1. ทุกหน้าที่ต้อง login ผ่าน layout ฝั่ง server ก่อน
2. layout เรียก `requireSession()` ตรวจบัตรผ่าน ถ้าไม่มีจะพาไป `/login`
3. layout เรียก `loadAppState()` ดึงข้อมูลทุกตาราง (10 ชุด) พร้อมกัน
4. ข้อมูลทั้งหมดส่งให้ `StoreProvider` เป็น snapshot
5. หน้าเว็บอ่าน snapshot ด้วย `useStore()` หน้าเว็บ**แก้ snapshot เองไม่ได้**

เขียน (กดปุ่มบันทึก)

1. ปุ่มเรียก Server Action เช่น `verifyOrderAction(orderId, ...)`
2. action ตรวจ login และข้อมูล แล้วส่งงานให้ use-case ผ่าน `runMutation()`
3. ถ้าสำเร็จ `runMutation` สั่ง `revalidatePath` ให้ถ่าย snapshot ใหม่

4. ถ้าพัง runMutation แปลง error ให้เป็นข้อความที่ปลอดภัยส่งกลับหน้าเว็บ

src/app/(app)/actions.ts:40

```
async function runMutation<T>(work: () => Promise<T>): Promise<MutationResult<T>> {
  try {
    const result = await work();
    revalidatePath("/", "layout");
    return { success: true, result };
  } catch (error) {
    return { success: false, error: mapDatabaseError(error).messageKey };
  }
}
```

อ่านโค้ดนี้ทีละบรรทัด

- บรรทัด 1: ฟังก์ชันรับ “งาน” (work) มาหนึ่งชิ้น
- บรรทัด 3: ลองทำงานนั้น
- บรรทัด 4: ทำสำเร็จ → สั่งให้หน้าเว็บโหลดข้อมูลใหม่
- บรรทัด 5: ส่งกลับ “สำเร็จ” พร้อมผล
- บรรทัด 6–7: ถ้าพัง → ส่งกลับ “ไม่สำเร็จ” พร้อมรหัสข้อความ error

2.3 สิ่งที่ ไม่มี ในระบบ

- **ไม่มี REST API**
  - ไม่มีไฟล์ route.ts
  - หน้าเว็บเรียก Server Action ตรง ๆ Next.js ส่งข้อมูลผ่านเน็ตให้เอง
- **ไม่มี middleware**: การกันหน้าที่ต้อง login ทำใน layout และต้นทุก action
- **ไม่มี Controller / Repository class**: ใช้ฟังก์ชันธรรมดาแยกไฟล์ตามเรื่อง
- **ไม่มี scheduler (ตั้งเวลา)**: งานอัตโนมัติรันต่อท้ายปุ่มของ Admin (บทที่ 7)
- **ไม่มี ORM**
  - เขียนคำสั่ง SQL เองผ่าน library pg
  - ค่าทุกตัวส่งแยกเป็น $1, $2 กันการแทรกคำสั่งอันตราย (SQL injection)

2.4 เทียบ object ใน sequence/collaboration diagram กับโค้ดจริง

Object ใน diagram	ของจริงในโค้ด
:XxxScreen (ชั้น View ของ MVC)	หน้าเว็บ page.tsx ใน src/app/(app)/... และ src/app/login เช่น :OrderVerifyScreen = verify/page.tsx; :AppShellScreen = layout.tsx
:XxxController (ชั้น Controller)	Server Action ใน src/app/(app)/actions.ts และ login/actions.ts เช่น verifyOrderAction (เป็นฟังก์ชัน ไม่มี class ชื่อ Controller)
:XxxUseCase, :ExecuteWorkflow (Model)	ฟังก์ชันใน src/lib/use-cases/*.ts และ execute-workflow.ts
:XxxQueries (Model)	ไฟล์ใน src/lib/queries/*.ts แยกตาม**เรื่อง** เช่น orders.ts, skus.ts (แทน Repository เดิม)
:DBConnector + query(Qx.n)	src/lib/db.ts (getPool, withTransaction) ชื่อค่าคงที่ในโค้ดเป็น Q2A_2 เลขอาจไม่ตรงเลขในเล่ม (คู่เทียบอยู่ใน qmap ของ usecases.json)
:SessionAuth	src/lib/auth/sessions.ts
:RakutenAdapter, :RslAdapter, :CarrierAdapter, :SupplierAdapter, :PrinterAdapter, :LabelServiceAdapter	adapter 6 ตัวใน src/lib/adapters/ (ตอนนี้เป็น mock)
Repository, Scheduler แบบเล่มเดิม	**ไม่มีแล้ว** ทั้งใน diagram และในโค้ด งานอัตโนมัติใช้ executeWorkflow แทน Scheduler

3 Frontend

อธิบายแบบภาษาคน

- frontend = ทุกอย่างที่เห็นบนจอ: เมนู ตาราง ปุ่ม ฟอร์ม ข้อความแจ้งเตือน
- frontend **ไม่เก็บข้อมูลธุรกิจเอง** แค่แสดง snapshot และส่งคำขอไป Server Action
- ทุกหน้าใช้ชิ้นส่วนสำเร็จรูปชุดเดียวกัน หน้าตาจึงเหมือนกันทั้งเว็บ

3.1 เครื่องมือที่ใช้ (Stack)

เครื่องมือ	ใช้ทำอะไร
Next.js 16.3 + React 19.2	สร้างหน้าเว็บ แบ่งหน้าตามโฟลเดอร์
Tailwind CSS 4	ตกแต่งหน้าตา สีอยู่ใน `src/app/globals.css`
shadcn/ui + Radix UI	ชิ้นส่วนสำเร็จรูป 27 ตัว (ปุ่ม, การ์ด, ตาราง, หน้าต่าง) ใน `src/components/ui/`
`@tanstack/react-table`	ตารางทุกหน้า (ค้นหา เรียง แบ่งหน้า)
`recharts`	กราฟบน dashboard
`sonner`	กล่องแจ้งเตือนเล็ก ๆ มุมจอ (toast)
`lucide-react`	ไอคอน

3.2 โครงโฟลเดอร์ฝั่งหน้าจอ

```
src/app/
  layout.tsx              กรอบนอกสุด (ฟอนต์, toast, ภาษา)
  page.tsx                "/" พาไป /login
  login/                  หน้า login
  (app)/                  ทุกหน้าที่ต้อง login (มีเมนูซ้าย)
    layout.tsx            ตรวจ login + โหลด snapshot + เมนู
    loading.tsx           หน้าระหว่างโหลด
    actions.ts            Server Action ทั้งหมด
    dashboard/  orders/  products/  shipping/  system/  dev/
  reports/                หน้าสำหรับพิมพ์ (ใบปะ, รายงานต้นทุน)
src/components/
  ui/                     ชิ้นส่วนสำเร็จรูป (shadcn)
  shared/                 ชิ้นส่วนที่ทีมทำเอง (ตาราง, ป้ายสถานะ, ...)
  app-shell/              เมนูซ้าย, แถบบน, ปุ่มเปลี่ยนภาษา
```

- `(app)` ในวงเล็บไม่ใช่ส่วนของ URL ใช้จัดกลุ่มหน้าที่ต้อง login
- `src/app/api/` เป็นโฟลเดอร์ว่าง ไม่มีอะไรข้างใน

3.3 โครงหน้า (App shell)

SiteHeader (`app-shell/site-header.tsx`) — breadcrumb + ปุ่มสลับภาษา

AppSidebar
(`app-sidebar.tsx`)
เมนูจาก `lib/nav.ts`
5 กลุ่ม: หน้าหลัก, คำสั่งซื้อ, สินค้าและสต๊อก, จัดส่ง, ระบบ
ล่างสุด: ผู้ใช้ + logout
(ถามยืนยันถ้ามีงานค้าง)

page.tsx ของแต่ละเมนู
PageHeader (หัวข้อ + คำอธิบาย)
Card → TableSearch + DataTable (columns จาก `columns.tsx`)
StatusBadge แสดงสถานะ order
Dialog / Sheet สำหรับฟอร์มและยืนยัน
EmptyState, SectionMessage, DatabaseErrorState
อ่านข้อมูล: `useStore().state`
เขียนข้อมูล: เรียก Server Action ใน `startTransition`

3.4 ทุกหน้าทำงานแบบเดียวกัน

1. หน้าเว็บอ่าน snapshot ด้วย `useStore()`
2. กรองเฉพาะ order ที่เกี่ยวกับหน้านั้น เช่น `labelQueue()` เอาเฉพาะ order ที่รอพิมพ์ใบปะ
3. ผู้ใช้กดปุ่ม → ปุ่มขึ้นสถานะ “กำลังทำ” (`useTransition`)
4. หน้าเว็บเรียก Server Action
5. ได้ผลกลับมา → แสดง toast หรือข้อความในแผง
6. หน้าเว็บไม่ต้องโหลดใหม่เอง snapshot ใหม่มาให้อัตโนมัติ

src/app/(app)/orders/verify/page.tsx:101

```
function confirm() {
  if (!selected) return;
  setError(null);
  const orderId = selected.order_id;
  startSubmitting(async () => {
    const action = await verifyOrderAction(orderId, shippingMethod);
    if (!action.success) {
      setError(t.errors[action.error]);
      return;
    }
    if (action.result === "ok") {
      setSelectedId(null);
      toast.success(t.verify.okVerified);
      return;
    }
  });
}
```

อ่านโค้ดนี้ทีละบรรทัด

- บรรทัด 2: ยังไม่ได้เลือก order → ไม่ทำอะไร
- บรรทัด 3: ล้างข้อความ error เก่า
- บรรทัด 5: เริ่ม “กำลังทำ” ปุ่มจะกดซ้ำไม่ได้
- บรรทัด 6: เรียก Server Action ตรวจสอบ order
- บรรทัด 7–9: ถ้าฐานข้อมูลพัง → แสดงข้อความ error
- บรรทัด 11–13: ถ้าสำเร็จ → ปิดแผงรายละเอียดและแสดง toast “ตรวจสอบแล้ว”

3.5 สิ่งที่เก็บในเบราว์เซอร์

- **ภาษาที่เลือก** (ไทย/อังกฤษ) เก็บใน localStorage key `rsl-hub-lang`
- นอกนั้น**ไม่เก็บอะไรเลย** ข้อมูลธุรกิจและ session อยู่ที่ server ทั้งหมด
- `store.tsx` จำแค่ว่ามีฟอร์มที่ยังไม่บันทึกไหม เพื่อถามยืนยันก่อน logout
- ข้อความทุกหน้าอยู่ใน `src/lib/i18n/dict.ts` มีทั้งไทยและอังกฤษ

3.6 ไฟล์ช่วยฝั่งหน้าจอ

ไฟล์ / ฟังก์ชัน	หน้าที่
`lib/nav.ts`	รายการเมนูซ้ายและ breadcrumb
`lib/order-status.ts`	22 สถานะ order (ลอกจากรายงาน ห้ามแก้เอง) และสีของป้าย
`lib/issues.ts`	order ที่ค้างต้องไปแก้หน้าไหน เช่น ไม่มีกฎ SKU → ไปหน้า `/products`
`lib/format.ts`	จัดรูปแบบเงิน วันที่ ตัวเลข
`workflow.ts`: `labelQueue`, `shipmentQueue`, `rslQueue`, `purchaseQueue`	เลือก order ที่จะแสดงในแต่ละหน้า
`workflow.ts`: `checkStock`, `checkCancel`, `unitCostOf`	คำนวณป้ายบนจอ: สต๊อกตำไหม, ยกเลิกได้ไหม, ต้นทุนเท่าไร
`workflow.ts`: `planRangeCleanup`, `planClosedCleanup`	สร้างรายการที่จะลบก่อนส่งให้ server

workflow.ts เป็นของเก่าที่ยังใช้อยู่ครึ่งหนึ่ง

- ไฟล์นี้ยาว 1816 บรรทัด เคยเป็นระบบทั้งหมดตอนเว็บยังเก็บข้อมูลในเบราว์เซอร์
- ฟังก์ชันบันทึกในไฟล์นี้ (`verifyOrder`, `cancelOrder`, ...) **ไม่ได้ใช้แล้ว**
- ส่วนที่ยังใช้: ตัวกรองคิว ตัวคำนวณป้าย และกฎบางข้อที่ server ใช้ร่วม
- จะแก้กฎธุรกิจ → แก้ที่ `src/lib/use-cases/` ไม่ใช่ไฟล์นี้

3.7 หน้าตอนพิเศษ

- **กำลังโหลด**: แสดงกล่องเทาจาง ๆ (`PageLoading`)
- **ฐานข้อมูลล่ม**: แสดงหน้าแจ้งว่าต่อฐานข้อมูลไม่ได้ (`DatabaseErrorState`)
- **error ที่ไม่คาดคิด**: `src/app/error.tsx`
- **ตารางว่าง**: แสดงข้อความ “ไม่มีรายการ” (`EmptyState`)
- **หน้าพิมพ์**: `reports/*` มีแถบปุ่มพิมพ์ (`PrintToolbar`)

3.8 Design system

- ทำตาม `DESIGN.md`: หน้าจอสำหรับ “ทำงาน” เน้นอ่านง่ายและเหมือนกันทุกหน้า
- สีทั้งหมดมาจาก **Atlassian Design System ธีม light**
- ทุกสีใน `globals.css` มีชื่อ token ต้นทางกำกับ เช่น สีหลัก `#1868db` = `color.background.brand.bold`
- ข้อยกเว้นเดียว: สี coral `#ff5a36` ของแบรนด์ ใช้แค่จุดท้ายหัวข้อใหญ่และหน้า login

3.9 ถ้าจะเพิ่มหรือแก้ฟีเจอร์

1. ต้องเก็บข้อมูลใหม่ → เพิ่ม migration `db/migrations/008_*.sql` และ seed
2. เขียน SQL ใน `src/lib/queries/<เรื่อง>.ts` ตั้งชื่อ `Q<UC>_<n>`
3. ส่งข้อมูลให้หน้าจอ → เพิ่มใน `loadAppState` (`queries/app-state.ts`)
4. เขียนกฎธุรกิจใน `src/lib/use-cases/<uc>.ts`
5. เพิ่ม Server Action ใน `src/app/(app)/actions.ts`
6. แก้หน้าเว็บ `page.tsx`, `columns.tsx` และข้อความใน `i18n/dict.ts` ทั้งสองภาษา
7. ถ้าเป็นหน้าใหม่ → เพิ่มเมนูใน `lib/nav.ts`
8. ตรวจด้วย `npm run lint`, `npx tsc --noEmit`, `npm run build` แล้วลองกดบนเว็บจริง

4 รันระบบในเครื่องตัวเอง

4.1 สิ่งที่ตั้งไว้ให้แล้วในเครื่องนี้

- ฐานข้อมูล PostgreSQL 16 รันใน Docker ชื่อ `rsl-pg` (port 5432)
- ไฟล์ตั้งค่า `01418321-final-project/.env.local` (ไม่ถูก push)

```
DATABASE_URL=postgresql://rsl_app:rsl_local@localhost:5432/rsl_fulfillment_hub
SEED_ADMIN_PASSWORD=demo1234
SESSION_DURATION_HOURS=8
RAKUTEN_ADAPTER=mock
```

- บรรทัด 1: ที่อยู่ของฐานข้อมูล (ชื่อผู้ใช้ รหัส เครื่อง ชื่อฐานข้อมูล)
- บรรทัด 2: รหัสผ่านของบัญชีทดสอบ
- บรรทัด 3: login แล้วอยู่ได้ 8 ชั่วโมง
- บรรทัด 4: ใช้ Rakuten จำลอง (adapter อื่นก็ตั้งเป็น `mock` เหมือนกัน ค่าอื่นจะ error)

4.2 เปิดเว็บ (ทำตามลำดับ)

1. เปิด Docker Desktop
2. รัน `docker start rsl-pg` เพื่อเปิดฐานข้อมูล
3. เข้าโฟลเดอร์ `01418321-final-project`
4. รัน `npm run dev`
5. เปิด `http://localhost:3000`
6. login ด้วย `admin@colorado.jp` / `demo1234`

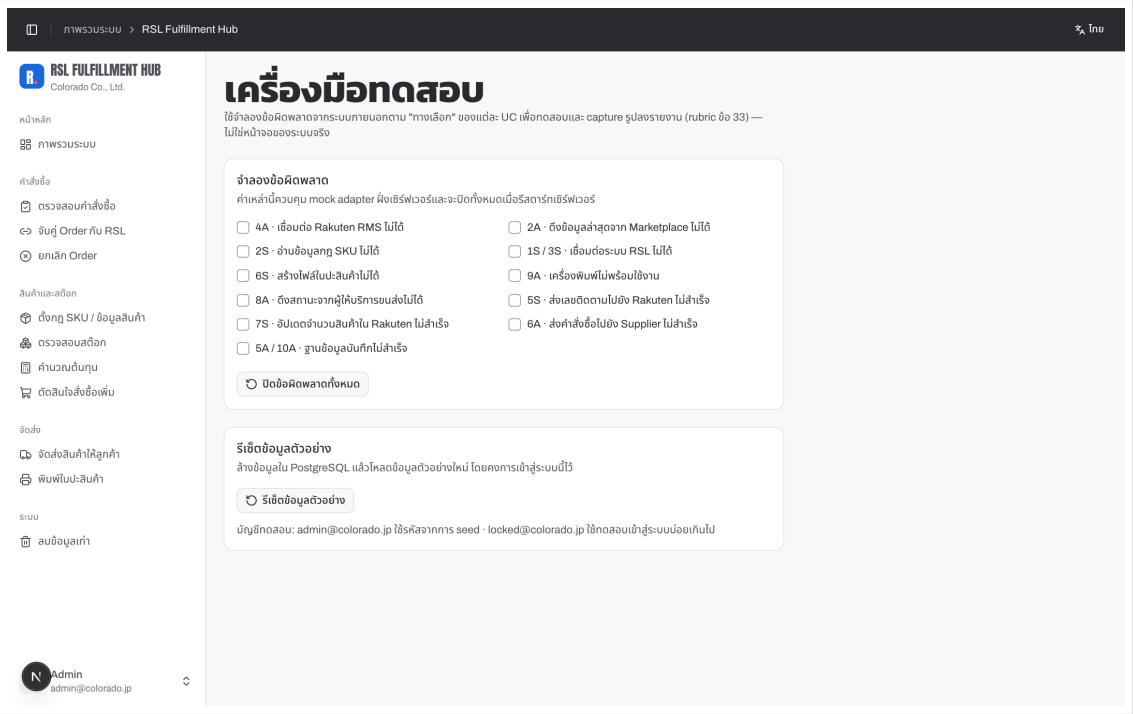
- บัญชี `locked@colorado.jp` (รหัสเดียวกัน) ถูกล็อก 15 นาทีหลัง reset ใช้ทดสอบกรณีบัญชีถูกล็อก (ปลดเร็วได้ที่ปุ่ม “ปลดล็อกบัญชี” ใน `/dev`)

4.3 คำสั่งอื่น

คำสั่ง	ทำอะไร
`npm run db:setup`	สร้างตาราง + ใส่ข้อมูลตัวอย่าง รันซ้ำได้ ไม่ลบของเดิม
`npm run db:reset`	**ลบข้อมูลทั้งหมด** แล้วใส่ข้อมูลตัวอย่างใหม่
`npm run test:phase8-reset`	automated test ตัวเดียวที่มี (บทที่ 10)
`npm run lint`	ตรวจรูปแบบโค้ด
`npx tsc --noEmit`	ตรวจชนิดข้อมูล
`npm run build`	ลองสร้างเว็บจริง

4.4 หน้า /dev (เครื่องมือทดสอบ)

- ไม่อยู่ในเมนู พิมพ์ URL `/dev` เอง (ต้อง login ก่อน)
- **สวิตช์จำลองความเสีย (fault)**
  - เปิดแล้วระบบภายนอกตัวนั้นจะ “ล่ม” เช่น `rms`, `rsl`, `carrier`, `printer`, `supplierSend`, `db`
  - ใช้ทดสอบว่าระบบรับมือได้ไหมเวลาระบบอื่นล่ม
  - รีสตาร์ทเว็บแล้วสวิตช์กลับเป็นปิดทั้งหมด
- **ปุ่ม reset ข้อมูลตัวอย่าง**
  - ลบข้อมูลทั้งหมดแล้วใส่ข้อมูลตัวอย่างใหม่
  - ระบบ login ให้ใหม่อัตโนมัติ



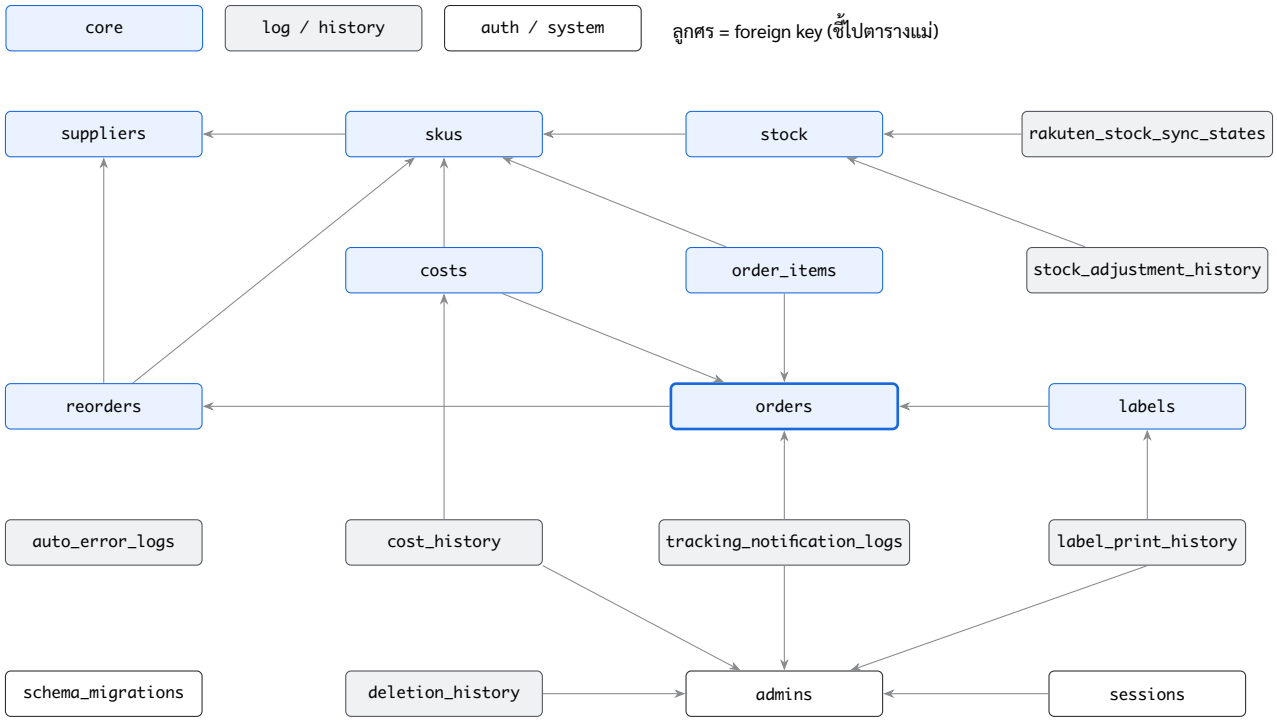
ภาพ: หน้า /dev — สวิตช์จำลองความเสียและปุ่ม reset ข้อมูล

5 ฐานข้อมูล

อธิบายแบบภาษาคน

- ฐานข้อมูลมี 17 ตาราง แบ่งเป็น 3 กลุ่ม
  - **ข้อมูลหลัก** (ฟ้า): order, สินค้า, สต๊อก, ต้นทุน, supplier, ใบปะ, การสั่งซื้อ
  - **ประวัติ/log** (เทา): เก็บว่าใครทำอะไรเมื่อไร หรือระบบพังตรงไหน
  - **ระบบ** (ขาว): ผู้ใช้, session, ประวัติ migration
- ตารางที่สำคัญที่สุดคือ `orders` ทุก flow หมุนรอบสถานะของ order

5.1 ER แบบย่อ



- ลูกศรชี้จากตารางลูกไปตารางแม่ (foreign key)
- ตัวอย่าง: `order_items` → `orders` แปลว่า สินค้าแต่ละรายการรู้ว่าอยู่ใน order ไหน

ตาราง	เก็บอะไร (คอลัมน์สำคัญ)	migration

5.2 แต่ละตารางเก็บอะไร

ตาราง	เก็บอะไร (คอลัมน์สำคัญ)	migration
ข้อมูลหลัก		
`orders`	คำสั่งซื้อ: เลข Rakuten (ไม่ซ้ำ), สถานะ 22 ค่า, เลข RSL, เลขติดตามพัสดุ, ขนส่ง, แจ้งลูกค้าแล้วหรือยัง, เหตุผลที่ค้าง (`manual_reason`), ประวัติแก้ไข, สถานะจากขนส่ง, ใบสั่งซื้อที่รออยู่ (`reorder_id`)	001, 003–006
`order_items`	สินค้าใน order: SKU, จำนวน, ส่งจากคลังไหน (in_house/rsl), ตัดสต๊อกแล้วหรือยัง	001
`skus`	สินค้า: รหัส SKU, รหัสบน Rakuten (`channel_sku` = กฎ SKU), ชื่อ, ราคาขาย, จุดสั่งซื้อซ้ำ, supplier, เปิด/ปิดขาย	001
`stock`	จำนวนในคลังเอง (`in_house_qty`) และในคลัง RSL (`rsl_qty`) ของแต่ละ SKU	001
`costs`	ส่วนประกอบต้นทุนและต้นทุนต่อหน่วยล่าสุด	001, 002
`suppliers`	ผู้ขาย: ชื่อ, ติดต่อ, ระยะเวลาส่งของ (วัน)	001
`reorders`	การสั่งซื้อจาก supplier: จำนวน, สถานะ (รวม ได้รับสินค้าแล้ว), วันที่คาดว่าได้ของ, รหัสคำขอ (กันส่งซ้ำ), วันที่และจำนวนที่ได้รับจริง (`received_qty`) ใบสั่งซื้อหนึ่งใบครอบคลุมหลาย order ได้	001, 004, 006
`labels`	ใบปะหน้าพัสดุ: แม่แบบ, เลข RSL, กล่องที่เท่าไร, สถานะ `ready/printed`	001
ประวัติ / log		
`cost_history`	ค่าต้นทุนก่อน/หลังทุกครั้งที่คำนวณ	002
`stock_adjustment_history`	ปรับสต๊อกเองเมื่อไร จากเท่าไรเป็นเท่าไร เพราะอะไร	004
`label_print_history`	สั่งพิมพ์ใบปะแต่ละครั้ง สำเร็จไหม พิมพ์ซ้ำไหม	005
`tracking_notification_logs`	ส่งเลขติดตามให้ลูกค้าแต่ละครั้ง ผลเป็นอย่างไร	002, 005
`rakuten_stock_sync_states`	ยอดสต๊อกส่งไป Rakuten แล้วหรือยัง (pending/synced)	002
`deletion_history`	order ที่ถูกลบ (เก็บสำเนาไว้ก่อนลบ)	002
`auto_error_logs`	งานอัตโนมัติพังที่ขั้นไหน รหัส error อะไร	002
ระบบ		
`admins`	บัญชีผู้ใช้: email, รหัสผ่านแบบ hash, จำนวนครั้งที่ใส่ผิด, ล็อกถึงเมื่อไร (`locked_until`), login ล่าสุด	001, 007
`sessions`	บัตรผ่านหลัง login (เก็บแค่ hash), หมดอายุเมื่อไร, logout แล้วหรือยัง	002
`schema_migrations`	ไฟล์ migration ไหนรันไปแล้ว	002

5.3 ค่าที่ต้องรู้

สถานะ order 22 ค่า (ตรงกับรายงานทุกค่า)

รอตรวจสอบคำสั่งซื้อ · รอจับคู่กฎ SKU · รอตรวจสอบสต๊อก · รอ Admin ตัดสินใจสั่งซื้อ · รอสั่งซื้อจาก Supplier · สั่งซื้อแล้ว · ยังไม่ได้จับคู่ · รอจัดรูปแบบใบปะสินค้า · รอพิมพ์ใบปะสินค้า · พิมพ์ใบปะสินค้าแล้ว · รอส่งคำสั่งซื้อ · รอส่งมอบ · อยู่ระหว่างจัดส่ง · รอแจ้งเลขติดตาม · แจ้งเลขติดตามแล้ว · จัดส่งสำเร็จ · จัดส่งไม่สำเร็จ · ตีกลับ/คืนสินค้า · รอยกเลิก Order · ยกเลิกแล้ว · รอดำเนินการด้วยตนเอง · รอดำเนินการพิเศษ

เหตุผลที่ order ค้าง “รอดำเนินการด้วยตนเอง” (`manual_reason`)

1. `incomplete`: ข้อมูล order ไม่ครบ
2. `sku-unregistered`: ยังไม่มีสินค้านี้ในระบบ
3. `sku-rule`: ไม่มีกฎ SKU หรือมีหลายกฎ
4. `rsl`: จับคู่กับ RSL ไม่ได้
5. `template`: ไม่มีแม่แบบใบปะสำหรับวิธีจัดส่งนี้

6. `address`: ไม่มีที่อยู่หรือวิธีจัดส่ง
7. `price`: สินค้าไม่มีราคาขาย
8. `supplier`: ไม่มี supplier หรือระยะเวลาส่งของ
9. `stock-deduct`: ตัดสต๊อกไม่พอ

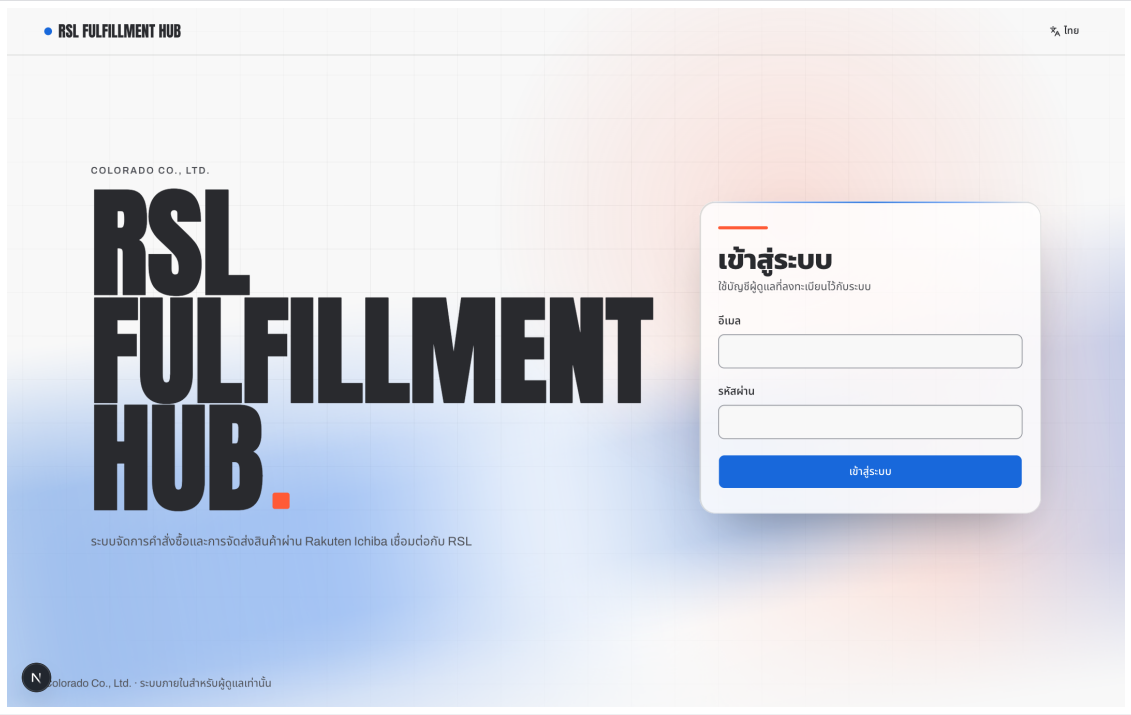
สถานะจากขนส่ง (`delivery_status`) แยกจากสถานะ order

- `in_transit` กำลังส่ง `delivered` ส่งถึงแล้ว `failed` ส่งไม่สำเร็จ `returned` ตีกลับ

5.4 สร้างตารางและใส่ข้อมูลตัวอย่าง

1. `npm run db:setup` เรียก `db/seed/cli.ts`
2. ขั้น migrate: รันไฟล์ `db/migrations/001..008` ที่ยังไม่เคยรัน (ทำในก้อนเดียว)
3. ขั้น seed: ใส่ข้อมูลตัวอย่างจาก `src/mock/*.ts`
   - ผู้ใช้ 2 คน (ปกติ 1, ถูกล็อก 1)
   - supplier 4 ราย, สินค้า สต๊อก ต้นทุน ของทุก SKU
   - order 28 รายการ แต่ละตัวตั้งใจให้อยู่คนละสถานะ เพื่อทดสอบได้ทุก flow
   - การสั่งซื้อตัวอย่าง และสถานะการส่งยอดไป Rakuten
4. ถ้ามีข้อมูลอยู่แล้วจะข้ามไป (`ON CONFLICT DO NOTHING`) จึงรันซ้ำได้
5. `db:reset` ต่างกันตรงที่ล้างทุกตารางก่อน (`TRUNCATE`)

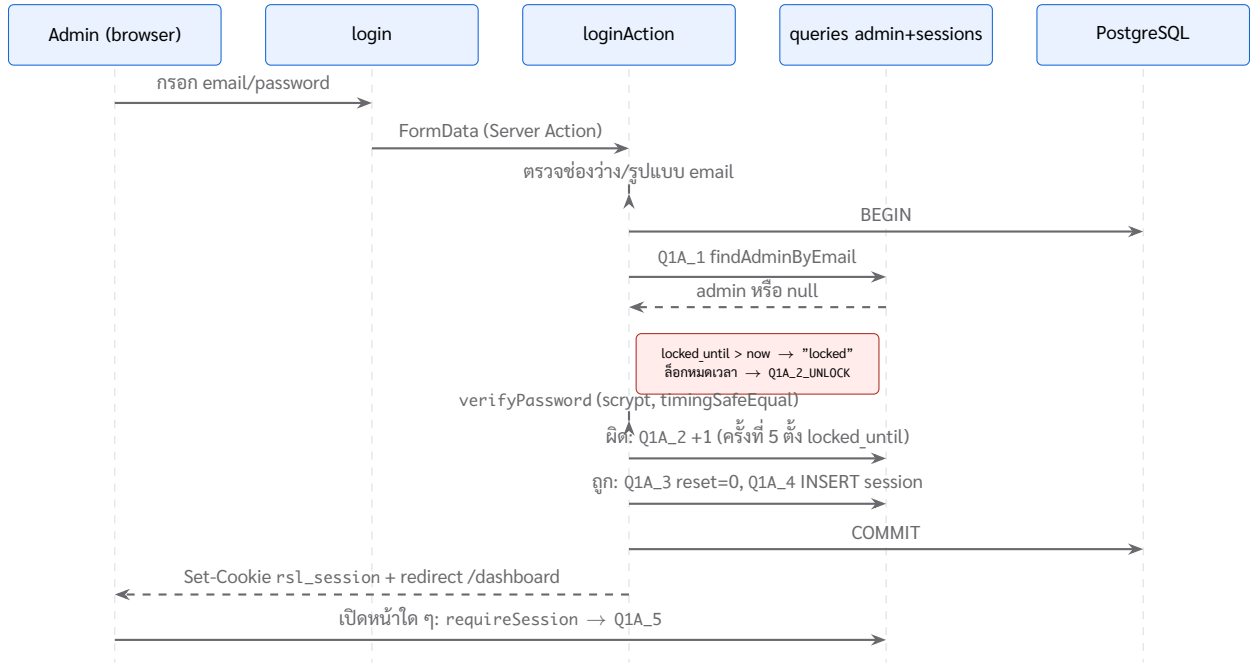
6 Login และการกันหน้า (UC 1A)



ภาพ: หน้า /login

เกิดอะไรขึ้นเมื่อกด “เข้าสู่ระบบ”

1. ระบบเช็คว่ากรอกครบ และ email ถูกรูปแบบ
2. ระบบหาบัญชีจาก email ไม่เจอ → “email หรือรหัสผ่านผิด”
3. บัญชียังอยู่ในช่วงล็อก (`locked_until` ยังไม่ถึง) → “บัญชีถูกล็อก กรุณารออีก N นาที” (เช็คก่อนดูรหัสผ่าน)
4. ล็อกหมดเวลาแล้ว → ล้างจำนวนครั้งที่ผิดและเวลาล็อกก่อน แล้วตรวจต่อ
5. ตรวจรหัสผ่าน ผิด → นับครั้งที่ผิดเพิ่ม 1 แล้วแจ้งว่าผิด ครั้งที่ 5 ตั้งล็อก 15 นาทีและแจ้งว่าถูกล็อกทันที
6. ถูก → ล้างจำนวนครั้งที่ผิด บันทึกเวลา login และออกบัตรผ่าน (session)
7. เบราว์เซอร์ได้ cookie บัตรผ่าน แล้วพาไป `/dashboard`
8. ทุกครั้งที่เปิดหน้า ระบบตรวจบัตรผ่าน ไม่มีหรือหมดอายุ → กลับไปหน้า login



src/app/login/actions.ts:58

```
const result = await withTransaction(async (client) => {
  const now = new Date();
  const admin = await findAdminByEmail(email, client);
  if (!admin) return { result: "wrong" };
  if (admin.locked_until && admin.locked_until > now)
    return { result: "locked", retryMinutes: minutesUntil(admin.locked_until, now) };
  if (admin.locked_until || admin.fail_attempts >= LOGIN_MAX_ATTEMPTS)
    await unlockAdmin(admin.admin_id, client);
  const valid = await verifyPassword(password, admin.user_password);
  if (!valid) {
    const failed = await recordFailedLogin(admin.admin_id, now, client);
    if (failed.lockedUntil && failed.lockedUntil > now)
      return { result: "locked", retryMinutes: minutesUntil(failed.lockedUntil, now) };
    return { result: "wrong" };
  }
  await recordSuccessfulLogin(admin.admin_id, now, client);
  newSession = await createSessionRecord(admin.admin_id, client, now);
  return "ok";
});
```

อ่านโค้ดนี้ทีละบรรทัด

- บรรทัด 1: เปิด transaction ทุกอย่างข้างในสำเร็จพร้อมกันหรือไม่เกิดเลย
- บรรทัด 3–4: หาบัญชีจาก email ไม่เจอ → ผิด (ไม่นับครั้ง เพราะไม่มีบัญชีให้นับ)
- บรรทัด 5–6: ยังอยู่ในช่วงล็อก → ตอบ locked พร้อมจำนวนนาทีที่เหลือ (ปัดขึ้น)
- บรรทัด 7–8: ล็อกหมดเวลาแล้ว → ล้างจำนวนครั้งและเวลาล็อก (Q1A_2_UNLOCK)
- บรรทัด 9–14: ตรวจรหัสผ่าน ผิด → นับเพิ่ม 1 ครั้งที่ 5 ตั้ง `locked_until` = ตอนนี้ + 15 นาที แล้วตอบ locked ทันที
- บรรทัด 16: ถูก → ล้างจำนวนครั้งที่ผิดและเวลาล็อก
- บรรทัด 17: สร้าง session ใหม่ในฐานข้อมูล

6.1 ความปลอดภัย

- **รหัสผ่าน** (`password-core.ts`)
  - ไม่เก็บรหัสจริง เก็บเป็น hash แบบ scrypt ผสมค่าสุ่ม (salt)
  - เทียบรหัสด้วยวิธีที่ใช้เวลาเท่ากันเสมอ กันการเดาจากเวลาที่ใช้
- **บัตรผ่าน (session)** (`sessions.ts`)
  - เบราว์เซอร์ถือ token สุ่ม 32 byte ใน cookie `rsl_session`

  - cookie อ่านด้วย JavaScript ไม่ได้ (httpOnly)
  - ฐานข้อมูลเก็บแค่ hash ของ token ต่อให้ข้อมูลรั่วก็เอาไปใช้ไม่ได้
  - หมดอายุใน 8 ชั่วโมง
- **กันหน้า**: `requireSession()` ถูกเรียกใน layout และต้นทุก Server Action
- **Logout**: ทำเครื่องหมายว่า session ถูกยกเลิก แล้วลบ cookie

ระวัง

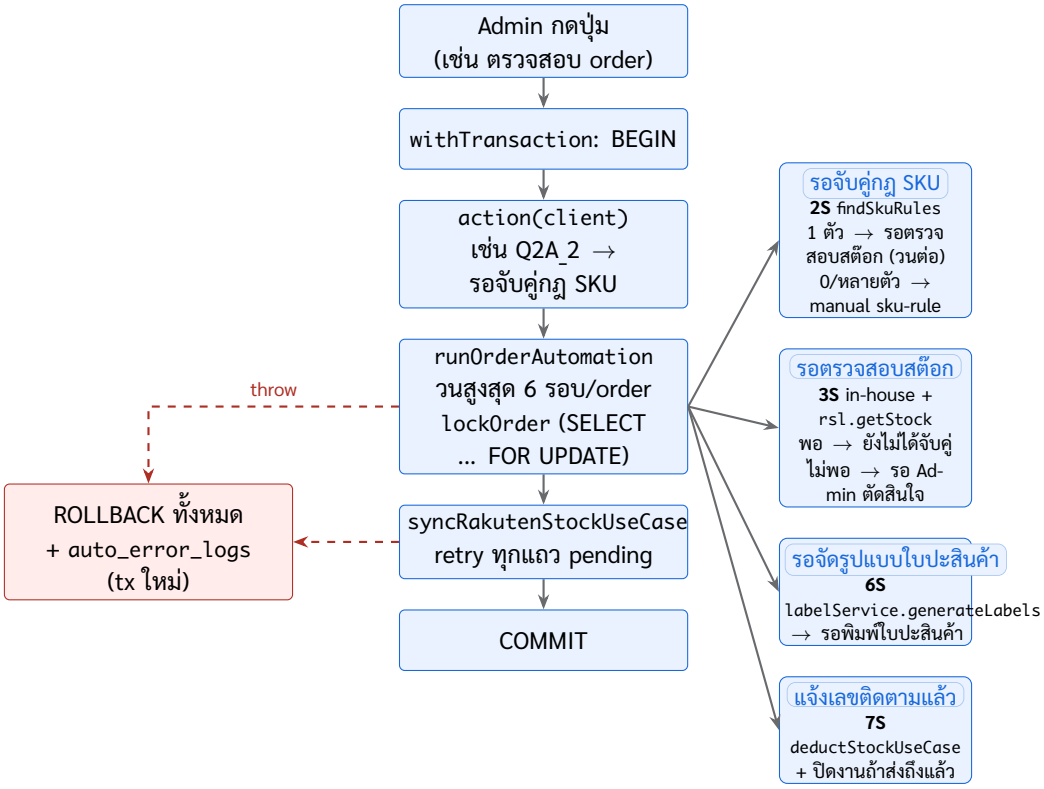
- ล็อกปลดเองเมื่อครบ 15 นาที (`LOGIN_LOCK_MINUTES`) ปุ่ม “ปลดล็อกบัญชี” ในหน้า `/dev` ปลดทุกบัญชีทันที (เครื่องมือทดสอบ ไม่ใช่ขั้นของ UC)
- ผิดครบ 5 ครั้งแล้วได้ข้อความล็อก แปลว่าบัญชีนั้นมีจริง (อีเมลที่ไม่มีได้ข้อความผิดแบบทั่วไปเสมอ)
- เลขในโค้ด (`Q1A_2_UNLOCK`, `Q1A_2`, `Q1A_3`…) ไม่ตรงกับเล่ม เล่มเรียงตามขั้นตอนเป็น Q1A.1–Q1A.7 (ปลดล็อก = Q1A.2, นับครั้งผิด = Q1A.3) ดูคอลัมน์ “ค่าคงที่ในโค้ด” ในตาราง 2.16

7 งานอัตโนมัติ (System auto) ทำงานอย่างไร

อธิบายแบบภาษาคน

1. Admin กดปุ่ม เช่น “ยืนยันคำสั่งซื้อ”
2. ระบบเปิด transaction (ก้อนงานเดียว)
3. ระบบทำงานที่ Admin สั่ง
4. ระบบดูสถานะใหม่ของ order แล้ว**ทำขั้นถัดไปที่ทำเองได้ต่อทันที** เช่น จับคู่กฎ SKU แล้วตรวจสต๊อก
5. ทำต่อไปเรื่อย ๆ จนถึงขั้นที่ต้องรอคนหรือรอระบบอื่น (สูงสุด 6 ขั้น)
6. เก็บตก order อื่นที่ค้าง: 2S ที่อ่านกฎไม่ได้ และ order แจ้งเลขติดตามแล้ว ที่ยังตัดสต๊อกไม่ครบ (Q7S_1_RETRY ครั้งละไม่เกิน 100 ข้ามแถวที่ถูกจอง) ทำ 7S ให้ และปิดงานถ้าขนส่งยืนยันแล้ว
7. ส่งยอดสต๊อกที่ค้างไป Rakuten
8. ทุกขั้นสำเร็จ → บันทึกทั้งก้อน
9. มีขั้นไหนพัง → ยกเลิกทั้งก้อน รวมถึงสิ่งที่ Admin เพิ่งกด แล้วบันทึก error แยกไว้

ผลคือ: ข้อมูลไม่มีวันค้างครึ่งทาง เช่น ไม่มีกรณีที่ตัดสต๊อกแล้วแต่สถานะไม่เปลี่ยน



7.1 ปุ่มไหนพางานอัตโนมัติอะไรตามมา

Admin กด	ระบบทำต่อเอง
UC 2A ยืนยันคำสั่งซื้อ	UC 2S จับคู่กฎ SKU → UC 3S ตรวจสต๊อก
UC 1S จับคู่ RSL	UC 6S สร้างใบปะ
UC 5S ส่งเลขติดตาม	UC 7S ตัดสต๊อก → ส่งยอดไป Rakuten → ปิดงานถ้าส่งถึงแล้ว
UC 3A บันทึก/นำเข้าสินค้า	order ที่ค้างเพราะกฎ SKU → UC 2S → UC 3S (→ UC 6S ถ้าส่งจากคลังบริษัท)
UC 6A รับสินค้า	เพิ่มสต๊อกคลังบริษัท → order ที่รอใบสั่งซื้อนี้ → UC 3S (→ UC 6S) → ส่งยอดไป Rakuten
ปรับสต๊อก	ส่งยอดไป Rakuten
ทุกปุ่มที่ผ่านระบบนี้	ลอง 2S ที่ค้าง, ลอง 7S กับ order ที่ยังตัดสต๊อกไม่ครบ, ลองส่งยอดสต๊อกที่ค้างอยู่ไป Rakuten อีกรอบ

งานที่ทำตอนเปิดหน้า

- เปิดหน้า `/shipping/label` → สร้างใบปะที่ยังค้าง (6S)
- เปิดหน้า `/shipping` → ถามขนส่งว่าพัสดุถึงไหนแล้ว (8A) แต่ละ order บันทึกผ่าน `executeWorkflow` ได้ `delivered` → ทำ 7S ที่ค้างแล้วปิดงานใน transaction เดียวกัน

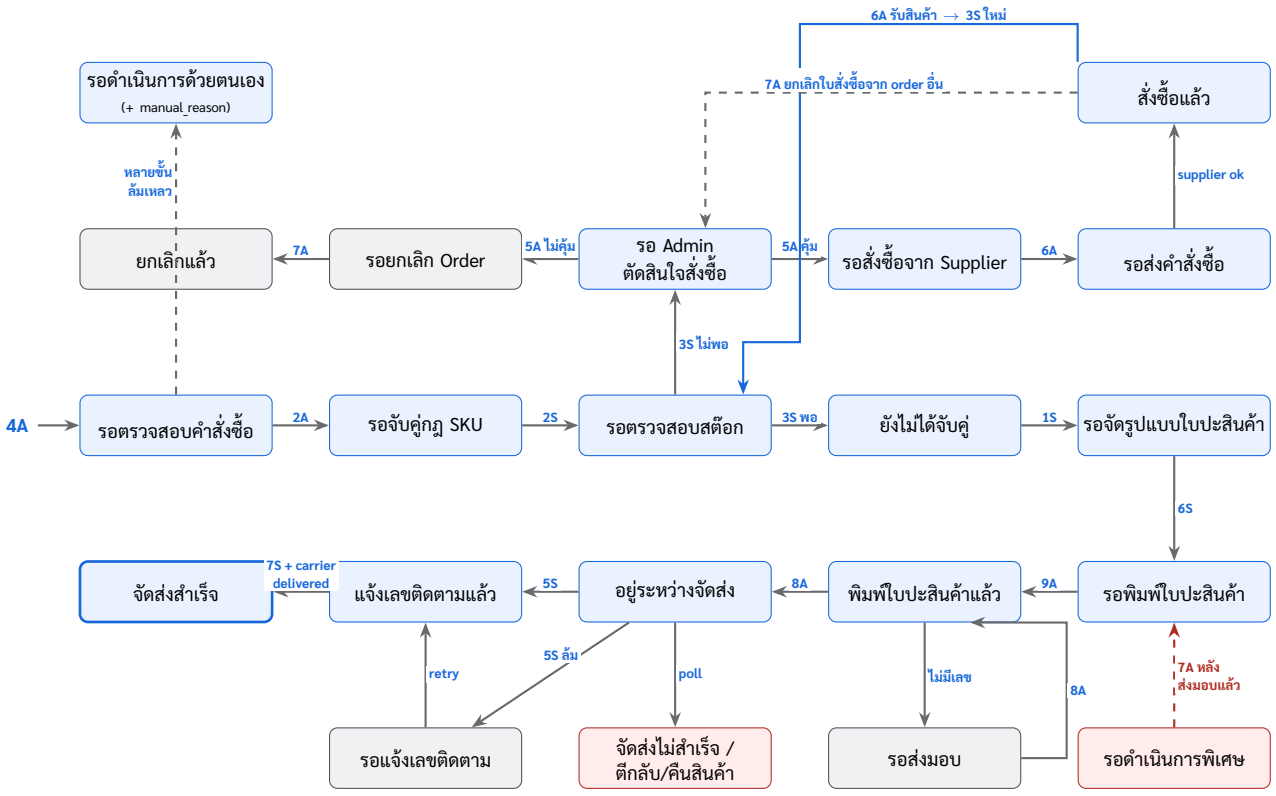
src/lib/use-cases/execute-workflow.ts:290

```
export async function executeWorkflow<T>(action, options = {}) {
  try {
    return await withTransaction(async (client) => {
      const result = await action(client);
      await runAutomation(client, options.orderIds ?? [], ...);
      return result;
    });
  } catch (error) {
    if (error instanceof AutomationStepError) {
      await recordRolledBackAutomationError(error);
      throw error.original;
    }
    throw error;
  }
}
```

อ่านโค้ดนี้ทีละบรรทัด

- บรรทัด 1: รับ “งานของ Admin” (`action`) และรายการ order ที่เกี่ยวข้อง
- บรรทัด 3: เปิด transaction
- บรรทัด 4: ทำงานของ Admin
- บรรทัด 5: ทำงานอัตโนมัติต่อ (ในก้อนเดียวกัน)
- บรรทัด 8: ถ้ามีอะไรพัง ทั้งก้อนถูกยกเลิกไปแล้ว
- บรรทัด 9–10: ถ้าพังที่ขั้นอัตโนมัติ → บันทึกลง `auto_error_logs`
- บรรทัด 11, 13: ส่ง error กลับไปให้หน้าเว็บแจ้งผู้ใช้

7.2 เส้นทางสถานะของ order (ตามโค้ดจริง)



อ่านภาพ

- กล่องฟ้า = เส้นทางปกติ
- กล่องเทา = ทางแยก หรือรอคน/รอระบบอื่น
- กล่องแดง = ผิดปกติ
- ป้ายบนลูกศร = UC ที่ทำให้สถานะเปลี่ยน

order จะ “จัดส่งสำเร็จ” ได้ต้องครบ 3 ข้อ

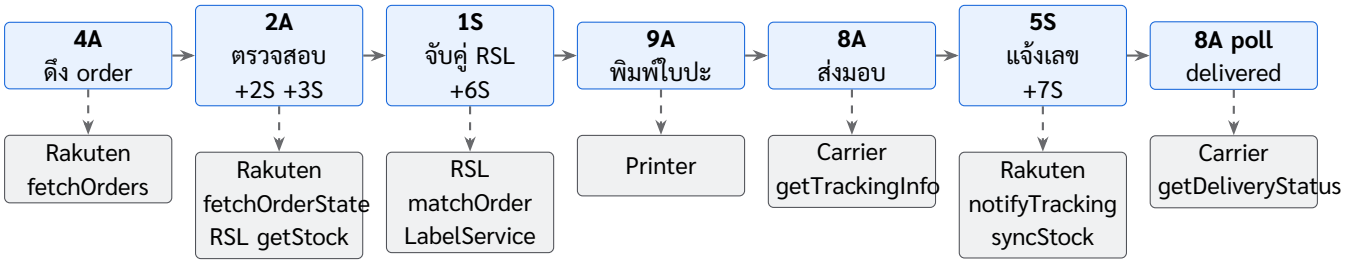
1. ขนส่งยืนยันว่าส่งถึงแล้ว (`delivery_status = 'delivered'`)
2. ส่งเลขติดตามให้ลูกค้าแล้ว
3. ตัดสต๊อกครบทุกรายการแล้ว

- ไม่ว่าข้อไหนเกิดก่อน ระบบรอจนครบ 3 ข้อแล้วค่อยเปลี่ยนสถานะ
- ต่างจากรายงานที่ให้ 7S เปลี่ยนเป็นจัดส่งสำเร็จทันที

8 Flow รายUse Case

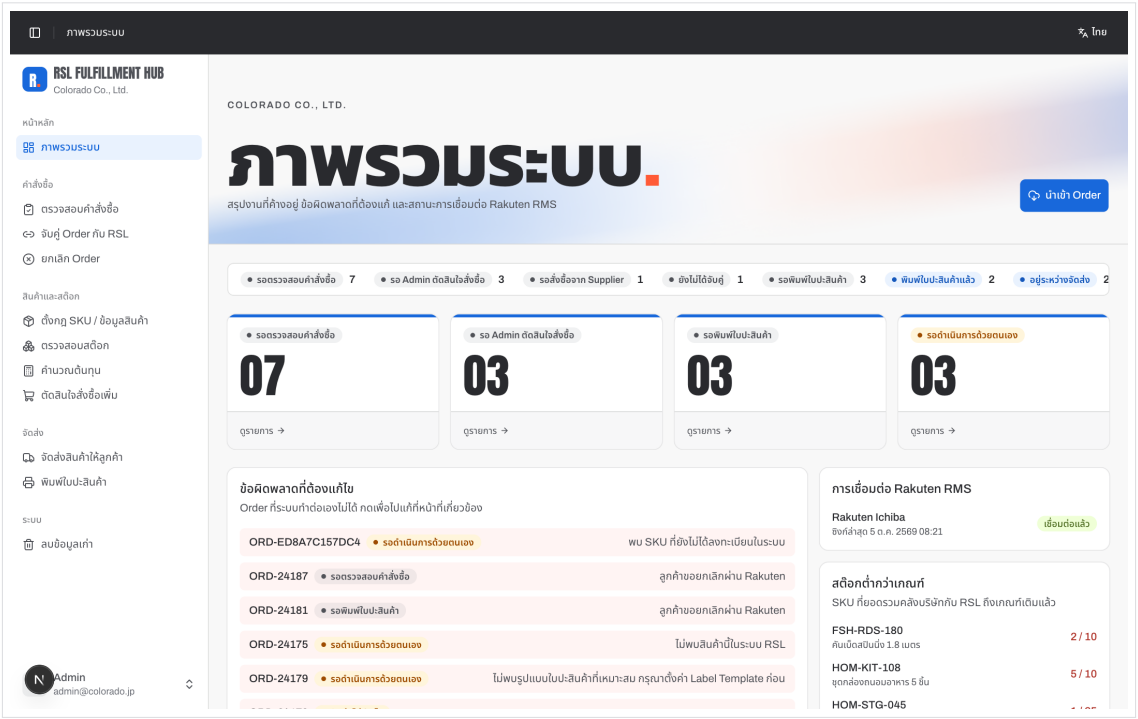
- เรียงตาม business process To-Be
- แต่ละ UC มี
  1. ภาพหน้าจอ
  2. **เกิดอะไรขึ้น** เป็นภาษาคน
  3. **ข้อมูลอ้างอิง** ชื่อ action, use-case, query, ตาราง, adapter (ไว้แก้รายงาน)

8.1 Flow A: order ปกติ ตั้งแต่ดึงเข้าจนส่งถึงลูกค้า



- กล่องบน = ขั้นของเรา, กล่องล่าง = ระบบภายนอกที่ถูกเรียก

UC 4A ดึงคำสั่งซื้อจาก Rakuten



ภาพ: หน้า /dashboard มีปุ่มดึง order

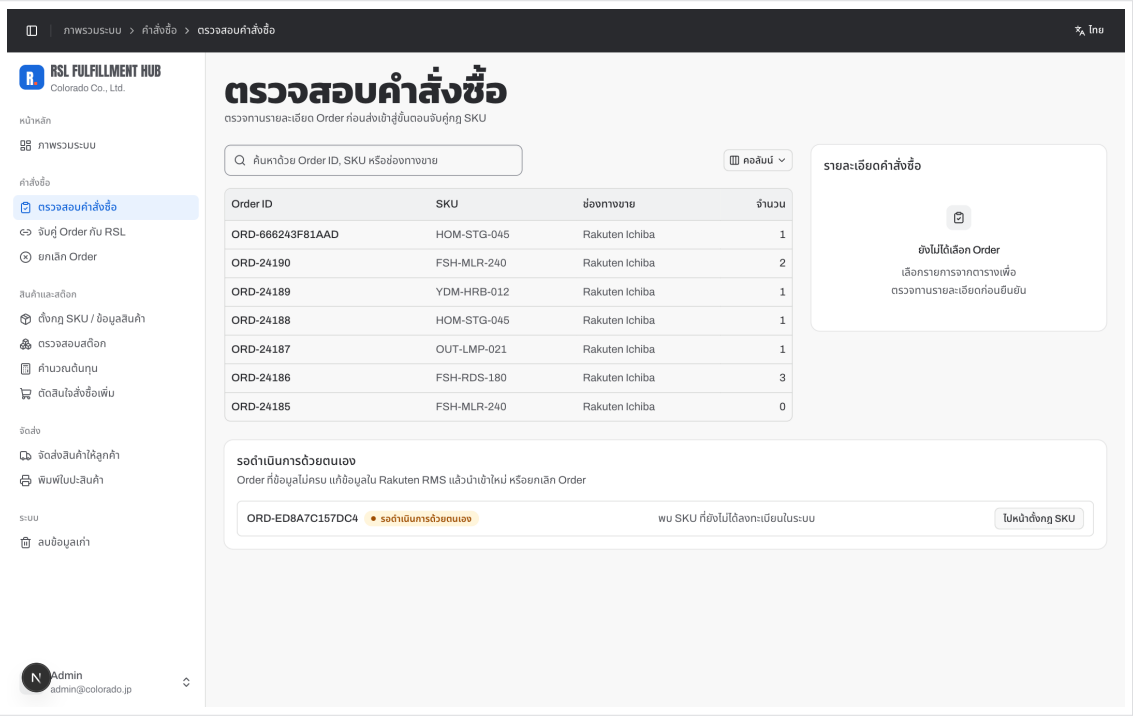
เกิดอะไรขึ้น

1. Admin กดปุ่มนำเข้าบน dashboard
2. ระบบขอ order ใหม่จาก Rakuten
3. order ที่ข้อมูลไม่ครบ → ข้าม
4. order ที่เคยดึงแล้ว → ข้าม (เลข Rakuten ซ้ำไม่ได้)
5. order ใหม่ → บันทึกเป็น `รอตรวจสอบคำสั่งซื้อ`
6. ถ้าไม่รู้จักสินค้าใน order → `รอดำเนินการด้วยตนเอง` (เหตุผล: ยังไม่มีสินค้า)
7. ถ้าลูกค้าขอยกเลิกบน Rakuten → ติดธงไว้ ไปจัดการตอนตรวจสอบ

ข้อมูลอ้างอิง

Action	`importOrdersAction`
Use-case	`importOrdersUseCase` (`import-orders.ts`) ไม่มีงานอัตโนมัติต่อ
Adapter	`rakuten.fetchOrders()`
SQL	`Q4A_1`, `Q4A_1_MARK_CANCELLATION`
หมายเหตุ	ตอนนำเข้าตั้ง `fulfill_source='rsl'` ไว้ก่อน 3S เป็นตัวเลือกคลังจริง (RSL พอ → rsl, ไม่พอแต่คลังบริษัทพอ → in_house)

UC 2A ตรวจสอบคำสั่งซื้อ (+ UC 2S UC 3S อัตโนมัติ)



ภาพ: หน้า /orders/verify

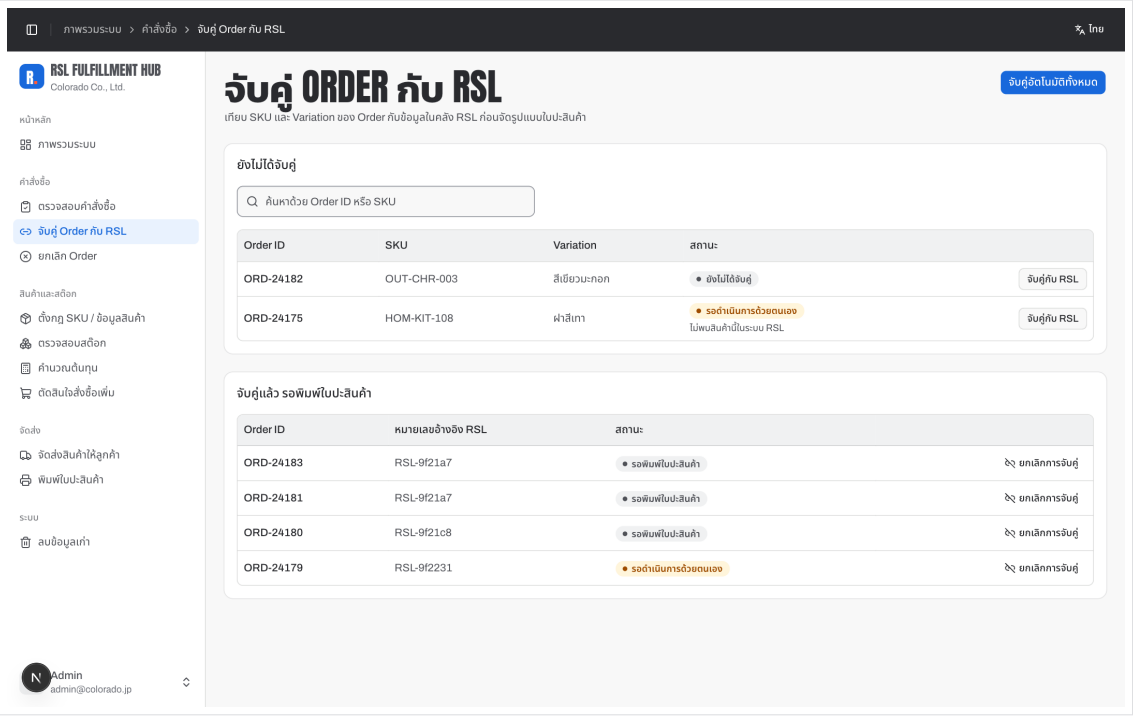
เกิดอะไรขึ้น

1. Admin เลือก order แล้วกด “ยืนยันคำสั่งซื้อ” (แก้วิธีจัดส่งได้)
2. ระบบดึงข้อมูลล่าสุดจาก Rakuten มาเทียบ
3. ลูกค้ายกเลิกแล้ว → `ยกเลิกแล้ว` จบ
4. ข้อมูลไม่ครบ → `รอดำเนินการด้วยตนเอง`
5. ผ่าน → `รอจับคู่กฎ SKU` และบันทึกประวัติถ้ามีการแก้
6. **2S อัตโนมัติ**: หาสินค้าที่รหัส Rakuten ตรงกัน
   - เจอ 1 ตัวพอดี → `รอตรวจสอบสต๊อก`
   - ไม่เจอ หรือเจอหลายตัว → `รอดำเนินการด้วยตนเอง`
7. **3S อัตโนมัติ**: รวมสต๊อกคลังเอง + คลัง RSL (ถาม RSL)
   - พอ → `ยังไม่ได้จับคู่` (ไปขั้นจับคู่ RSL)
   - ไม่พอ → `รอ Admin ตัดสินใจสั่งซื้อ` (ไป Flow B)
   - RSL ล่ม → ถือว่าคลัง RSL ไม่มีของ และบันทึก error

ข้อมูลอ้างอิง

Action	`verifyOrderAction`
Use-case	`verifyOrderUseCase` (`verify-order.ts`) ผ่าน `executeWorkflow`
Adapter	`rakuten.fetchOrderState()`, `rsl.getStock()`
SQL	Q2A_2; 2S: Q2S_1, Q2S_2; 3S: Q3S_1_ORDER, Q3S_1_ROUTE

UC 1S จับคู่ Order กับ RSL (+ UC 6S สร้างใบปะ)



ภาพ: หน้า /orders/rsl-match

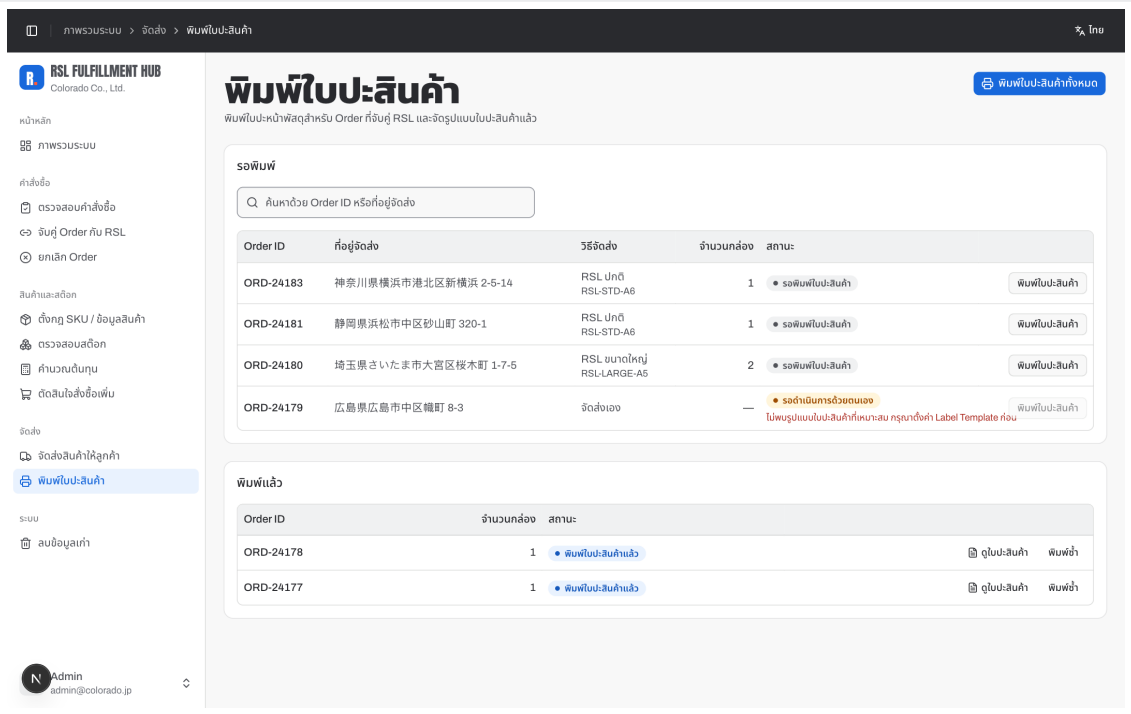
เกิดอะไรขึ้น

1. Admin กดจับคู่ทีละ order หรือทั้งหมด
2. ระบบถาม RSL ว่า order นี้ตรงกับรายการไหนในคลัง RSL
3. เจอหลายรายการ → ให้ Admin เลือกเอง
4. ไม่เจอ → `รอดำเนินการด้วยตนเอง`
5. เจอ 1 รายการ → บันทึกเลข RSL → `รอจัดรูปแบบใบปะสินค้า`
6. **6S อัตโนมัติ**: สร้างใบปะ แบ่งกล่องตามจำนวนสินค้าต่อกล่อง → `รอพิมพ์ใบปะสินค้า`
   - ไม่มีที่อยู่ หรือไม่มีแม่แบบใบปะ → `รอดำเนินการด้วยตนเอง`
7. ยกเลิกการจับคู่ได้ ระบบลบใบปะที่สร้างไว้ → กลับเป็น `ยังไม่ได้จับคู่`

ข้อมูลอ้างอิง

Action	`matchRslAction`, `matchAllRslAction`, `unmatchRslAction`
Use-case	`matchRslUseCase` (`match-order.ts`)
Adapter	`rsl.matchOrder()`, `labelService.generateLabels()`
SQL	Q1S_1, Q1S_1_UNMATCH; 6S: Q6S_2_INSERT, Q6S_2_ADVANCE
หมายเหตุ	แม่แบบใบปะอยู่ใน `src/mock/delivery.ts` ไม่ได้อยู่ในฐานข้อมูล

UC 9A พิมพ์ใบปะสินค้า



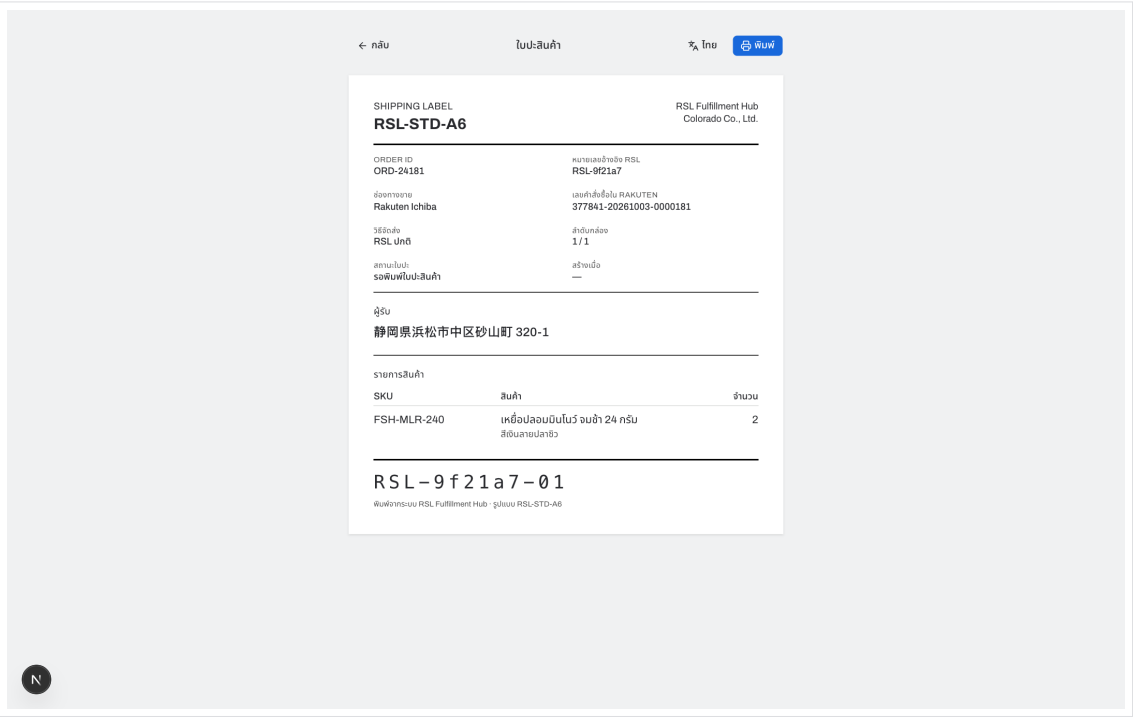
ภาพ: หน้า /shipping/label

เกิดอะไรขึ้น

1. เปิดหน้านี้ → ระบบสร้างใบปะที่ยังค้างให้ก่อน (6S)
2. Admin เลือก order แล้วกดพิมพ์
3. ระบบส่งใบปะไปเครื่องพิมพ์ทีละกล่อง กล่องไหนพิมพ์ไม่ได้ → หยุด
4. ทุกครั้งที่สั่งพิมพ์ ระบบบันทึกประวัติ
5. พิมพ์ครบทุกกล่อง → พิมพ์ใบปะสินค้าแล้ว
6. พิมพ์ซ้ำได้ บันทึกประวัติแต่ไม่เปลี่ยนสถานะ

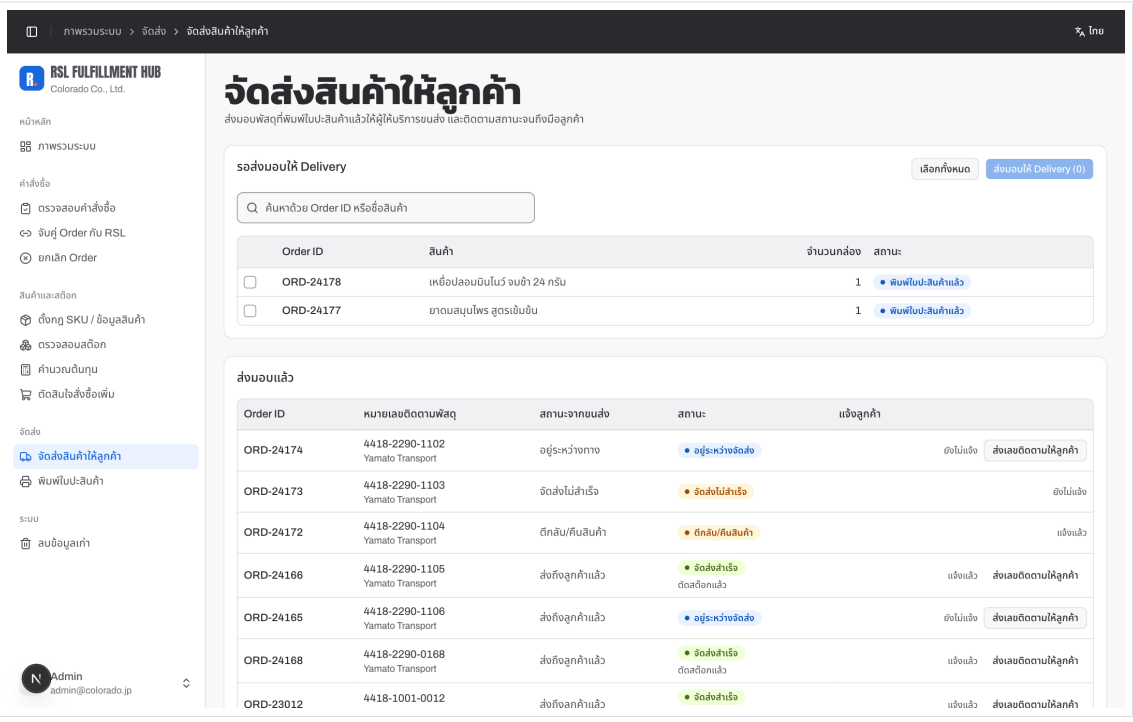
ข้อมูลอ้างอิง

Action	`formatLabelsAction`, `printLabelsAction`
Use-case	`formatLabelsUseCase`, `printLabelsUseCase` (`print-label.ts`)
Adapter	`printer.printLabel()`
SQL	`Q9A_1_PRINT`, `Q9A_2_HISTORY`, `Q9A_2_COMPLETE`



ภาพ: หน้าพิมพ์ใบปะ /reports/label/[orderId]

UC 8A ส่งมอบให้ขนส่ง และติดตามพัสดุ



ภาพ: หน้า /shipping

เกิดอะไรขึ้นตอนส่งมอบ

1. Admin เลือก order ที่พิมพ์ใบปะแล้ว กด “ส่งมอบให้ Delivery”
2. ระบบขอเลขติดตามพัสดุจากขนส่ง
3. ยังไม่ได้เลข → `รอส่งมอบ` (ลองใหม่ทีหลัง)
4. ได้เลข → บันทึกเลขและชื่อขนส่ง → `อยู่ระหว่างจัดส่ง`

เกิดอะไรขึ้นตอนเปิดหน้านี้ (ติดตามพัสดุ)

1. ระบบถามขนส่งสถานะของพัสดุทุกชิ้นที่มีเลขติดตาม
2. ส่งไม่สำเร็จ → `จัดส่งไม่สำเร็จ`
3. ตีกลับ → `ตีกลับ/คืนสินค้า` และคืนสต๊อกถ้าตัดไปแล้ว
4. ส่งถึงแล้ว → `จัดส่งสำเร็จ` เฉพาะเมื่อแจ้งเลขและตัดสต๊อกแล้ว ไม่งั้นจำไว้ก่อน

ข้อมูลอ้างอิง

Action	`dispatchOrdersAction`, `pollCarrierAction`
Use-case	`dispatchOrdersUseCase`, `pollCarrierUseCase`
Adapter	`carrier.getTrackingInfo()`, `carrier.getDeliveryStatus()`
SQL	`Q8A_1_SAVE`, `Q8A_1_WAIT`, `Q8A_1_POLL`, `Q8A_1_STATUS`, `Q8A_1_DELIVERED`

UC 5S ส่งเลขติดตามให้ลูกค้า (+ UC 7S ตัดสต๊อก)

เกิดอะไรขึ้น

1. Admin กด “ส่งเลขติดตามให้ลูกค้า” ในหน้า `/shipping`
2. ถ้าเคยส่งแล้ว → ถามยืนยันก่อนส่งซ้ำ
3. ระบบบันทึกว่า “กำลังส่ง” แล้วส่งเลขให้ Rakuten แจ้งลูกค้า
4. ส่งไม่ได้ → `รอแจ้งเลขติดตาม` (กดส่งใหม่ได้ ใช้รหัสคำขอเดิม)
5. ส่งได้ → `แจ้งเลขติดตามแล้ว`
6. **7S อัตโนมัติ**: ตัดสต๊อก (ดูขั้นตอนด้านล่าง)
7. ส่งยอดสต๊อกใหม่ไป Rakuten
8. ถ้าขนส่งยืนยันว่าถึงแล้ว **และ**ตัดสต๊อกครบทุกรายการของ order → `จัดส่งสำเร็จ`

แก้โค้ดแล้ว (7 พ.ย.)

- แก้แล้ว (commit `d30bde2`, PR #28): `Q5S_2` ในโค้ด (= `Q5S.4` ในเล่ม) เดิมปิดงานเมื่อ**มีรายการใดรายการหนึ่ง**ตัดสต๊อกแล้ว
- ตอนนี้ปิดงานเมื่อ**ทุกรายการ**ตัดแล้ว เหมือน `Q8A.4`, `Q8A.5`, `Q7S.7`

เงื่อนไขปิดงานใน `Q5S_2` (ส่วนที่แก้)

```
WHEN $2 = 'sent' AND delivery_status = 'delivered'
  AND EXISTS (SELECT 1 FROM order_items WHERE order_id = o.order_id)
  AND NOT EXISTS (
    SELECT 1 FROM order_items
    WHERE order_id = o.order_id AND stock_deducted = FALSE
  ) THEN 'จัดส่งสำเร็จ'
```

- บรรทัด 1: ส่งเลขให้ลูกค้าสำเร็จ และขนส่งยืนยันว่าถึงแล้ว
- บรรทัด 2: order นี้ต้องมีรายการสินค้าอย่างน้อย 1 รายการ
- บรรทัด 3–6: ต้อง**ไม่มี**รายการไหนที่ยังไม่ได้ตัดสต๊อก

7S ตัดสต๊อกแบบไม่ตัดซ้ำ

1. จองแถวสต๊อกไว้ คนอื่นต้องรอ
2. ถ้าตัดไปแล้ว → ไม่ทำอะไร
3. ถ้าคลังไหนของไม่พอ → ไม่ตัดเลยสักชิ้น → รอดำเนินการด้วยตนเอง
4. ตัดจำนวนออกจากคลังที่สินค้าส่งออกไป
5. ติดธงว่า “ตัดแล้ว”
6. ตั้งให้ส่งยอดใหม่ไป Rakuten

- ทีมทดสอบแล้ว: กดส่ง order เดียวกันพร้อมกัน 20 ครั้ง → ตัดสต๊อกครั้งเดียว
- order ที่ค้าง แจ้งเลขติดตามแล้ว แต่ยังตัดไม่ครบ ถูกหยิบมาทำซ้ำในปุ่มถัดไปที่ผ่าน `executeWorkflow` (`Q7S_1_RETRY`) ของไม่พอ → ไปค้างที่ รอดำเนินการด้วยตนเอง จึงไม่วนซ้ำ

src/lib/queries/stock.ts:177 (Q7S_2)

```
UPDATE stock
SET in_house_qty = CASE WHEN $2 = 'in_house' THEN in_house_qty - $3 ELSE in_house_qty END,
    rsl_qty      = CASE WHEN $2 = 'rsl' THEN rsl_qty - $3 ELSE rsl_qty END,
    updated_at = CURRENT_TIMESTAMP
WHERE stock_id = $1
  AND CASE WHEN $2 = 'in_house' THEN in_house_qty >= $3
           ELSE rsl_qty IS NOT NULL AND rsl_qty >= $3 END
```

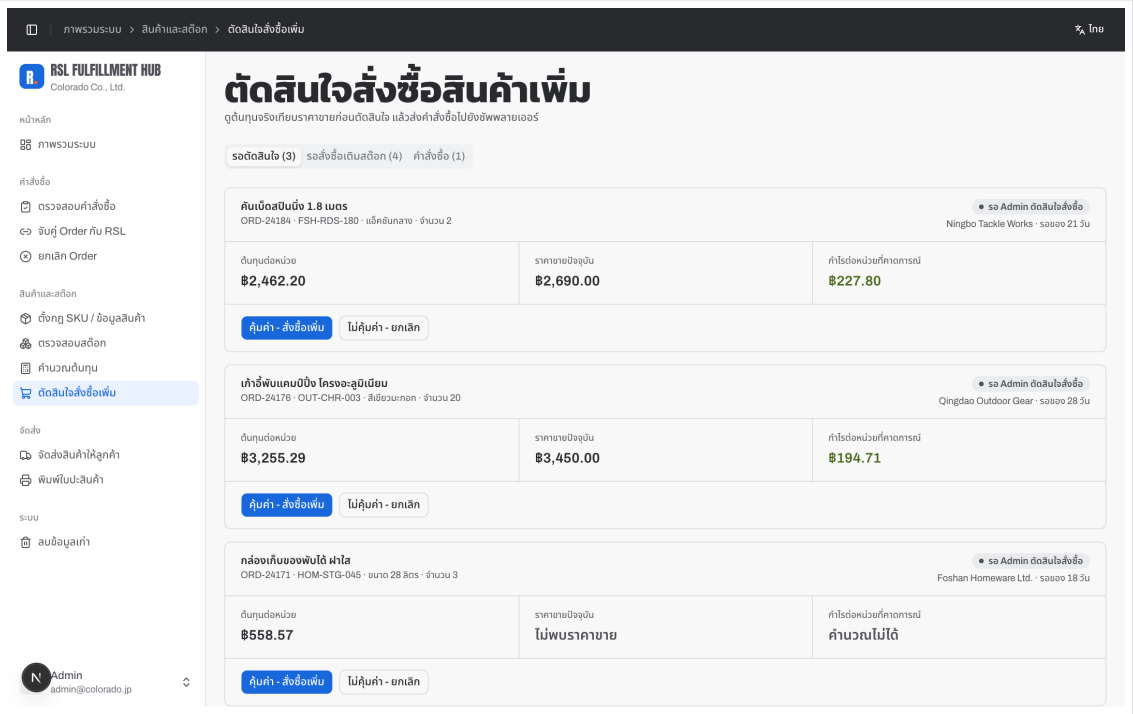
อ่าน SQL นี้ทีละบรรทัด

- $1 = รหัสสต๊อก, $2 = คลัง (in_house หรือ rsl), $3 = จำนวนที่จะตัด
- บรรทัด 1: แก้ตาราง `stock`
- บรรทัด 2: ถ้าเป็นคลังเอง → ลบจำนวนออกจาก `in_house_qty`
- บรรทัด 3: ถ้าเป็นคลัง RSL → ลบจำนวนออกจาก `rsl_qty`
- บรรทัด 4: บันทึกเวลาที่แก้
- บรรทัด 5–7: แก้เฉพาะแถวนี้ และ**เฉพาะเมื่อของพอ** ของไม่พอจะไม่มีแถวไหนถูกแก้ สต๊อกจึงไม่ติดลบ

ข้อมูลอ้างอิง

Action	`notifyTrackingAction`
Use-case	`notifyTrackingUseCase`; 7S: `deductStockUseCase`, `syncRakutenStockUseCase`
Adapter	`rakuten.notifyTracking()`, `rakuten.syncStock()`
SQL	`Q5S_1`, `Q5S_1_RETRY`, `Q5S_1_ATTEMPT`, `Q5S_2`; 7S: `Q7S_1_RETRY`, `Q7S_1`–`Q7S_4_PENDING`, `Q7S_5_*`

8.2 Flow B: สต๊อกไม่พอ → ตัดสินใจ → สั่งซื้อ



ภาพ: หน้า /products/reorder

UC 5A ตัดสินใจสั่งซื้อเพิ่ม

เกิดอะไรขึ้น

1. order ที่สต๊อกไม่พอมาอยู่ในหน้านี้
2. ระบบแสดงราคาขายและต้นทุนต่อหน่วยให้เทียบ
3. ไม่มีราคาขาย → รอดำเนินการด้วยตนเอง (ใส่ราคาในหน้าสินค้าแล้วกลับเข้าคิวเอง) order แบบนี้ยังแสดงในแท็บแต่ซ่อนปุ่มตัดสินใจ ถ้าส่งคำขอมาเองได้ “ไม่พบราคาขาย” พร้อมลิงก์ไปหน้าสินค้า
4. จำนวนที่จะสั่งไม่ใช่จำนวนเต็มบวก → “กรุณากรอกจำนวนที่จะสั่งซื้อเป็นจำนวนเต็มบวก”
5. Admin เลือก “คุ้มค่า” → รอสั่งซื้อจาก Supplier
6. Admin เลือก “ไม่คุ้มค่า” → รอยกเลิก Order

UC 6A สั่งซื้อจาก Supplier

เกิดอะไรขึ้น

1. Admin กดสั่งซื้อ
2. ไม่มี supplier หรือระยะเวลาส่งของ → รอดำเนินการด้วยตนเอง
3. มีใบสั่งซื้อค้างอยู่แล้ว → ถามยืนยันก่อน
4. ระบบสร้างใบสั่งซื้อ → order **ทุกตัว**ของ SKU นี้ที่รอสั่งซื้อ เป็น รอส่งคำสั่งซื้อ และจำเลขใบสั่งซื้อไว้ที่ `orders.reorder_id`
5. ส่งใบสั่งซื้อให้ supplier
6. supplier รับ → ใบสั่งซื้อและ order เหล่านั้นเป็น สั่งซื้อแล้ว
7. ส่งไม่ได้ → ค้างที่ รอส่งคำสั่งซื้อ กดใหม่ได้ (ใช้รหัสคำขอเดิม ไม่สั่งซ้ำ)

UC 6A รับสินค้าจาก Supplier

เกิดอะไรขึ้น

1. ของมาถึง → Admin กด “รับสินค้า” ที่ใบสั่งซื้อ `สั่งซื้อแล้ว` ในแท็บ “คำสั่งซื้อ”
2. กรอกจำนวนที่ได้รับ (ตั้งต้น = จำนวนที่สั่ง แก้ได้ถ้าได้ไม่ครบหรือเกิน)
3. ระบบจองใบสั่งซื้อและแถวสต๊อก (`Q6A_10_FOR_UPDATE`)
4. ใบสั่งซื้อเป็น `ได้รับสินค้าแล้ว` พร้อมเวลาและจำนวน (`Q6A_11_RECEIVE`) รับได้ครั้งเดียว กดซ้ำหรือกดพร้อมกันไม่บวกสต๊อกซ้ำ
5. บวกจำนวนเข้าคลังบริษัท (`in_house_qty`) บันทึกประวัติปรับสต๊อก และตั้งให้ส่งยอดไป Rakuten
6. order ที่ผูกกับใบสั่งซื้อนี้กลับไป `รอตรวจสอบสต๊อก` (`Q6A_12_RELEASE_ORDERS`) แล้ว 3S ตรวจใหม่ทันทีในก้อนเดียวกัน

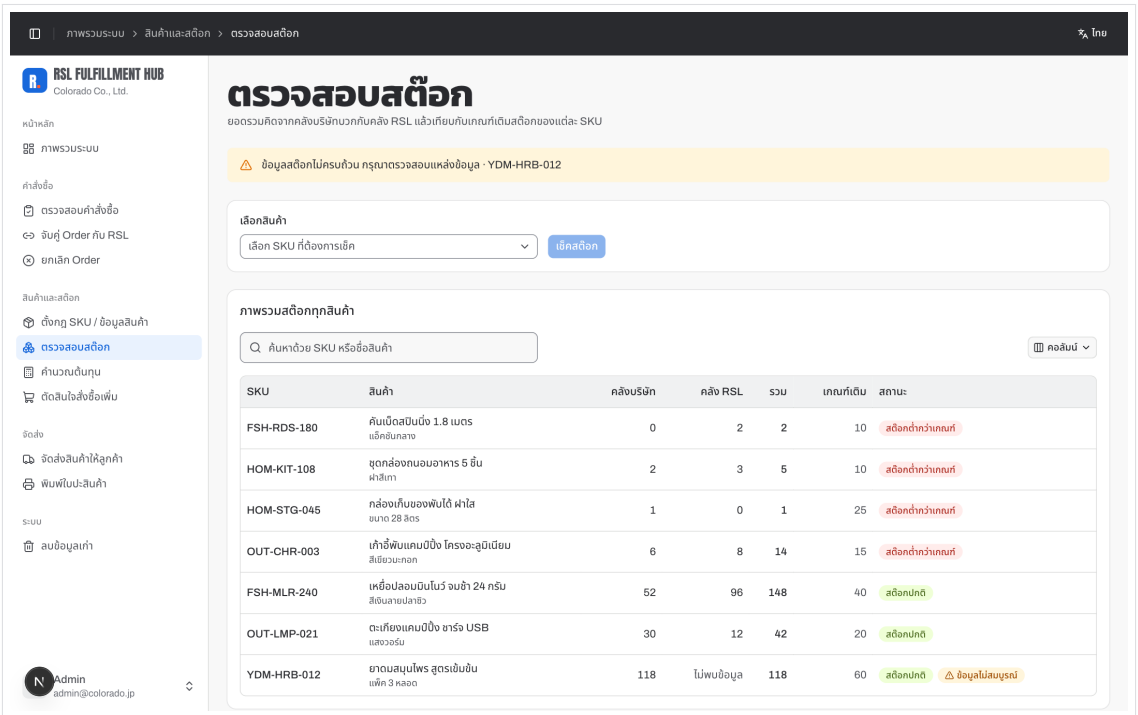
ข้อมูลอ้างอิง (5A + 6A)

Action	`decideReorderAction`, `purchaseReorderAction`
Use-case	`decideReorderUseCase`, `purchaseReorderUseCase` (`reorders.ts`)
Adapter	`supplier.sendPurchaseOrder()`
SQL	`Q5A_1_FOR_UPDATE`, `Q5A_4`, `Q6A_2_FOR_UPDATE`, `Q6A_1_PENDING`, `Q6A_1_RETRY`, `Q6A_1_NEXT_NUMBER`, `Q6A_1_INSERT`, `Q6A_1_MARK_ORDERS`, `Q6A_1_COMPLETE`; รับสินค้า: `Q6A_10_FOR_UPDATE`, `Q6A_11_RECEIVE`, `Q3S_1_ADJUST`, `Q3S_1_INSERT_HISTORY`, `Q6A_12_RELEASE_ORDERS`

ระวัง

- รับของได้ครั้งเดียวต่อใบสั่งซื้อ ได้ไม่ครบก็ปิดใบ ถ้าสต๊อกยังไม่พอ 3S ส่ง order กลับไปให้ 5A ตัดสินใจใหม่
- ของเข้าคลังบริษัทเสมอ ไม่เข้าคลัง RSL

8.3 UC 3S ตรวจสอบและปรับสต๊อกด้วยมือ



ภาพ: หน้า /products/stock

เกิดอะไรขึ้น

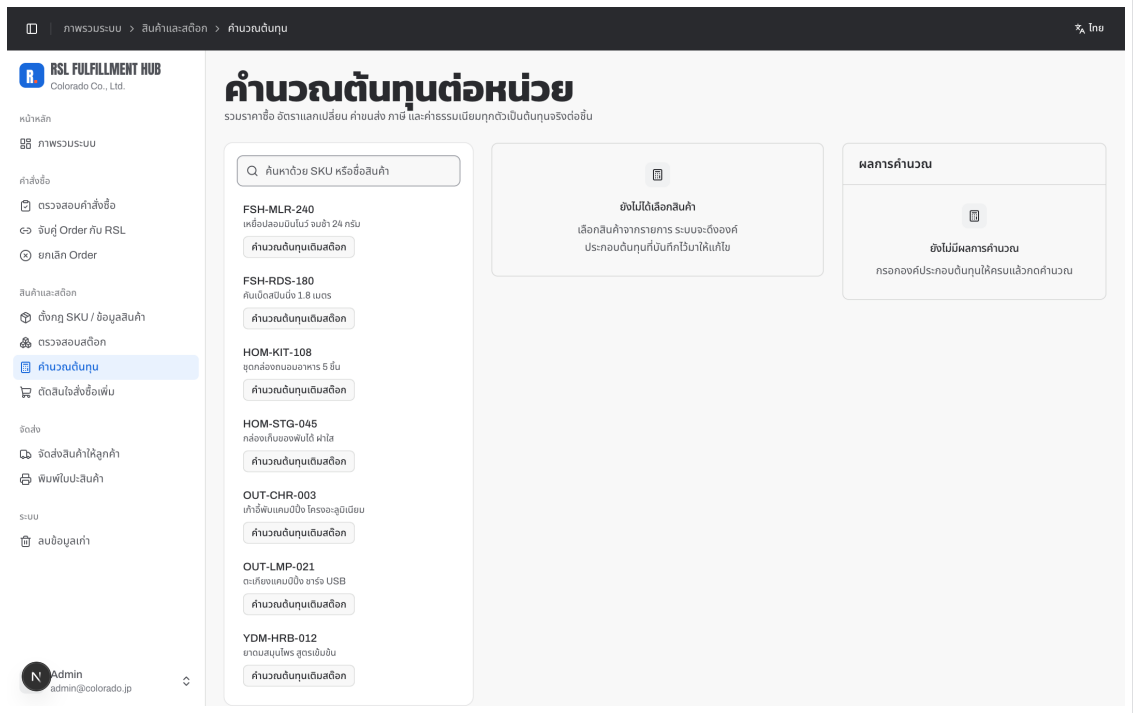
1. Admin แก้จำนวนสต๊อกและใส่เหตุผล
2. ระบบบันทึกค่าใหม่และประวัติการปรับ
3. ระบบส่งยอดใหม่ไป Rakuten

- ป้าย “สต๊อกต่ำ” คำนวณบนหน้าจอจากจุดสั่งซื้อซ้ำ **ไม่ได้สั่งซื้อให้อัตโนมัติ**

ข้อมูลอ้างอิง

Action	`adjustStockAction` → `adjustStockUseCase` (`stock.ts`)
SQL	`Q3S_1_FOR_UPDATE`, `Q3S_1_ADJUST`, `Q3S_1_INSERT_HISTORY`

8.4 Flow C: UC 4S คำนวณต้นทุนต่อหน่วย



ภาพ: หน้า /products/cost

เกิดอะไรขึ้น

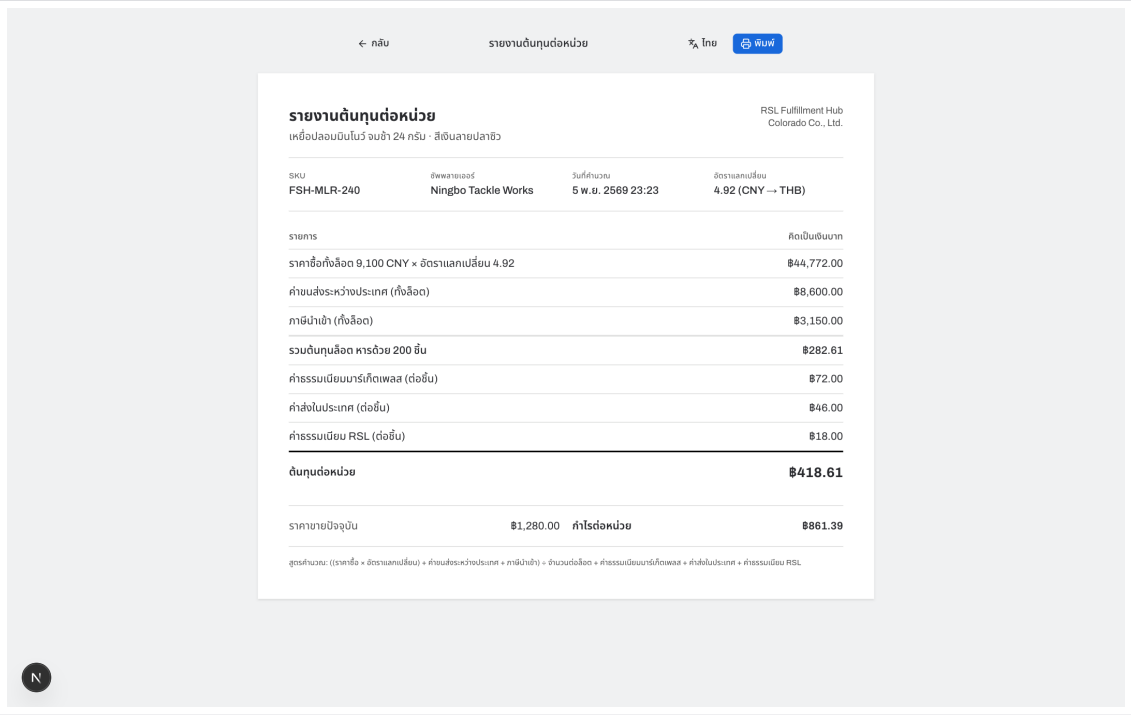
1. Admin กรอกราคาซื้อ อัตราแลกเปลี่ยน ค่าขนส่ง ค่าภาษี ค่าธรรมเนียม
2. ไม่มีอัตราแลกเปลี่ยน → แจ้งให้กรอก
3. ระบบคำนวณต้นทุนต่อหน่วย
4. ติ๊ก “จำค่า” → บันทึกเป็นค่าล่าสุดของสินค้า
5. บันทึกประวัติทุกครั้ง
6. ต้นทุนเกินราคาขาย → แสดงเตือน

สูตร (`calcUnitCost` ตรงกับรายงาน)

$$\text{ต้นทุนต่อหน่วย} = \frac{\text{ราคาซื้อ} \times \text{เรต} + \text{ขนส่งระหว่างประเทศ} + \text{ภาษี}}{\text{จำนวนที่สั่ง}} + \text{ค่าธรรมเนียม} + \text{ขนส่งในประเทศ} + \text{ค่า RSL}$$

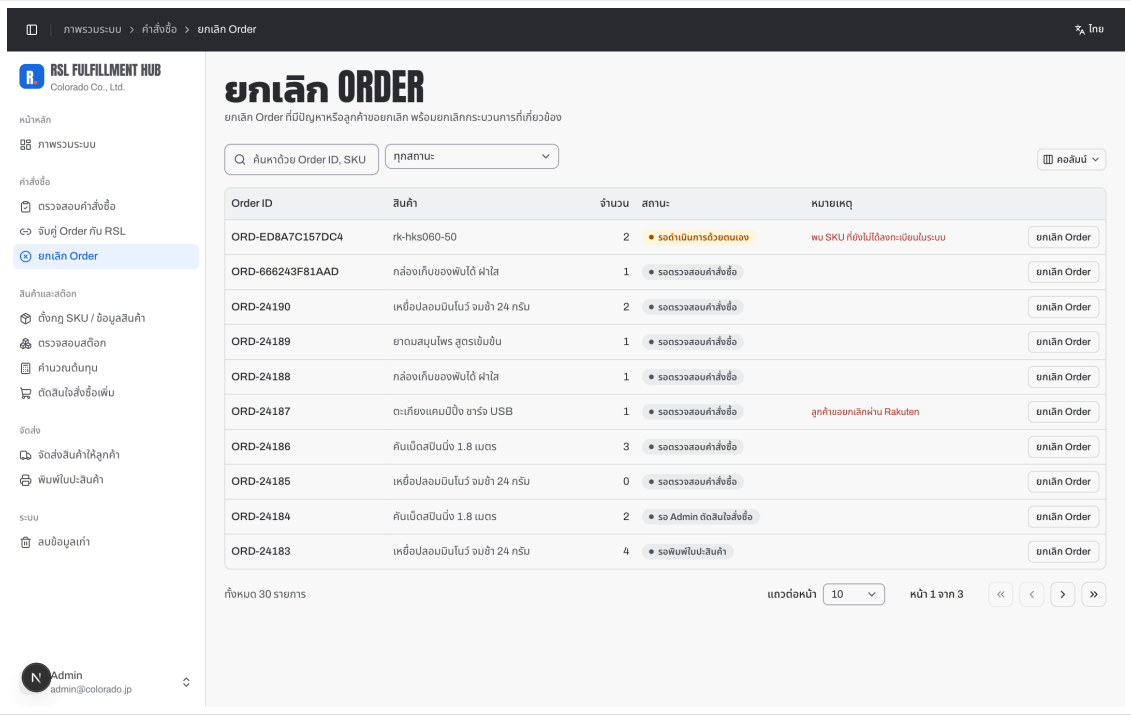
ข้อมูลอ้างอิง

Action	`calculateCostAction` → `calculateCostUseCase` (`costs.ts`)
SQL	`Q4S_1_FOR_UPDATE`, `Q4S_2_SAVE`, `Q4S_2_INSERT_HISTORY`



ภาพ: รายงานต้นทุน /reports/unit-cost/[sku]

8.5 Flow D: UC 7A ยกเลิกคำสั่งซื้อ



ภาพ: หน้า /orders/cancel

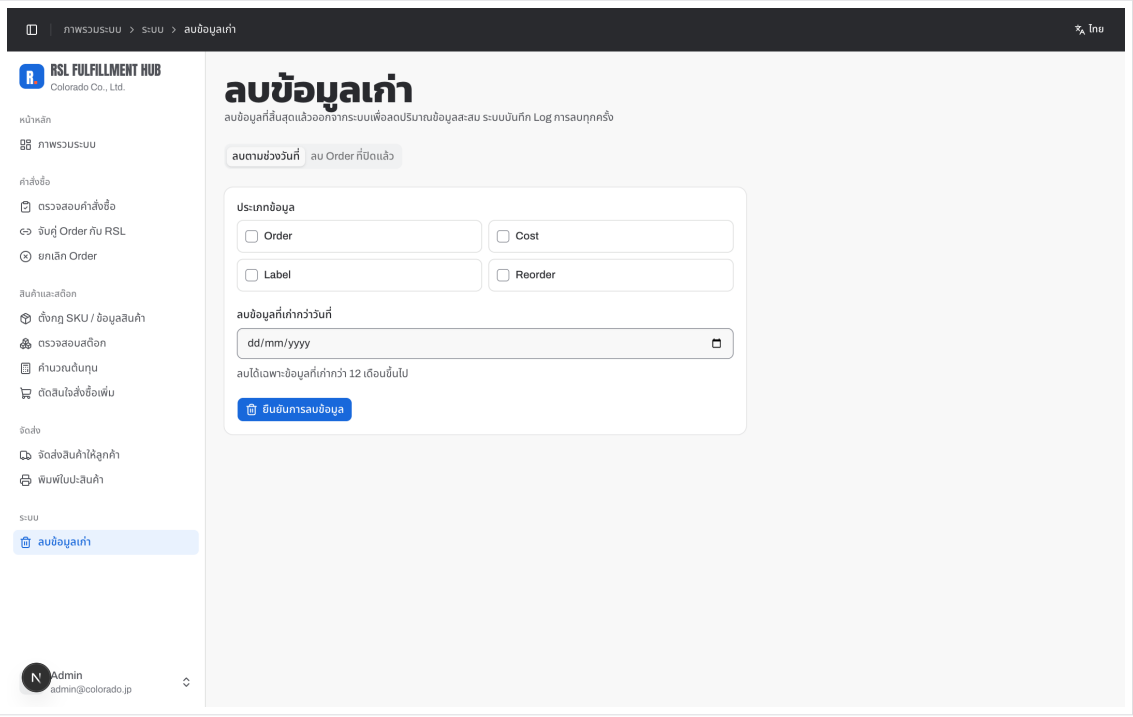
เกิดอะไรขึ้น

1. Admin เลือก order ใส่เหตุผล แล้วกดยกเลิก
2. จัดส่งสำเร็จ แล้ว → ยกเลิกไม่ได้
3. ยกเลิกไปแล้ว → ไม่ทำอะไร
4. ส่งให้ขนส่งไปแล้ว → รอดำเนินการพิเศษ (ต้องคุยกับขนส่งเอง)
5. ยังไม่ได้ส่ง → คืนสต๊อกถ้าตัดไปแล้ว
6. ยกเลิกใบสั่งซื้อที่เกี่ยวข้องได้ (ถ้าเลือก) เฉพาะใบที่ยังไม่ได้รับสินค้า order อื่นที่รอใบเดียวกันกลับไป รอ Admin ตัดสินใจสั่งซื้อ ให้ 5A ตัดสินใจใหม่
7. order เป็น ยกเลิกแล้ว พร้อมเหตุผลและเวลา

ข้อมูลอ้างอิง

Action	`cancelOrderAction` → `cancelOrderUseCase` (`cancel-order.ts`)
SQL	Q7A_2, Q7A_1_CANCEL_REORDER, Q7A_1_RELEASE_ORDERS, Q7A_1

8.6 Flow E: UC 10A ลบข้อมูลเก่า



ภาพ: หน้า /system/cleanup

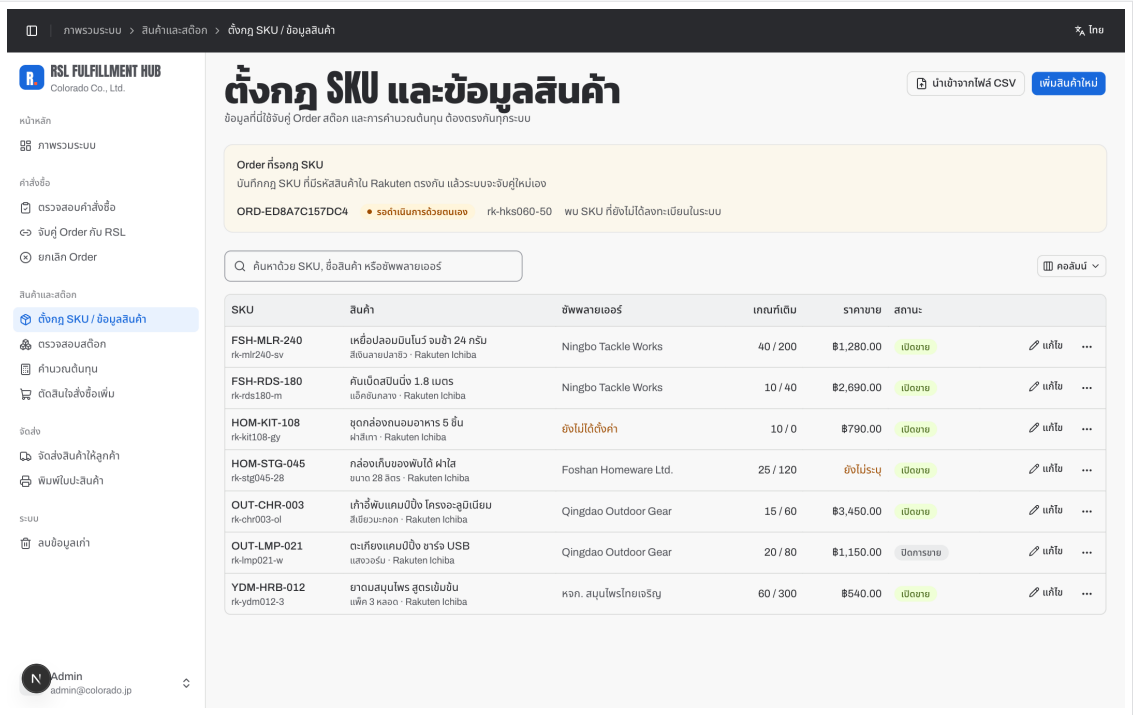
เกิดอะไรขึ้น

1. Admin เลือกช่วงเวลา หรือเลือกลบ order ที่ปิดงานแล้ว
2. หน้าจอแสดงรายการที่จะลบ (ดาวน์โหลด CSV ได้)
3. Admin กดยืนยัน
4. server ตรวจซ้ำ: ลบได้เฉพาะ จัดส่งสำเร็จ และ ยกเลิกแล้ว
5. เก็บสำเนา order ไว้ใน `deletion_history` ก่อนลบ
6. ลบ order พร้อมรายการสินค้าและใบปะ, ใบสั่งซื้อเก่า (ยกเลิกแล้ว / ได้รับสินค้าแล้ว / เลยวันรับของ และไม่มี order ที่ยังไม่ปิดผูกอยู่ผ่าน `orders.reorder_id`), ประวัติต้นทุนเก่า
7. พังกลางทาง → ยกเลิกทั้งหมด ไม่มีอะไรถูกลบ

ข้อมูลอ้างอิง

Action	`cleanupOrdersAction` → `cleanupOrdersUseCase` (`cleanup-orders.ts`)
SQL	`Q10A_5_DELETE_ORDER`, `Q10A_3`, `Q10A_4`, `Q10A_2`, `Q10A_6`

8.7 Flow F: UC 3A ตั้งกฎ SKU / ข้อมูลสินค้า



ภาพ: หน้า /products

เกิดอะไรขึ้น

1. Admin เพิ่ม/แก้สินค้า หรืออัปโหลด CSV
2. ระบบตรวจข้อมูล: ช่องว่าง, ตัวเลขผิด, รหัสซ้ำ, supplier ไม่มีจริง
3. ผิด → ปฏิเสธพร้อมบอกเหตุผล (CSV ผิดแถวเดียว ไม่บันทึกทั้งไฟล์)
4. ถูก → บันทึกสินค้า (สินค้าใหม่ได้สต๊อกเริ่มต้น 0)
5. ระบบดึง order ที่ค้างเพราะสินค้านี้กลับเข้าคิวเอง
   - ค้างเพราะไม่มีกฎ SKU → รอจับคู่กฎ SKU
   - ค้างเพราะยังไม่มีสินค้า → รอตรวจสอบคำสั่งซื้อ
   - ค้างเพราะไม่มีราคา → รอ Admin ตัดสินใจสั่งซื้อ
6. order ที่ค้างเพราะกฎ SKU วิ่งต่อ 2S → 3S (→ 6S) ทันทีในก้อนเดียวกัน (`executeWorkflow`) ส่วนอีกสองแบบรอ Admin ที่ 2A / 5A
7. ลบสินค้าที่มี order อ้างอิงไม่ได้

- “กฎ SKU” ไม่มีตารางแยก คือคอลัมน์ `skus.channel_sku` (รหัสบน Rakuten)

ข้อมูลอ้างอิง

Action	`saveProductAction`, `setProductActiveAction`, `deleteProductAction`, `importProductsAction`
Use-case	`products.ts`
SQL	`Q3A_1_FOR_UPDATE`, `Q3A_1_FIND_CONFLICT`, `Q6A_2_SUPPLIER_EXISTS`, `Q3A_1_INSERT`, `Q3A_1_UPDATE`, `Q3A_1_HAS_RELATED`, `Q3A_1_DELETE`, `Q3A_1_REQUEUE`

9 ระบบภายนอก (Adapters และ Mock)

อธิบายแบบภาษาคน

- ระบบเราต้องคุยกับ 6 ระบบนอก: Rakuten, RSL, บริการสร้างใบปะ, เครื่องพิมพ์, ขนส่ง, supplier
- แต่ละระบบมี “ตัวกลาง” (adapter) ที่พูดภาษาเดียวกันหมด
- ตอนนี้ตัวกลางทุกตัวเป็นของจำลอง ตอบกลับเหมือนจริงแต่ไม่ได้ต่อจริง
- วันหนึ่งจะต่อของจริง → เปลี่ยนแค่ตัวกลาง ส่วนอื่นไม่ต้องแก้

9.1 ทุก adapter ตอบกลับแบบเดียวกัน

src/lib/adapters/contracts.ts:16

```
export type AdapterResult<T> =
  | { ok: true; data: T }
  | { ok: false; error: { code: "unavailable" | "rejected" | "invalid_response";
                          retryable: boolean; referenceId?: string } };
```

- สำเร็จ → `ok: true` พร้อมข้อมูล
- ไม่สำเร็จ → `ok: false` พร้อมบอกว่า
  - ติดต่อไม่ได้ (`unavailable`) / ถูกปฏิเสธ (`rejected`) / ตอบมาผิดรูป (`invalid_response`)
  - ลองใหม่ได้ไหม (`retryable`)

9.2 adapter แต่ละตัว

ระบบ	ใช้ทำอะไร	ของจำลองตอบอะไร	สวิตช์ใน /dev
Rakuten	ดึง order, เช็ค order ล่าสุด, แจ้งเลขติดตาม, ส่งยอดสต๊อก	order ตัวอย่างคงที่, ส่งสำเร็จเสมอ	`rms`, `marketplaceVerify`, `trackingNotify`, `mpStockSync`
RSL	จับคู่ order, ดูสต๊อกคลัง RSL	ข้อมูลใน `src/mock/rsl.ts` (**ไม่ได้อ่านจากฐานข้อมูล**)	`rsl`
บริการใบปะ	สร้างใบปะ แบ่งกล่อง	แบ่งตามกฎใน `src/mock/delivery.ts`	`labelService`
เครื่องพิมพ์	พิมพ์ใบปะ	สำเร็จเสมอ	`printer`
ขนส่ง	ขอเลขติดตาม, ดูสถานะพัสดุ	เลขสร้างจากรหัส order, สถานะจากรายการตัวอย่าง	`carrier`
Supplier	ส่งใบสั่งซื้อ	ตอบ `MOCK-<รหัสคำขอ>`	`supplierSend`

9.3 ถ้าจะต่อระบบจริง

1. เขียน adapter ตัวจริงใน `src/lib/adapters/` ให้ตอบแบบ `AdapterResult`
2. เพิ่มโหมดใหม่ เช่น `"live"` ใน `adapters/index.ts`
3. ตั้งค่าใน `.env.local` เช่น `RAKUTEN_ADAPTER=live`
4. use-case และหน้าเว็บไม่ต้องแก้

10 การทดสอบ (Testing)

สรุปตรง ๆ

- ไม่มีเครื่องมือ test (jest, vitest, playwright) และไม่มีระบบทดสอบอัตโนมัติบน GitHub
- มี automated test **ไฟล์เดียว**: `db/seed/reset-smoke.ts`
- use-case ทั้ง 17 ไฟล์**ไม่มี test**
- การตรวจรับงานใช้ “กดทดสอบด้วยมือตามขั้นตอน” (quickstart) + เครื่องมือตรวจโค้ด

10.1 Automated test ตัวเดียวที่มี

รันด้วย `npm run test:phase8-reset` (ต้องเปิดฐานข้อมูลก่อน)

มันทดสอบอะไร

1. สร้างพื้นที่ทดสอบชั่วคราวในฐานข้อมูล ไม่แตะข้อมูลจริง
2. สร้างตาราง + ใส่ข้อมูลตัวอย่าง แล้วนับจำนวนแถวทุกตารางเก็บไว้
3. ทำให้ข้อมูล “เลอะ”: แก้จำนวนครั้งที่ใส่รหัสผิด เพิ่ม supplier และ session ปลอม
4. สั่ง reset
5. ตรวจว่า
   - จำนวนแถวทุกตารางกลับมาเท่าเดิม
   - จำนวนครั้งที่ใส่รหัสผิดกลับเป็น 0
   - supplier ปลอมหายไป
6. ลบพื้นที่ทดสอบทิ้ง

มันไม่ได้ทดสอบ: login, use-case ใด ๆ, หน้าเว็บ

10.2 เครื่องมือตรวจโค้ด

- `npm run lint`: ตรวจรูปแบบโค้ด → ผ่าน
- `npx tsc --noEmit`: ตรวจชนิดข้อมูล → ผ่าน
- `npm run build`: ลองสร้างเว็บจริง → ผ่าน
- ผลบันทึกไว้เมื่อ 5 พ.ย. 2569 (phase 9)

10.3 ทดสอบด้วยมือ (quickstart)

ชุดที่ 1: กฎสถานะ order (spec 001) มี 13 ขั้น

1. เปิดหน้า `/shipping` ครั้งแรก ดูว่าแสดงถูก
2. ส่งเลขติดตาม ORD-24174
3. เปิดหน้าซ้ำ ต้องไม่ตัดสต๊อกซ้ำ
4. ORD-24165 ต้องกลายเป็นจัดส่งสำเร็จ
5. กดส่งเลขซ้ำ ต้องมีหน้าต่างถามยืนยัน
6. ORD-24173 ส่งไม่สำเร็จ, ORD-24172 ตีกลับและคืนสต๊อก
7. ยกเลิก order ที่แจ้งเลขแล้ว
8. ยกเลิก order ที่ส่งสำเร็จแล้ว ต้องถูกปฏิเสธ
9. หน้าลบข้อมูลต้องแสดงเฉพาะ order ที่ปิดงานแล้ว
10. เปิดสวิตช์ `mpStockSync` ให้ส่งยอดพัง แล้วลองใหม่
11. ตรวจสต๊อกตอนนำเข้า order
12. ไม่มีสถานะนอก 22 ค่า

13. order ที่ส่งสำเร็จต้องคงสถานะ

ชุดที่ 2: ฐานข้อมูลจริง (spec 002) มี 11 ขั้น

1. ฐานข้อมูลว่าง → setup 3 รอบ จำนวนแถวต้องไม่เปลี่ยน
2. login ถูก / ผิด / ถูกล็อก / ปิดเปิดเว็บใหม่ / logout
3. เปิด URL ภายในก่อน login ต้องถูกพาไปหน้า login
4. เปิด 2 เบราว์เซอร์ ต้องเห็นข้อมูลเดียวกัน
5. ทุกหน้าต้องอ่านจากฐานข้อมูล
6. ทำชุดที่ 1 ทั้ง 13 ขั้นซ้ำบนฐานข้อมูลจริง
7. 2 แท็บกดพร้อมกัน 20 ครั้ง ต้องตัดสต๊อกครั้งเดียว
8. บังคับให้พังกลางทาง ข้อมูลต้องไม่ค้างครึ่งทาง
9. ทดสอบสวิตช์ใน `/dev`
10. ตรวจว่า SQL ทุกตัวมีเลข UC/Q
11. ตรวจว่าเบราว์เซอร์ไม่เก็บข้อมูลธุรกิจ

10.4 ทำไปแล้วแค่ไหน

- **ผ่านแล้ว**
  - lint, type check, build, setup ฐานข้อมูล
  - ชุดที่ 2 ขั้น 3
  - ชุดที่ 2 ขั้น 7–8 (ทดสอบด้วยสคริปต์ใน phase 7 แต่ไม่ได้เก็บสคริปต์ไว้)
- **ยังไม่ได้ทำ**
  - ชุดที่ 2 ขั้น 4, 6, 7 (บนเบราว์เซอร์จริง), 9, 11
  - ทางเลือก (alternative flow) ผ่านหน้าเว็บ: 3A #1–3, 5A #1–2, 6A #2, 9A #1,3,4, 1S #1–3, 3S #2, 5S #2–4, 10A #1–2
- **งานค้าง**: T050 = เดินทดสอบทุกขั้นแล้วจดผล ยังไม่ได้ทำ (commit “excluded T050, needed testing”)

10.5 ไฟล์ข้อมูลทดสอบ

- อยู่ใน `specs/002-real-database/test-data/` ใช้อัปโหลดในหน้า `/products`
- `products-import-3a2-validation.csv`
  - ถูก 2 แถว ผิด 6 แถว
  - ผิดแบบ: ชื่อว่าง, จุดสั่งซื้อ 0, จำนวนเป็นตัวอักษร, supplier ไม่มีจริง, รหัสซ้ำ, SKU ว่าง
- `products-import-3a3-sku-mapping.csv`: แก้สินค้าเดิม 2 ตัว

10.6 ส่วนที่ไม่มี test เลย

- use-case ทั้ง 17 ไฟล์
- กฎใน `workflow.ts`
- ระบบ login (รหัสผ่าน, session, ล็อกบัญชี)
- query ทุกไฟล์ และ adapter
- ทดสอบแบบกดหน้าเว็บอัตโนมัติ (end-to-end)

11 จุดอ่อนของ Backend

1. **สวิตช์** `skuRuleDb` **ใน /dev ไม่มีผล**: ทดสอบกรณี 2S พังไม่ได้
2. **error ของงานอัตโนมัติไม่ถูกแสดง**: บันทึกลง `auto_error_logs` แต่ไม่มีหน้าไหนแสดง
3. **รับสินค้าบางส่วนไม่ได้**: รับได้ครั้งเดียวต่อใบสั่งซื้อ ได้ไม่ครบต้องให้ 5A/6A สั่งใหม่
4. **ลำดับการจองต่างกัน**: รับสินค้าจองใบสั่งซื้อก่อน order ส่วน 7A จอง order ก่อนใบสั่งซื้อ ถ้าชนกัน Postgres ยกเลิกฝั่งหนึ่ง (กดใหม่ได้ ข้อมูลไม่เสีย)
5. **5S ถือการจองแถวไว้ขณะรอ Rakuten**: ถ้า Rakuten ตอบช้า คนอื่นต้องรอนาน
6. **กฎจัดส่งยังเป็นค่าตายตัวในโค้ด**: `SHIPPING_RULES`, `SHIPPING_METHODS` แก้ผ่านหน้าเว็บไม่ได้
7. **เอกสารยังไม่ตรง**: `Q1A_2`–`Q1A_6` เป็นเลขชั่วคราว, spec 002 ยังเขียนว่า Draft, README บรรทัด 93 เก่า
8. **ลบข้อมูลไม่มีเงื่อนไขวันที่ฝั่ง server**: เชื่อรายการที่หน้าเว็บส่งมา

12 ข้อเสนอแนะ: ต้องแก้อะไรต่อ

หลักการ

- โค้ดคือระบบที่ทำงานได้จริงแล้ว และออกแบบรัดกุมกว่า
- จึงแนะนำ **แก้รายงานและสไลด์ให้ตรงโค้ด**
- แก้โค้ดเฉพาะจุดที่ขาดสิ่งที่ requirement ต้องการจริง

สถานะ (6 พ.ย.)

รายการ (ก) และ (ข) ด้านล่างเป็นความต่างระหว่าง**รายงานรอบแรก**กับโค้ด ตอนนี้แก้ครบแล้ว ทั้งเล่ม สไลด์ สคริปต์ และ Q&A ตรงกับโค้ด branch dev (PR #28 และ #29, 17 ตาราง, SQL 113 ข้อตรงกับค่าคงที่ในโค้ดทุกข้อ) เก็บรายการไว้เป็นประวัติว่าเปลี่ยนอะไรไป

12.1 (ก) รายงาน: ตารางที่ต่างจากโค้ด

รายงาน	โค้ด	ต่างกันอย่างไร
`users`	`admins`	เปลี่ยนชื่อ; session แยกไปตาราง `sessions`
`suppliers`	`suppliers`	`lead_time` → `lead_time_days`
`products`	`skus`	เปลี่ยนชื่อ; เพิ่มรหัสบน Rakuten (`channel_sku`)
`sku_rules`	ไม่มี	รวมเข้า `skus.channel_sku`
`stock`	`stock`	key ใหม่ `stock_id`; ไม่มี `variation`
`product_cost` + `cost_calculation_log`	`costs` + `cost_history`	รวมเป็นตารางเดียว ประวัติแยกไป cost_history
`purchase_orders`	`reorders`	เปลี่ยนชื่อ; เพิ่มรหัสคำขอและ error ล่าสุด
`shipping_rules`	ไม่มี	อยู่ใน `src/mock/delivery.ts`
`orders`	`orders` + `order_items`	สินค้าแยกไป order_items; ตัดคอลัมน์ “ใครทำ” หลายตัว; เพิ่มเหตุผลค้าง ประวัติแก้ สถานะขนส่ง
`label`	`labels`	สถานะเป็น `ready/printed` แทน รอพิมพ์/พิมพ์แล้ว
`rsl_shipments`	ไม่มี	ข้อมูลมาจาก RSL adapter
`delivery_partners`, `carrier_tracking`	ไม่มี	ข้อมูลมาจาก Carrier adapter เก็บผลไว้ใน `orders`
`deletion_log`	`deletion_history`	เก็บสำเนา order ทั้งก้อน
—	**ใหม่ 8 ตาราง**	`sessions`, `auto_error_logs`, `cost_history`, `tracking_notification_logs`, `rakuten_stock_sync_states`, `stock_adjustment_history`, `label_print_history`, `schema_migrations`

ขั้นตอน/สถานะที่ต่างจาก use case description

- **4A**: สินค้าที่ไม่รู้จักค้างตั้งแต่นำเข้า (รายงาน: เป็นรอตรวจสอบเสมอ)
- **2A**: ดึงข้อมูลล่าสุดจาก Rakuten ก่อน, ลูกค้ายกเลิกแล้วยกเลิกเลย, บันทึกประวัติแก้
- **2S**: เจอหลายกฎ → ค้างให้ Admin (รายงาน: แค่แจ้งเตือน)
- **6A**: มีสถานะกลาง `รอส่งคำสั่งซื้อ` ก่อน `สั่งซื้อแล้ว`
- **7A**: ส่งให้ขนส่งแล้ว → `รอดำเนินการพิเศษ`; คืนสต๊อกได้ทั้งสองคลัง
- **8A**: ไม่มีเลขติดตาม → `รอส่งมอบ`; จัดส่งสำเร็จต้องครบ 3 ข้อ
- **7S**: ตัดราย item พร้อมจองแถว; ไม่ได้เปลี่ยนเป็นจัดส่งสำเร็จเอง
- **10A**: ลบประวัติต้นทุนแทน log คำนวณ; ใบสั่งซื้อลบเมื่อยกเลิกหรือเลยวันส่ง
- **เลข Q**: โค้ดใช้ `Q2A_1`, `Q8A_1_SAVE` … ซึ่ง**ไม่ตรง**กับ Q2A.1/Q8A.1 ในรายงาน

ไฟล์รายงานที่ต้องแก้ (ทำตามลำดับ)

1. `report-doc/content/usecases.json`: ข้อความ query และสถานะใหม่ของ 6A, 7A, 8A, 7S
2. `04-chapter2-diagrams-database/sql/schema.sql`, `seed.sql`: เปลี่ยนเป็น 17 ตาราง
3. รัน `npm run sql && npm run dict` → ได้ตาราง 2.12, 2.16, 2.17, 2.23 ใหม่
4. `report-doc/content/tables.json`: คำอธิบายตาราง 2.17
5. รัน `python3 diagrams/gen_diagrams.py && npm run diagrams` → ER, class, state, component ใหม่
6. แก้ `*-sequence.mmd`, `*-collaboration.mmd`: ชื่อ Repository, เลข Q, ตัด Scheduler
7. `report-doc/content/ch2.py` บรรทัด 123: ยังเขียนว่าเว็บเก็บข้อมูลใน localStorage
8. รัน `check_diagrams.py` แล้ว `./report-doc/export_pdf.sh`

12.2 (ข) สไลด์ สคริปต์ และ Q&A

ต้องแก้ในรายงาน/สไลด์

1. `report-doc/presentation/build_deck.js`: สไลด์ CRUD “17 Use Case × 15 entity”
2. `speaker_script/script.tex` บรรทัด 149, 302, 338–388, 448–454, 559: 15 ตาราง / 63 SQL / Repository
3. `qna_prep/qna.tex` บรรทัด 394, 498–530, 542, 616–633, 687–690, 734
   - รวมประโยค “ขั้นต่อไปคือให้ Repository เรียก db.ts” ซึ่งทำไปแล้ว
4. เพิ่มคำตอบเตรียมไว้จากบทที่ 13

12.3 (ค) Backend ที่ควรแก้ (เรียงความสำคัญ)

แก้แล้วใน PR #28 (`fix/002-stock-route-and-gaps`)

- 3S เลือกคลังจริง (RSL ก่อน แล้วคลังบริษัท) หักยอดที่ order อื่นจองไว้ RSL จำลองอ่านสต๊อกจากฐานข้อมูล
- ระบบอัตโนมัติเรียก `matchSkuUseCase`/`checkStockUseCase` ตัวเดียวกัน ไม่มีโค้ดซ้ำแล้ว
- 6A รับสินค้าจาก supplier (migration 006) order ที่ สั่งซื้อแล้ว กลับไป 3S ได้
- ใบสั่งซื้อเดียวผูกหลาย order (`orders.reorder_id`) 7A ยกเลิกใบแล้ว order อื่นกลับไป 5A
- 7S ทำซ้ำให้ order ที่ค้างตัดสต๊อก, ดึงสถานะขนส่งได้ delivered แล้วปิดงานทันที
- 3A บันทึกสินค้าแล้ว order ที่คืนคิววิ่งต่อทันที
- 1A ล็อก 15 นาทีแล้วปลดเอง (migration 007) มีปุ่มปลดล็อกใน /dev
- 5A ตรวจจำนวนและกรณีไม่มีราคาขายถูกข้อความ
- รายงาน (usecases.json, ER, schema, state, sequence) อัปเดตตามโค้ดแล้ว 113 query

1. **ทำ T050**: เดินทดสอบทุกขั้นบนเบราว์เซอร์ แล้วจดผลใน `specs/002-real-database/quickstart.md`
2. **ต่อสวิตช์** `skuRuleDb` เข้ากับ 2S
3. **แสดง error ของงานอัตโนมัติ** บน dashboard หรือ /dev
4. **เพิ่ม automated test ขั้นต่ำ**
   - ตัดสต๊อกครั้งเดียว (`deductStockUseCase`)
   - ล็อกบัญชีหลังผิด 5 ครั้ง (`loginAction`)
   - สูตรต้นทุน (`calcUnitCost`)
   - ใช้ `reset-smoke.ts` เป็นต้นแบบ
5. **ย้ายกฎจัดส่งเข้าฐานข้อมูล** ถ้าจะให้ตรงตาราง `shipping_rules` ในรายงาน
6. **อัปเดตเอกสาร**: README, สถานะ spec, เลข Q1A

13 เตรียมตอบคำถาม: Code

- คำถามที่น่าจะโดนถามตอนนำเสนอ
- คำตอบเป็นข้อสั้น ๆ พูดตามได้ทันที

13.1 สถาปัตยกรรม

Q: ระบบแบ่งเป็นกี่ชั้น

- 5 ชั้น: หน้าเว็บ → Server Action → use-case → query → ฐานข้อมูล
- ระบบภายนอกคุยผ่าน adapter
- แต่ละชั้นทำหน้าที่เดียว แก้ชั้นหนึ่งไม่กระทบชั้นอื่น

Q: ทำไมไม่มี REST API

- Next.js มี Server Action หน้าเว็บเรียกได้เหมือนฟังก์ชัน
- ไม่ต้องเขียน endpoint เอง และชนิดข้อมูลตรงกันทั้งสองฝั่ง
- ผู้ใช้มีคนเดียวคือ Admin ไม่ต้องเปิด API ให้ระบบอื่น

Q: Controller และ Repository อยู่ตรงไหนในโค้ด

- Controller = Server Action + use-case function
- Repository = ไฟล์ใน `src/lib/queries/`
- DBConnector = `src/lib/db.ts`

Q: หน้าเว็บรู้ได้อย่างไรว่าข้อมูลเปลี่ยน

- บันทึกเสร็จ ระบบสั่ง `revalidatePath`
- layout โหลดข้อมูลใหม่ แล้วหน้าเว็บแสดงใหม่เอง

13.2 ความถูกต้องของข้อมูล

Q: กดซ้ำหรือสองคนกดพร้อมกัน สต๊อกจะถูกตัดสองครั้งไหม

- ไม่
- ระบบจองแถวสต๊อกไว้ คนที่สองต้องรอ
- มีธง “ตัดแล้ว” ต่อสินค้าแต่ละรายการ
- คำสั่งตัดทำเฉพาะเมื่อของพอ
- ทดสอบกดพร้อมกัน 20 ครั้ง ตัดครั้งเดียว

Q: ถ้าระบบพังกลางขั้นตอน ข้อมูลจะค้างครึ่งทางไหม

- ไม่
- งานของ Admin และงานอัตโนมัติอยู่ใน transaction เดียว
- พังที่ไหนก็ยกเลิกทั้งก้อน แล้วบันทึก error แยก

Q: กัน order ซ้ำตอนดึงจาก Rakuten อย่างไร

- เลข order ของ Rakuten ห้ามซ้ำในฐานข้อมูล
- ดึงซ้ำ ระบบข้ามให้เอง

Q: กดส่งซ้ำตอนระบบนอกล่ม จะส่งซ้ำไหม

- ไม่
- ส่งเลขติดตามใช้รหัสคำขอเดิมของรอบที่ล้ม
- สั่งซื้อ supplier ใช้รหัสคำขอเดิม

- ระบบปลายทางรู้ว่าเป็นคำขอเดียวกัน

Q: ทำไม order บางตัวค้าง “รอดำเนินการด้วยตนเอง”

- ระบบตัดสินใจเองไม่ได้ เช่น ไม่มีกฎ SKU
- ระบบเก็บเหตุผลไว้ และหน้าเว็บพาไปหน้าที่ต้องแก้
- แก้เสร็จ order กลับเข้าคิวเอง

13.3 ความปลอดภัย

Q: เก็บรหัสผ่านอย่างไร

- ไม่เก็บรหัสจริง เก็บ hash แบบ scrypt ผสมค่าสุ่ม
- เทียบรหัสด้วยวิธีที่ใช้เวลาเท่ากันเสมอ

Q: ถ้าฐานข้อมูลรั่ว เอา session ไปใช้ได้ไหม

- ไม่ได้
- ฐานข้อมูลเก็บแค่ hash ของ token
- cookie อ่านด้วย JavaScript ไม่ได้ และหมดอายุใน 8 ชั่วโมง

Q: กัน SQL injection อย่างไร

- ค่าทุกตัวส่งแยกจากคำสั่ง SQL (`$1`, `$2`)
- ตรวจรูปแบบรหัสที่ส่งมาก่อนใช้

Q: ล็อกบัญชีเมื่อไร

- ใส่รหัสผิดครบ 5 ครั้ง → ล็อก 15 นาที (`admins.locked_until`) แล้วปลดเอง
- ระหว่างล็อกไม่ตรวจรหัสผ่าน และบอกจำนวนนาทีที่เหลือ
- ตอนทดสอบปลดทันทีได้ที่ปุ่มในหน้า `/dev`

13.4 งานอัตโนมัติและระบบภายนอก

Q: งาน System (S) ทำงานเมื่อไร ทำไมไม่มี scheduler

- รันต่อทันทีหลังปุ่มที่ทำให้ order พร้อม เช่น ยืนยัน order แล้วจับคู่ SKU และตรวจสต๊อกต่อเลย
- ผลเห็นทันที และอยู่ในก้อนเดียวกับงานของ Admin
- งานที่ต้องถามระบบนอก (ติดตามพัสดุ, สร้างใบปะที่ค้าง) รันตอนเปิดหน้า

Q: ต่อ Rakuten, RSL, ขนส่งจริงหรือยัง

- ยัง ใช้ของจำลองทุกตัว
- ทุกตัวตอบรูปแบบเดียวกัน เปลี่ยนเป็นของจริงได้โดยไม่แก้ use-case

Q: ทดสอบกรณีระบบนอกล่มอย่างไร

- เปิดสวิตช์ในหน้า `/dev` ทีละระบบ
- ดูว่า order ไปสถานะรอ/ลองใหม่ถูกต้อง

Q: คำนวณต้นทุนต่อหน่วยอย่างไร

- (ราคาซื้อ × เรต + ขนส่งระหว่างประเทศ + ภาษี) ÷ จำนวนที่สั่ง
- บวก ค่าธรรมเนียม + ขนส่งในประเทศ + ค่า RSL

13.5 การทดสอบ

Q: มี test ไหม

- automated test 1 ตัว ทดสอบการ reset ข้อมูล
- เครื่องมือตรวจโค้ด lint, type check, build ผ่านทั้งหมด
- ทดสอบด้วยมือตามขั้นตอน 13 + 11 ขั้น
- ยังไม่มี unit test ของ use-case

Q: ข้อมูลตัวอย่างมาจากไหน

- `npm run db:setup` ใส่จาก `src/mock/*`
- order แต่ละตัวอยู่คนละสถานะ ทดสอบได้ครบทุก flow
- reset ได้ที่หน้า `/dev`

14 เตรียมพรี: สิ่งที่อาจารย์ตรวจจริง

ข้อมูลจากกลุ่มที่พรีไปแล้ว (7 พ.ย.)

- อาจารย์เน้น 3 อย่าง: **Sequence diagram**, **ER diagram** และ **Query** ทำให้ตรงแบบที่อาจารย์สอนมากที่สุด
- Sequence ต้องเป็นแบบ **MVC** มี **View** และ **Controller** ชัดเจน
- **ห้ามมีลูกศรส่งกลับไปที่ Actor**
- ต้อง**เล่นโปรแกรมให้ดู** และระหว่างเล่นต้อง**เปิด DB แล้ว SELECT ให้ดู**ว่าข้อมูลเปลี่ยนจริง
- **เปิด DB ไม่เป็น หรือเขียน SELECT ไม่เป็น = แตก**
- ต้องมี **State diagram** ของข้อมูลที่ถูก update
- ภาพ diagram ต้อง**ชัดมาก** แนะนำให้ใช้ **SVG** (สุดท้ายอาจารย์ก็ zoom ดูเอง)
- อาจารย์เปิดไฟล์เล่ม (docs) ดูระหว่างพรี ไม่มีกระดาษ ส่งงานใน Classroom อย่างเดียว
- ถ้า Sequence ถูกและตัวใหญ่เห็นชัด ส่วนนี้ก็ผ่าน

14.1 เช็กลิสต์เทียบกับงานเรา

เกณฑ์ข้อ 24 ห้ามตั้งชื่อ object ว่า view

- **เกณฑ์ข้อ 24 ในไฟล์คะแนนเขียนว่า object ที่ติดกับ actor ต้องเป็น user interface “ห้ามใช้ชื่อว่า view”** เกณฑ์ชนะคำแนะนำในแชต
- จึงใช้ `:XxxScreen` (ชั้น View ของ MVC) และ `:XxxController` ทุกรูป ไม่มี object ชื่อลงท้าย View แล้ว `check_diagrams.py` ตรวจให้
- ถ้าอาจารย์ถาม: “Screen คือหน้าจอ (user interface) ชั้น View ของ MVC ชื่อหน้าจอในหัวข้อ UI (2.21) ใช้ชื่อเดียวกัน เช่น `:OrderVerifyScreen` = หน้าจอ OrderVerifyScreen”

สิ่งที่อาจารย์ตรวจ	งานเราตอนนี้	สถานะ
Sequence เป็น MVC	ทั้ง 17 UC: `:XxxScreen` → `:XxxController` → `:XxxUseCase` / `:XxxQueries` (Model) → `:DBConnector` ภาพยาวถูกตัดเป็นส่วน (1/n) หัว participant ซ้ำทุกส่วน	**เสร็จ** (seq-mvc, rename-screen ใช้ชื่อ Screen ตามเกณฑ์ข้อ 24)
Screen ไม่เรียก DB เอง	Screen คุยกับ Controller เท่านั้น ส่วน Queries / DBConnector ถูกเรียกจาก Controller หรือ UseCase	**เสร็จ** `check_diagrams.py` ตรวจให้
ห้ามส่งกลับ Actor	ไม่มีลูกศรกลับหา Admin ผลลัพธ์แสดงเป็น self message บน Screen เช่น `showOrdersWaitingVerify()`	**เสร็จ** `check_diagrams.py` ตรวจทั้ง Sequence และ Collaboration
Collaboration	สร้างจาก mmd ชุดเดียวกับ Sequence ชื่อ object และเลขลำดับตรงกัน	**เสร็จ** สร้างใหม่ทั้ง 17 ด้วย `gen_collab.py` ตัวอักษรใหญ่ขึ้น
ER	17 ตาราง 22 ความสัมพันธ์ แบบ Chen ตรงโค้ด	เสร็จ (ซ้อมตอบ PK/FK/cardinality)
Query	113 SQL ตรงโค้ดทุกตัว (`usecases.json` = `src/lib/queries`) รวม `Q5S.4` ที่แก้แล้ว	**เสร็จ** `npm run sql` (รันในrepo รายงาน `04-chapter2-diagrams-database`): 113 ข้อ 0 failed
State diagram	Order (`order_status` 2 รูป) + `reorders.reorder_status`, `labels.label_status`, `skus.sku_status`, `orders.delivery_status`, `rakuten_stock_sync_states.sync_status` เขียนแบบ `Class.state` และ `[guard] event / UC (Qid)`	**เสร็จ** (state-svg; final-check2 เพิ่มเส้น `unmatchRsl / 1S (Q1S.4)` กลับ ยังไม่ได้จับคู่ ในรูป Order 2/2)

สิ่งที่อาจารย์ตรวจ	งานเราตอนนี้	สถานะ
ภาพชัด	เล่ม Word ใช้ PNG ความละเอียด 300 dpi ขึ้นไป ปิดการบีบรูปของ Word แล้ว Sequence ตัดเป็นส่วนให้อ่านได้ ไฟล์ SVG อยู่ใน `diagrams/svg/`, `diagrams/png/*.svg` และ `report-doc/output/รูป-diagram/` ไว้เปิด zoom	**เสร็จ** ไม่ใส่ SVG ในเล่ม เพราะ Word/PowerPoint ไม่แสดงตัวหนังสือใน SVG ของ mermaid
เดโม + SELECT	ทดสอบสด 1A, 4A, 2A, 9A, 8A, 5A และ 5S+7S ผ่านทุกข้อบน ข้อมูล `db:reset` (code-demo) ชุด SELECT อยู่หัวข้อ 16 และสไลด์ 54	**เสร็จ** เหลือซ้อมให้คล่อง

ยังเปิดอยู่ (8 พ.ย.)

1. ข้อมูลของเจ้าของงาน: ตาราง 2.25 และสไลด์ 55 เติมแล้ว (ธนภัทร 16, อชิระ 17, ปิยากร 20 ชิ้น คะแนน 5/5 ทุกคน) ผู้รับผิดชอบทุกข้อยืนยันแล้ว หัวหน้าทีมคือธนภัทร
2. ซ้อมเดโม: รัน `npm run db:reset` ก่อนพรี และอย่าเปิดหน้า `/shipping` ก่อนเดโม (เปิดแล้วระบบดึงสถานะขนส่งเอง)

14.2 MVC ใน Sequence: เทียบกับโค้ดจริง

- **Actor**: Super Admin เป็นคนเริ่มเรื่อง ลูกศรออกจาก Actor ได้ แต่**ไม่มีลูกศรกลับเข้า Actor**
- **Screen** (ชั้น View ของ MVC) = `:XxxScreen`: หน้าเว็บ `page.tsx` รับคลิกและแสดงผล (`:LoginScreen` = `login/page.tsx`, `:AppShellScreen` = `layout.tsx`)
- **Controller** = `:XxxController`: Server Action ใน `(app)/actions.ts` (`:LoginController` = `login/actions.ts`) รับคำสั่งจากหน้าเว็บแล้วเรียก use-case
- ตอนเปิดหน้า รูปวาดเป็น Screen → Controller `loadAppState()` → Queries → DB (ของจริงคือ `queries/app-state.ts` ที่ `layout.tsx` เรียก)
- **Model** = `:XxxUseCase`, `:ExecuteWorkflow`, `:XxxQueries`: ทำงานกับข้อมูล
- **Database** = `:DBConnector` (`db.ts`) ต่อ PostgreSQL (`rsl-pg`)
- ระบบภายนอก (Rakuten, RSL, ขนส่ง, Supplier, เครื่องพิมพ์) เป็น `:XxxAdapter` เรียกจาก Model
- ชื่อ Screen และ Controller เป็นแค่ชื่อบทบาทในรูป ไม่ได้สร้าง class ใหม่ในโค้ด (โค้ดเป็นฟังก์ชันธรรมดา)

ตัวอย่าง: UC 2A ตรวจสอบคำสั่งซื้อ (ย่อจาก `2A-sequence.mmd`)

```
Admin ->> :OrderVerifyScreen      : clickConfirm(order_id, shipping_method)
:OrderVerifyScreen ->> :OrderVerifyController : verifyOrderAction(orderId, shippingMethod)
:OrderVerifyController ->> :VerifyOrderUseCase : verifyOrderUseCase(orderId, ...)
:VerifyOrderUseCase ->> :OrderQueries : lockOrder(orderId)
:OrderQueries ->> :DBConnector    : query(Q2A.2)
:DBConnector -->> :OrderQueries   : return snapshot
:OrderQueries -->> :VerifyOrderUseCase : return snapshot
:VerifyOrderUseCase -->> :OrderVerifyController : return ok
:OrderVerifyController -->> :OrderVerifyScreen : return result
:OrderVerifyScreen ->> :OrderVerifyScreen : showSuccess()   (no arrow back to Admin)
```

15 ทัวร์หน้าเว็บทีละหน้า

อธิบายแบบภาษาคน

- บทนี้พาเดินทีละหน้าตามเมนูซ้าย บวกหน้า login, หน้ารายงาน, `/dev` และ `/sitemap`
- แต่ละหน้าบอก: บนจอมีอะไร, กดปุ่มแล้วเกิดอะไร, ระวังอะไร, อาจารย์น่าจะถามอะไร, SELECT ไว้พิสูจน์
- เลข Query ใช้เลขเดียวกับเล่มรายงาน (`usecases.json`) เช่น `Q2A.3` ไม่ใช่ชื่อค่าคงที่ในโค้ด
- รายละเอียดเชิงลึกของแต่ละ UC อยู่ในบทที่ 8 บทนี้ไม่เขียนซ้ำ
- ภาพทุกภาพจับจากเว็บจริงหลัง `npm run db:reset` (8 พ.ย. 2569)

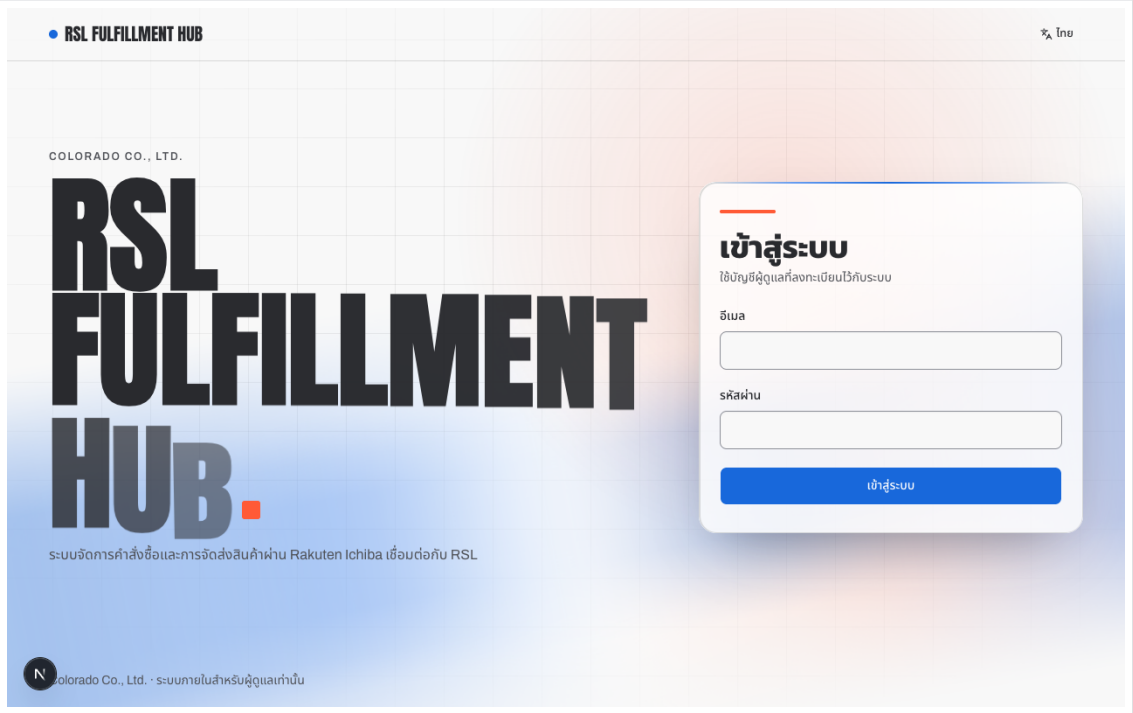
15.1 ภาพรวมทุกหน้า

URL	เมนู	UC	Actor	เปิดแล้วข้อมูลเปลี่ยน?
`/login`	เข้าสู่ระบบ	1A	เจ้าของ	ไม่
`/dashboard`	ภาพรวมระบบ	4A	เจ้าของ, Marketplace	ไม่
`/orders/verify`	ตรวจสอบคำสั่งซื้อ	2A (+2S 3S 6S)	เจ้าของ, Marketplace	ไม่
`/orders/rsl-match`	จับคู่ Order กับ RSL	1S (+6S)	เจ้าของ, Marketplace (RSL)	ไม่
`/orders/cancel`	ยกเลิก Order	7A	เจ้าของ, Marketplace	ไม่
`/products`	ตั้งกฎ SKU / ข้อมูลสินค้า	3A (+4S 2S 3S)	เจ้าของ	ไม่
`/products/stock`	ตรวจสอบสต๊อก	3S	เจ้าของ, Marketplace (RSL)	ไม่
`/products/cost`	คำนวณต้นทุน	4S	เจ้าของ	ไม่
`/products/reorder`	ตัดสินใจสั่งซื้อเพิ่ม	5A, 6A	เจ้าของ, Supplier	ไม่
`/shipping`	จัดส่งสินค้าให้ลูกค้า	8A, 5S (+7S)	เจ้าของ, Delivery, Marketplace	เปลี่ยน
`/shipping/label`	พิมพ์ใบปะสินค้า	9A (+6S)	เจ้าของ	อาจเปลี่ยน
`/system/cleanup`	ลบข้อมูลเก่า	10A	เจ้าของ	ไม่
`/reports/...`	(เปิดจากหน้าอื่น)	9A, 4S	เจ้าของ	ไม่
`/dev`	(ไม่อยู่ในเมนู)	เครื่องมือทดสอบ	—	ปุ่ม reset ลบข้อมูล
`/sitemap`	(ไม่อยู่ในเมนู)	ผังหน้าจอ	—	ไม่

จำ 3 ข้อก่อนเดินทัวร์

- ในโค้ด actor ที่กดปุ่มมีคนเดียวคือ Admin (= เจ้าของ ในเล่มเรียก Super Admin) อีก 3 actor เป็นระบบภายนอกที่ระบบเราเรียกผ่าน adapter (mock)
- ทุกปุ่มที่เขียนข้อมูลเรียก Server Action ใน `src/app/(app)/actions.ts` แล้วไป use-case และ SQL (บทที่ 2)
- ปุ่มส่วนใหญ่วิ่งผ่าน `executeWorkflow` จึงมีงานอัตโนมัติ 2S/3S/6S/7S วิ่งต่อในก้อนเดียวกัน (บทที่ 7)

15.2 หน้า Login (`/login`)



ภาพ: หน้าเข้าสู่ระบบ

บนจอมีอะไร

- ซ้าย: ชื่อระบบ RSL Fulfillment Hub (ตกแต่ง)
- ขวา: ช่องอีเมล, ช่องรหัสผ่าน, ปุ่ม “เข้าสู่ระบบ”
- มุมขวาบน: ปุ่มสลับภาษาไทย/อังกฤษ

กด “เข้าสู่ระบบ” แล้วเกิดอะไร (UC 1A)

1. หาบัญชีจากอีเมล (`Q1A.1`)
2. ถ้าเลยเวลาล็อกแล้ว → ปลดล็อกให้ (`Q1A.2`)
3. รหัสผิด → นับครั้งผิด ผิดครบ 5 ครั้ง → ล็อก 15 นาที (`Q1A.3`)
4. รหัสถูก → ล้างครั้งผิด บันทึก `last_login` (`Q1A.4`) และสร้าง session (`Q1A.5`)
5. ไปหน้า `/dashboard`

- ทุกหน้าหลัง login ตรวจ session ด้วย `Q1A.6`, ปุ่ม logout ใช้ `Q1A.7`
- ตารางที่แตะ: `admins`, `sessions`
- ไม่มีงานอัตโนมัติตามมา

Q: ใส่รหัสผิดหลายครั้งแล้วเกิดอะไร

- ครบ 5 ครั้งบัญชีถูกล็อก 15 นาที ลองด้วย `locked@colorado.jp` ได้

Q: เก็บรหัสผ่านอย่างไร

- เก็บเป็น hash แบบ scrypt ไม่เก็บรหัสจริง session ก็เก็บแค่ hash ของ token

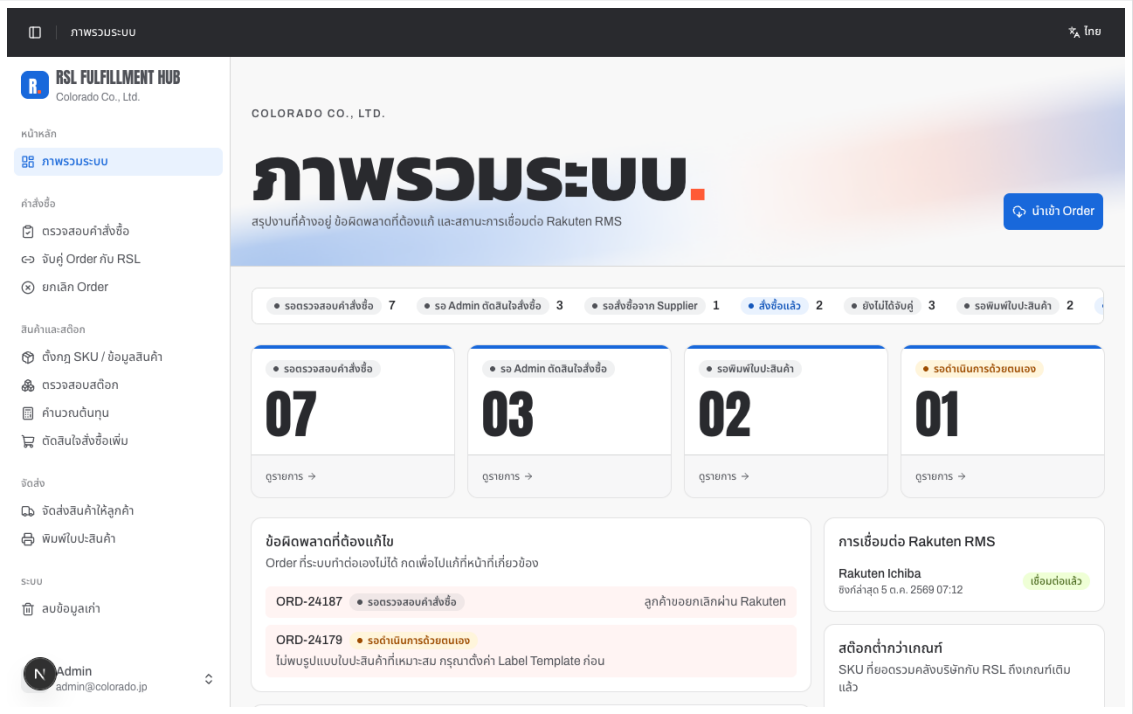
Q: login แล้วอยู่ได้นานแค่ไหน

- 8 ชั่วโมง (`SESSION_DURATION_HOURS`) logout แล้ว session ถูกยกเลิกทันที

SELECT พิสูจน์

```
SELECT user_email, fail_attempts, locked_until, last_login FROM admins;
SELECT session_id, created_at, expires_at, revoked_at
FROM sessions ORDER BY created_at DESC LIMIT 3;
```

15.3 ภาพรวมระบบ (/dashboard)



ภาพ: หน้าภาพรวมระบบ

บนจอมีอะไร

- แถบป้ายบนสุด: จำนวน order แยกตามสถานะ
- การ์ดตัวเลขใหญ่ 4 ใบ: งานค้างที่สำคัญ กด “ดูรายการ” ไปหน้าที่ต้องทำ
- “ข้อผิดพลาดที่ต้องแก้ไข”: order ที่ระบบทำต่อเองไม่ได้ กดแล้วพาไปหน้าที่แก้ได้
- “การเชื่อมต่อ Rakuten RMS”: ดึงล่าสุดเมื่อไร
- “สต๊อกต่ำกว่าเกณฑ์”, “สถานะการสั่งซื้อเติมสต๊อก”, “ต้นทุนต่อหน่วย”, กราฟ order 7 วันล่าสุด
- ปุ่มขวาบน “นำเข้า Order”

กด “นำเข้า Order” แล้วเกิดอะไร (UC 4A)

- `importOrdersAction` → `importOrdersUseCase` → ขอ order ใหม่จาก Rakuten (mock)
- ตรวจว่ารู้จักสินค้าไหม (`Q4A.4`) แล้วบันทึก order ใหม่ (`Q4A.5`) เลข Rakuten ซ้ำ → ข้าม
- ลูกค้าขอยกเลิกบน Rakuten → ติดธง `customer_cancel_requested` (`Q4A.6`)
- สถานะ: (ไม่มี) → `รอตรวจสอบคำสั่งซื้อ` หรือ `รอดำเนินการด้วยตนเอง` ถ้าไม่รู้จักสินค้า
- ตาราง: `orders`, `order_items`
- ไม่มีงานอัตโนมัติตามมา ต้องไปกดตรวจสอบที่ `/orders/verify`
- ตัวเลขบนหน้าอ่านด้วย `Q4A.1`–`Q4A.3`

ระวัง

- กดนำเข้ารอบแรกหลัง reset จะเห็น toast เตือนหลายอัน (ลูกค้ายกเลิก, SKU ไม่รู้จัก, ข้อมูลไม่ครบ) เป็นข้อมูลตัวอย่างที่ตั้งใจใส่ ไม่ใช่ bug
- ตัวเลขบน dashboard เปลี่ยนหลังเปิดหน้า `/shipping` ครั้งแรก (ดูหน้า `/shipping`)

Q: ดึง order จาก Rakuten ตามเวลาไหม

- ไม่ ต้องกดปุ่ม “นำเข้า Order” เอง ระบบไม่มี scheduler

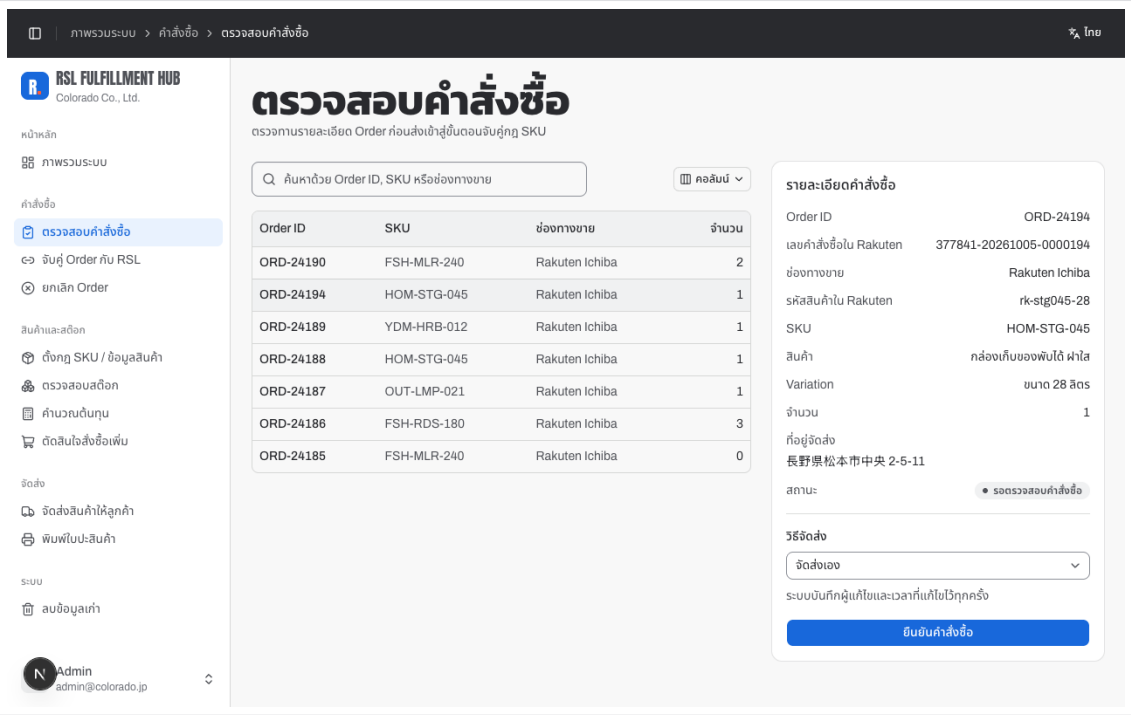
Q: ดึงซ้ำแล้ว order ซ้ำไหม

- ไม่ซ้ำ เลข Rakuten (`marketplace_order_id`) เป็น unique ระบบข้ามตัวที่มีแล้ว

SELECT พิสูจน์

```
SELECT order_status, COUNT(*) FROM orders
GROUP BY order_status ORDER BY 2 DESC;
SELECT order_id, marketplace_order_id, order_status, manual_reason,
       customer_cancel_requested
FROM orders ORDER BY created_at DESC LIMIT 5;
```

15.4 ตรวจสอบคำสั่งซื้อ (`/orders/verify`)



ภาพ: หน้าตรวจสอบคำสั่งซื้อ หลังคลิก ORD-24194

บนจอมีอะไร

- ซ้าย: ตาราง order ที่ `รอตรวจสอบคำสั่งซื้อ` ค้นหาได้
- ขวา: แผงรายละเอียดของ order ที่คลิก (เลข Rakuten, SKU, จำนวน, ที่อยู่)
- ช่องเลือก “วิธีจัดส่ง” แก้ได้ก่อนยืนยัน (ระบบจดว่าแก้อะไรใน `edit_log`)
- ปุ่ม “ยืนยันคำสั่งซื้อ”

กด “ยืนยันคำสั่งซื้อ” แล้วเกิดอะไร (UC 2A)

- `verifyOrderAction` → `verifyOrderUseCase` ผ่าน `executeWorkflow`
- อ่านและล็อก order (`Q2A.1`, `Q2A.2`) ถามสถานะล่าสุดจาก Rakuten
- ผ่าน → `รอจับคู่กฎ SKU` (`Q2A.3`)
- **2S ต่อเอง**: หากฎ SKU (`Q2S.2`) เจอ 1 ตัว → `รอตรวจสอบสต๊อก` (`Q2S.3`)
- **3S ต่อเอง**: รวมสต๊อกคลังบริษัท + RSL หักของที่ order อื่นจองไว้ (`Q3S.1`–`Q3S.4`)
  - พอ (RSL) → `ยังไม่ได้จับคู่` พอ (คลังบริษัท) → **6S** สร้างใบปะ → `รอพิมพ์ใบปะสินค้า`
  - ไม่พอ → `รอ Admin ตัดสินใจสั่งซื้อ`
- ลูกค้ายกเลิกแล้ว → `ยกเลิกแล้ว`, ข้อมูลไม่ครบ → `รอดำเนินการด้วยตนเอง`

- ตาราง: `orders`, `order_items` (+ `labels` ถ้า 6S วิ่ง)

Q: กดครั้งเดียวทำไมสถานะกระโดดหลายขั้น

- เพราะงานอัตโนมัติ 2S → 3S → 6S วิ่งต่อทันทีใน transaction เดียวกัน (บทที่ 7)

Q: ORD-24187 มีข้อความ “ลูกค้าขอยกเลิกผ่าน Rakuten” กดยืนยันแล้วเป็นอะไร

- ระบบถาม Rakuten อีกครั้ง ถ้ายกเลิกจริง order เป็น `ยกเลิกแล้ว` ไม่วิ่งต่อ

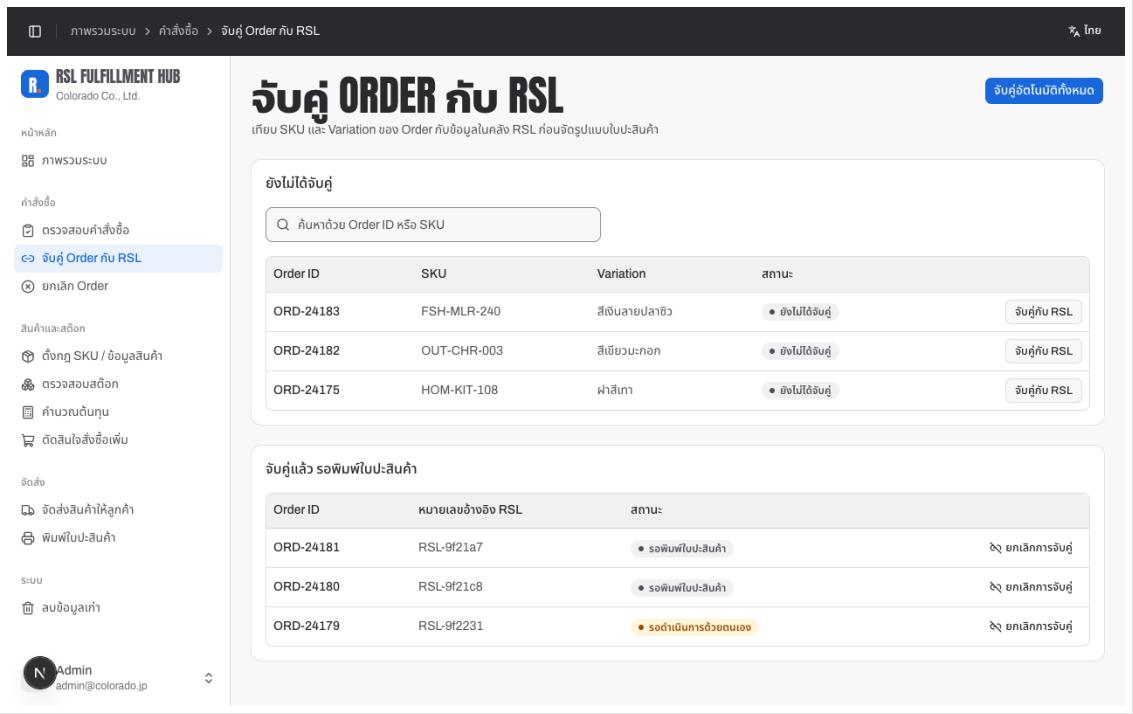
Q: ถ้า RSL ล่มตอนตรวจสต๊อก

- ถือว่าคลัง RSL ไม่มีของ บันทึกลง `auto_error_logs` ลองได้ด้วยสวิตช์ `1S / 3S` ใน `/dev`

SELECT พิสูจน์ (ก่อน: `รอตรวจสอบคำสั่งซื้อ`, หลัง: `รอพิมพ์ใบปะสินค้า` และ `in_house`)

```
SELECT o.order_id, o.order_status, o.manual_reason, o.verified_at,
       i.sku_id, i.fulfill_source
FROM orders o JOIN order_items i USING (order_id)
WHERE o.order_id = 'ORD-24194';
```

15.5 จับคู่ Order กับ RSL (`/orders/rsl-match`)



ภาพ: หน้าจับคู่ Order กับ RSL

บนจอมีอะไร

- ตารางบน “ยังไม่ได้จับคู่”: order ที่สต๊อกพอและจะส่งจากคลัง RSL แต่ละแถวมีปุ่ม “จับคู่กับ RSL”
- ตารางล่าง “จับคู่แล้ว”: มีเลขอ้างอิง RSL (`RSL-...`) และปุ่ม “ยกเลิกการจับคู่”
- ปุ่มขวาบน “จับคู่อัตโนมัติทั้งหมด”

กดจับคู่แล้วเกิดอะไร (UC 1S)

- `matchRslAction` / `matchAllRslAction` → `matchRslUseCase`
- หาคิว (`Q1S.1`) ล็อก order (`Q1S.2`) ถาม RSL ว่าตรงกับรายการไหน
- เจอ 1 รายการ → บันทึกเลข RSL (`Q1S.3`) `ยังไม่ได้จับคู่` → `รอจัดรูปแบบใบปะสินค้า`
- **6S ต่อเอง**: สร้างใบปะ (`Q6S.4`, `Q6S.5`) → `รอพิมพ์ใบปะสินค้า`
- เจอหลายรายการ → ขึ้นหน้าต่างให้เลือก, ไม่เจอ → `รอดำเนินการด้วยตนเอง`
- “ยกเลิกการจับคู่” (`Q1S.4`) → ลบใบปะ กลับเป็น `ยังไม่ได้จับคู่`

- ตาราง: `orders`, `labels`

Q: RSL คืออะไร ทำไมต้องจับคู่

- Rakuten Super Logistics คลังของ Rakuten ต้องได้เลขอ้างอิง RSL ก่อนจึงพิมพ์ใบปะได้

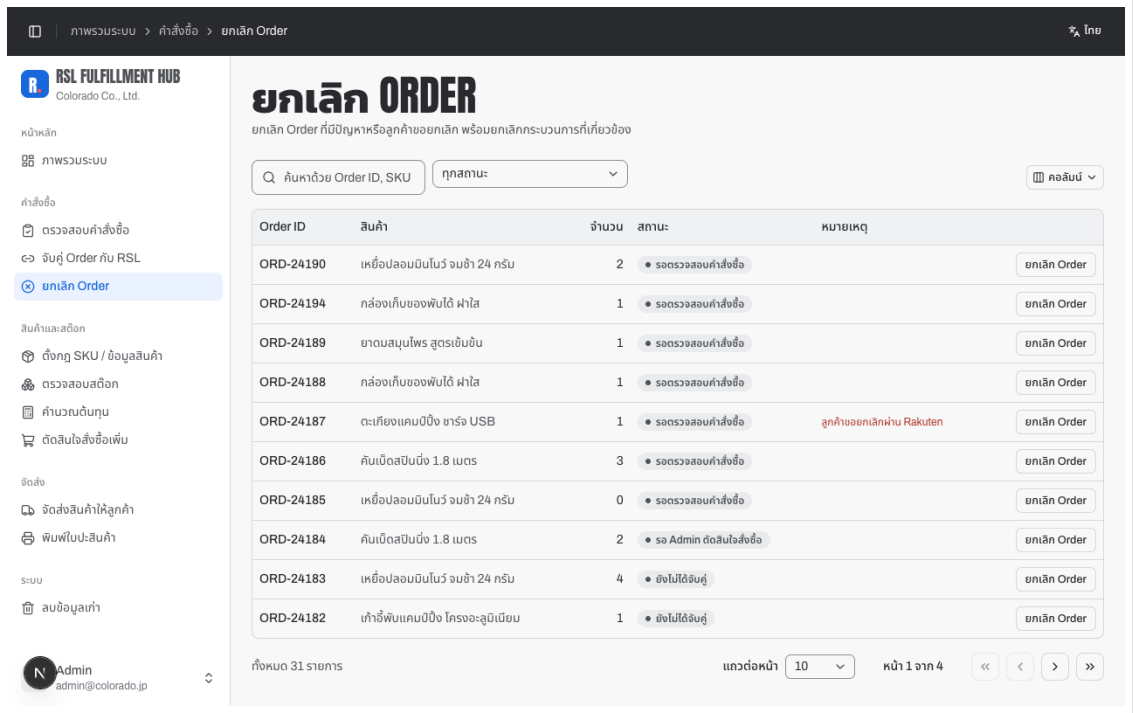
Q: order ที่ส่งจากคลังบริษัทเองผ่านหน้านี้ไหม

- ไม่ผ่าน ข้ามไป 6S ทันทีหลัง 3S

SELECT พิสูจน์ (ลองกดจับคู่ ORD-24183)

```
SELECT o.order_id, o.order_status, o.rsl_no, o.rsl_matched_at,
       l.label_id, l.label_status
FROM orders o LEFT JOIN labels l USING (order_id)
WHERE o.order_id = 'ORD-24183';
```

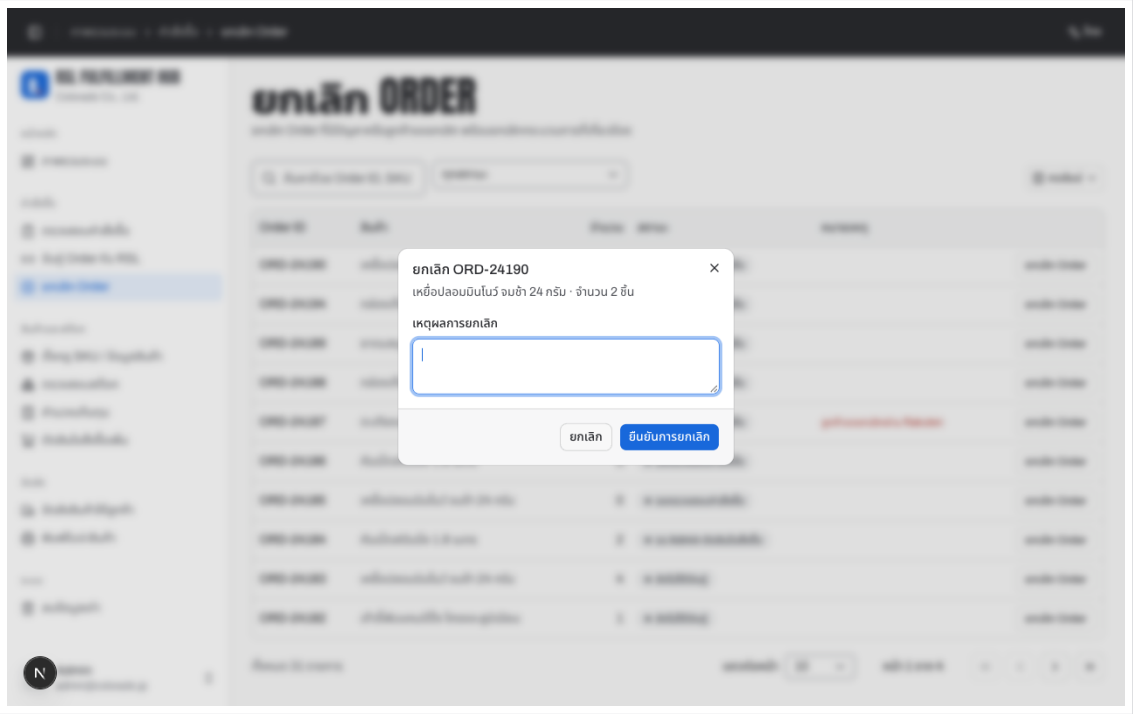
15.6 ยกเลิก Order (`/orders/cancel`)



ภาพ: หน้ายกเลิก Order

บนจอมีอะไร

- ตาราง order ทุกตัว กรองตามสถานะได้ (“ทุกสถานะ”)
- คอลัมน์ “หมายเหตุ”: เช่น ลูกค้าขอยกเลิกผ่าน Rakuten
- แต่ละแถวมีปุ่ม “ยกเลิก Order” ถ้า order ยกเลิกไม่ได้ (เช่น จัดส่งสำเร็จ) หรือยกเลิกไปแล้ว กดแล้วขึ้น toast แดง ไม่เปิดหน้าต่าง



ภาพ: หน้าต่างยืนยัน: ต้องกรอกเหตุผลก่อน

กด “ยืนยันการยกเลิก” แล้วเกิดอะไร (UC 7A)

- `cancelOrderAction` → `cancelOrderUseCase`
- ล็อก order (`Q7A.1`) แล้วเลือกทาง
  - ยังไม่ส่ง → ยกเลิกแล้ว พร้อมเหตุผล (`Q7A.2`) คืนสต๊อกถ้าตัดไปแล้ว (`Q7A.3`)
  - ส่งให้ขนส่งแล้ว → รอดำเนินการพิเศษ
- ติ๊ก “ยกเลิกคำสั่งซื้อเติมสต๊อกนี้ด้วย” (ขึ้นเฉพาะ order ที่ผูกใบสั่งซื้อ) → ยกเลิกใบสั่งซื้อที่ยังไม่ได้รับของ (`Q7A.4`) order อื่นที่รอใบเดียวกันกลับไป รอ Admin ตัดสินใจสั่งซื้อ (`Q7A.5`)
- ตาราง: `orders`, `stock`, `reorders`

Q: ยกเลิก order ที่ส่งถึงลูกค้าแล้วได้ไหม

- ไม่ได้ จัดส่งสำเร็จ ยกเลิกไม่ได้ ถ้าอยู่ระหว่างส่งได้แค่ รอดำเนินการพิเศษ

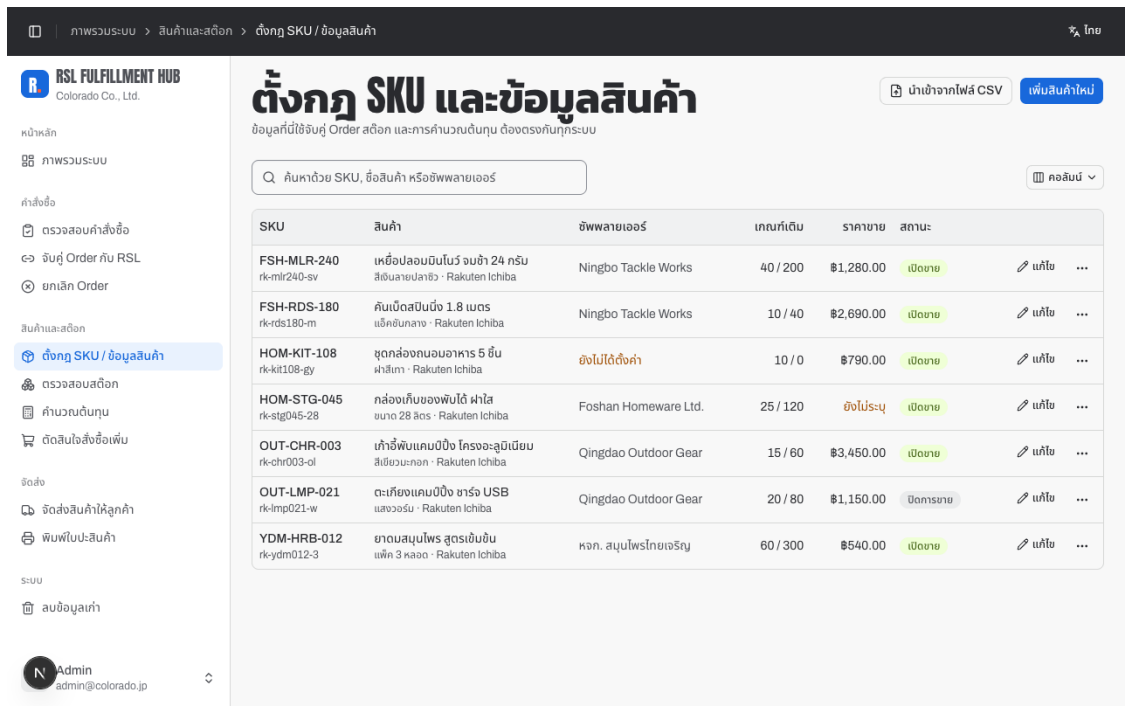
Q: ยกเลิกแล้วสต๊อกกลับไหม

- กลับ ถ้าเคยตัดไปแล้ว ระบบบวกคืนคลังเดิมในก้อนเดียวกัน

SELECT พิสูจน์

```
SELECT order_id, order_status, cancel_reason, cancelled_at, reorder_id
FROM orders WHERE order_id = 'ORD-24190';
```

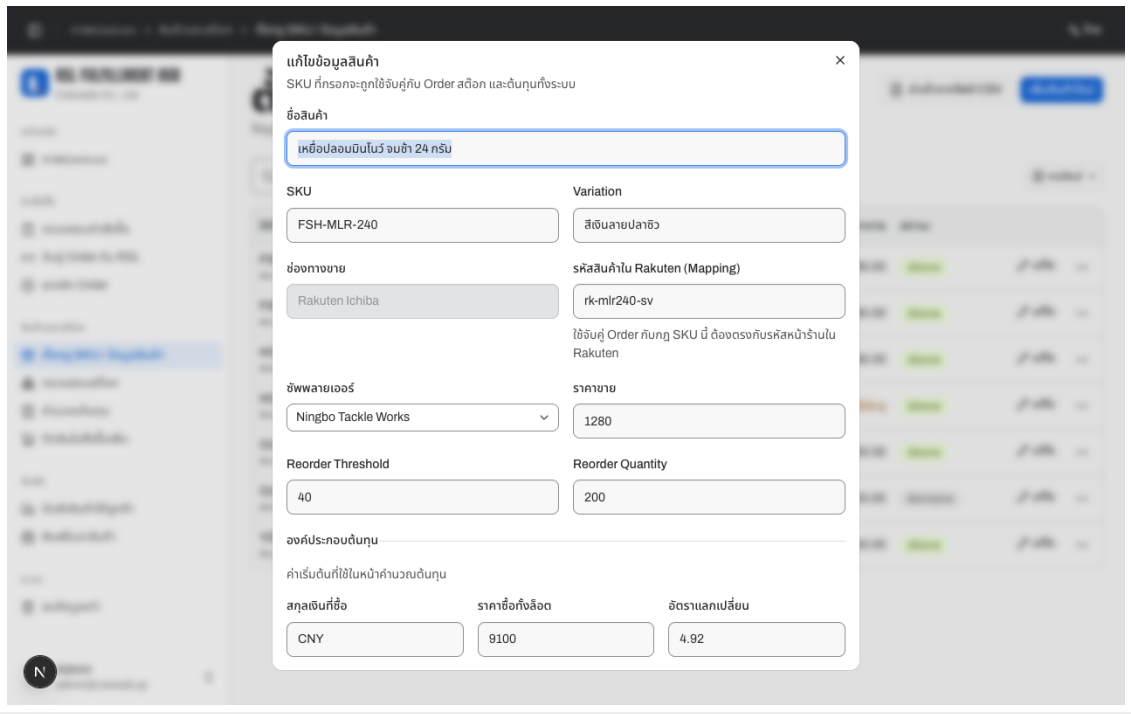
15.7 ตั้งกฎ SKU / ข้อมูลสินค้า (/products)



ภาพ: หน้าตั้งกฎ SKU และข้อมูลสินค้า

บนจอมีอะไร

- ตารางสินค้า: SKU (บรรทัดล่างสีเทา = รหัสบน Rakuten = “กฎ SKU”), ชื่อ, supplier, เกณฑ์เติม / จำนวนสั่ง, ราคาขาย, เปิด/ปิดขาย
- ปุ่มบน: “นำเข้าจากไฟล์ CSV”, “เพิ่มสินค้าใหม่”
- แต่ละแถว: “แก้ไข” และเมนู ... (เปิด/ปิดขาย, ลบ)
- “ยังไม่ได้ตั้งค่า” สีส้ม = ไม่มี supplier, “ยังไม่ระบุ” = ไม่มีราคาขาย (ข้อมูลตัวอย่างตั้งใจใส่)



ภาพ: หน้าต่างแก้ไขสินค้า มีองค์ประกอบต้นทุนด้านล่าง

กดบันทึกแล้วเกิดอะไร (UC 3A + UC 4S)

- `saveProductAction` → `saveProductUseCase` ผ่าน `executeWorkflow`

- ตรวจรหัสซ้ำ (Q3A.4) supplier มีจริง (Q3A.5) แล้วเพิ่ม (Q3A.6) หรือแก้ (Q3A.7)
- กรอกต้นทุน → 4S คำนวณและเก็บ (Q4S.5, Q4S.6)
- ดึง order ที่ค้างเพราะสินค้านี้กลับเข้าคิว (Q3A.8) แล้ว 2S → 3S (→ 6S) วิ่งต่อ
- เปิด/ปิดขาย (Q3A.9), ลบ: มี order อ้างอิงลบไม่ได้ (Q3A.10, Q3A.11)
- CSV ผิดแถวเดียว → ไม่บันทึกทั้งไฟล์
- ตาราง: skus, stock (สินค้าใหม่ได้สต๊อก 0), costs, cost_history, orders

Q: กฎ SKU เก็บที่ไหน

- คอลัมน์ skus.channel_sku ไม่มีตารางกฎแยก

Q: แก้ราคาขายแล้ว order ที่ค้างไปต่อไหม

- ไปต่อ order ที่ค้างเพราะไม่มีราคากลับไป รอ Admin ตัดสินใจสั่งซื้อ

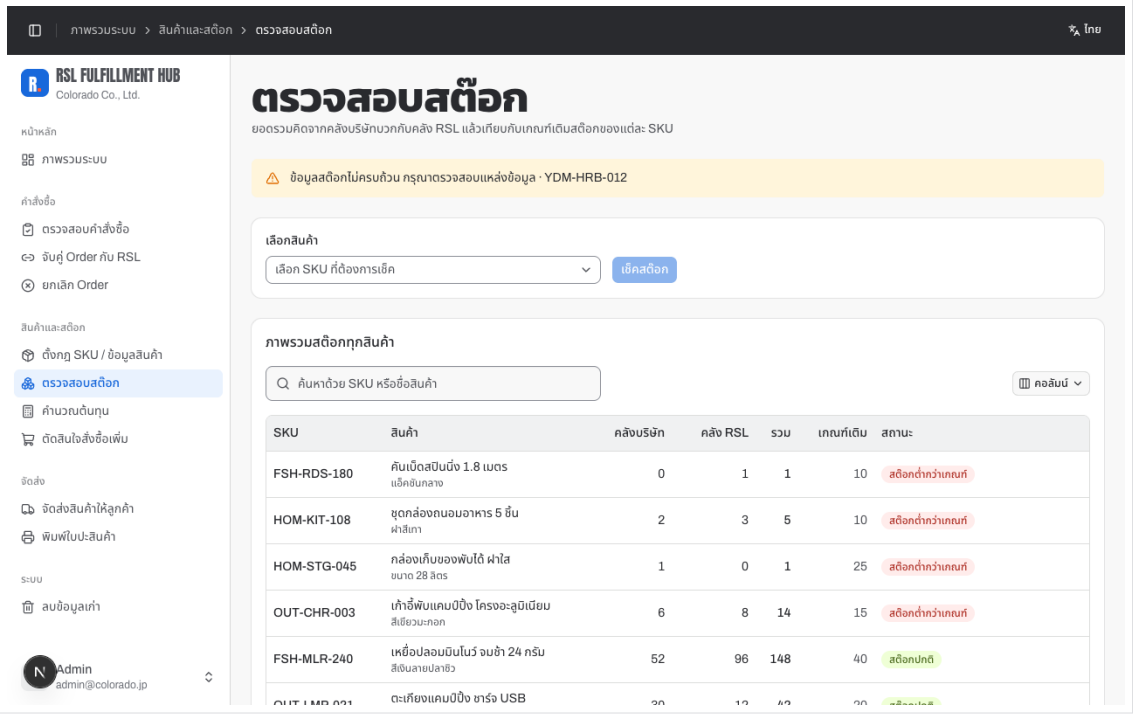
Q: ลบสินค้าที่มี order แล้วได้ไหม

- ไม่ได้ ให้ปิดขายแทน

SELECT พิสูจน์

```
SELECT sku, channel_sku, sku_status, selling_price,
       reorder_threshold, supplier_id
FROM skus ORDER BY sku;
SELECT order_id, order_status, manual_reason FROM orders
WHERE manual_reason IN ('sku-rule', 'sku-unregistered', 'price');
```

15.8 ตรวจสอบสต๊อก (/products/stock)



ภาพ: หน้าตรวจสอบสต๊อก

บนจอมีอะไร

- แถบเหลือง “ข้อมูลสต๊อกไม่ครบถ้วน · YDM-HRB-012”: สินค้านี้ไม่มีตัวเลขคลัง RSL (ตั้งใจใส่ไว้ทดสอบ)
- “เลือกสินค้า” + ปุ่ม “เช็คสต๊อก”: ดูยอดของ SKU เดียว **อ่านอย่างเดียว ไม่เขียน DB**
- ตาราง “ภาพรวมสต๊อกทุกสินค้า”: คลังบริษัท, คลัง RSL, รวม, เกณฑ์เติม, ป้าย “สต๊อกต่ำกว่าเกณฑ์”
- กด “เช็คสต๊อก” แล้วจะมีฟอร์มปรับสต๊อกของ SKU นั้นโผล่ด้านล่าง (จำนวนใหม่ + เหตุผล)

กดบันทึกการปรับสต๊อกแล้วเกิดอะไร (UC 3S ฝั่ง Admin)

- `adjustStockAction` → `adjustStockUseCase`
- ล็อกแถว (`Q3S.7`) แก้ยอด (`Q3S.8`) เก็บประวัติ (`Q3S.9`)
- ตั้งให้ส่งยอดใหม่ไป Rakuten (`rakuten_stock_sync_states`)
- ไม่มีการเปลี่ยนสถานะ order

Q: ป้ายสต๊อกต่ำแล้วระบบสั่งซื้อให้เองไหม

- ไม่ ป้ายคำนวณบนจอ SKU นั้นไปโผล่ในแท็บ “รอสั่งซื้อเติมสต๊อก” ของหน้า reorder ให้ Admin กดเอง

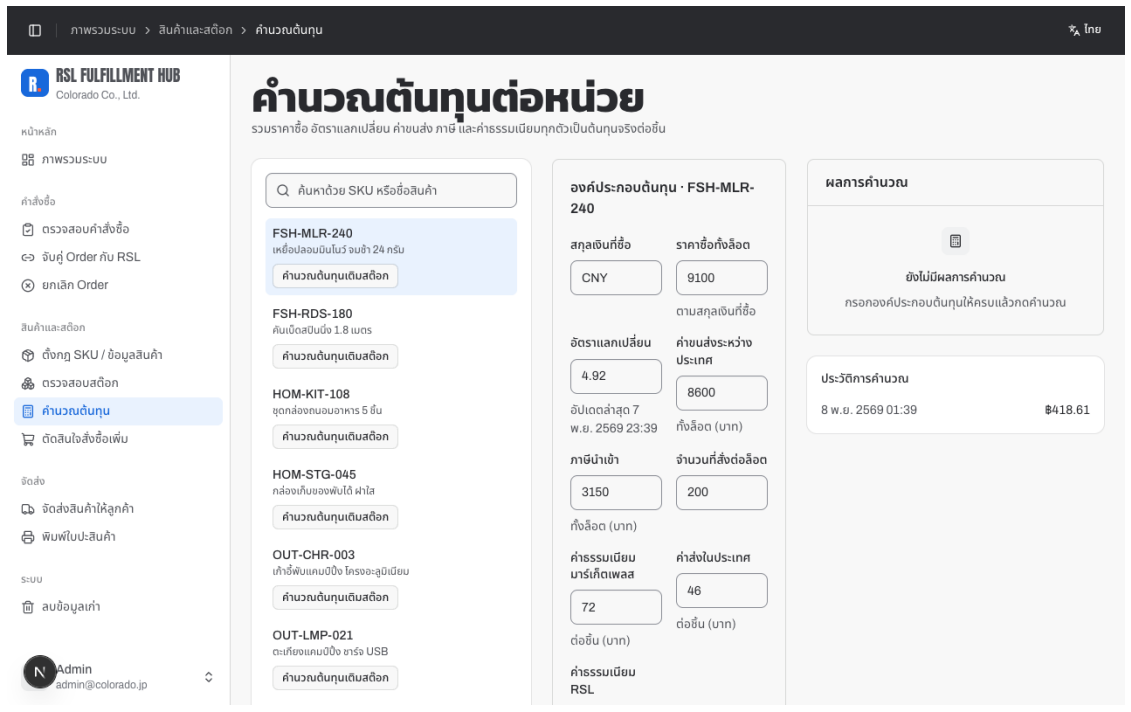
Q: สต๊อกรวมคิดอย่างไร

- คลังบริษัท + คลัง RSL เทียบกับเกณฑ์เติม (`reorder_threshold`)

SELECT พิสูจน์

```
SELECT k.sku, s.in_house_qty, s.rsl_qty, s.updated_at
FROM stock s JOIN skus k USING (sku_id) WHERE k.sku = 'FSH-MLR-240';
SELECT previous_in_house_qty, new_in_house_qty, previous_rsl_qty,
       new_rsl_qty, reason, adjusted_at
FROM stock_adjustment_history ORDER BY adjusted_at DESC LIMIT 3;
```

15.9 คำนวณต้นทุนต่อหน่วย (`/products/cost`)



ภาพ: หน้าคำนวณต้นทุน หลังเลือก FSH-MLR-240

บนจอมีอะไร

- ซ้าย: รายการสินค้า กด “คำนวณต้นทุนเติมสต๊อก” เพื่อเลือก
- กลาง: ฟอร์มองค์ประกอบต้นทุน (สกุลเงิน, ราคาซื้อทั้งล็อต, อัตราแลกเปลี่ยน, ค่าขนส่ง, ภาษี, จำนวนที่สั่ง, ค่าธรรมเนียม)
- ช่องติ๊ก “บันทึกค่าที่กรอกไว้ใช้ครั้งถัดไป” (เรียกสั้น ๆ ว่า “จำค่า” ติ๊กไว้ตั้งต้น)
- ขวา: ผลคำนวณ + “ประวัติการคำนวณ” + ลิงก์ไปหน้ารายงาน

กด “คำนวณ” แล้วเกิดอะไร (UC 4S)

- `calculateCostAction` → `calculateCostUseCase`
- ล็อกแถวต้นทุน (`Q4S.4`) คำนวณด้วยสูตรในบทที่ 8
- ติ๊ก “จำค่า” → บันทึกเป็นค่าล่าสุด (`Q4S.5`)
- **ทุกครั้ง** เก็บประวัติ (`Q4S.6`) แม้ไม่ติ๊กจำค่า

- อ่านราคาขาย (Q4S.7) ต้นทุนเกินราคาขาย → เตือน
- ตาราง: `costs`, `cost_history`

Q: 4S เป็น System auto แต่ทำไมต้องกดปุ่ม

- ระบบคำนวณเองเมื่อ Admin กรอกค่า (หน้านี้ หรือหน้าสินค้า) คนไม่ได้คิดเลขเอง

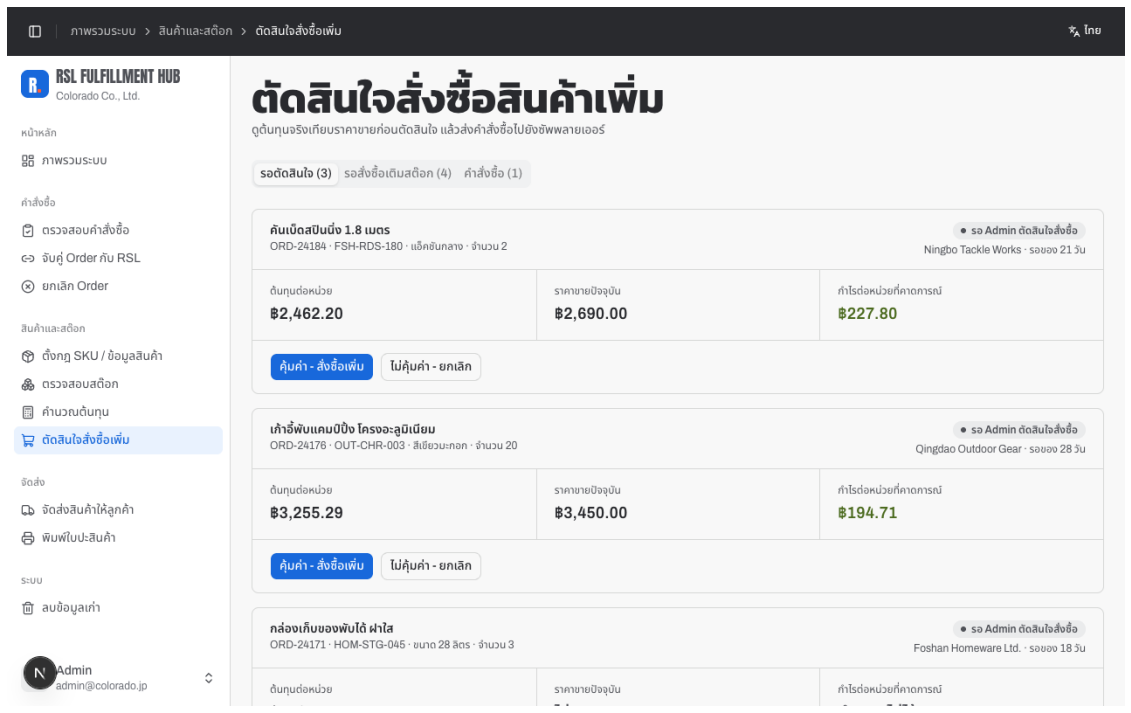
Q: อัตราแลกเปลี่ยนมาจากไหน

- Admin กรอกเอง ระบบจำเวลาที่แก้ (`exchange_rate_updated_at`)

SELECT พิสูจน์

```
SELECT k.sku, c.unit_cost, c.exchange_rate, c.order_qty, c.calculated_at
FROM costs c JOIN skus k USING (sku_id) WHERE k.sku = 'FSH-MLR-240';
SELECT changed_at, new_value FROM cost_history
ORDER BY changed_at DESC LIMIT 3;
```

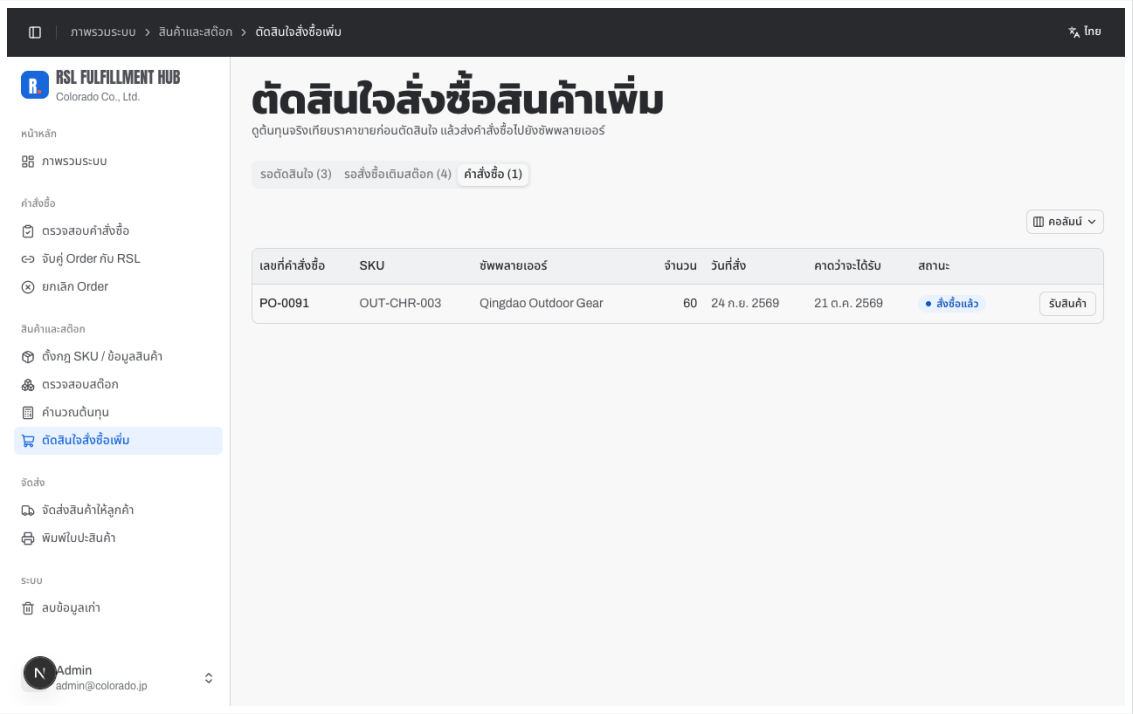
15.10 ตัดสินใจสั่งซื้อเพิ่ม (`/products/reorder`)



ภาพ: แท็บ “รอตัดสินใจ”

บนจอมีอะไร (3 แท็บ)

- **รอตัดสินใจ** (UC 5A): การ์ดละ order ที่สต๊อกไม่พอ แสดงต้นทุน/ราคาขาย/กำไรต่อหน่วย ปุ่ม “คุ้มค่า - สั่งซื้อเพิ่ม” และ “ไม่คุ้มค่า - ยกเลิก”
- **รอสั่งซื้อเติมสต๊อก** (UC 6A): รวม SKU ที่สต๊อกต่ำกว่าเกณฑ์ + SKU ที่ตัดสินใจ “คุ้มค่า” แล้ว ปุ่มสั่งซื้อ
- **คำสั่งซื้อ**: ใบสั่งซื้อ (PO) ทั้งหมด ปุ่ม “รับสินค้า” ที่ใบ สั่งซื้อแล้ว



ภาพ: แท็บ “คำสั่งซื้อ”: PO-0091 รอรับสินค้า

กดแล้วเกิดอะไร

- **คุ้มค่า / ไม่คุ้มค่า** (`decideReorderAction`): ล็อก (Q5A.1) บันทึก (Q5A.2) รอ Admin ตัดสินใจสั่งซื้อ → รอสั่งซื้อจาก Supplier หรือ รอยกเลิก Order; ไม่มีราคาขาย → รอดำเนินการด้วยตนเอง (Q5A.3)
- **สั่งซื้อ** (`purchaseReorderAction`): ตรวจ supplier (Q6A.3) มีใบค้างไหม (Q6A.4) ออกเลข PO (Q6A.6) สร้างใบ (Q6A.7) ผูก order (Q6A.8) ส่งให้ supplier แล้ว (Q6A.9) order → รอส่งคำสั่งซื้อ → สั่งซื้อแล้ว
- **รับสินค้า** (`receiveReorderAction`): ล็อก (Q6A.10) ใบเป็น ได้รับสินค้าแล้ว (Q6A.11) บวกคลังบริษัท (Q6A.12, Q6A.13) order กลับ รอตรวจสอบสต๊อก (Q6A.14) แล้ว **3S (** → **6S)** วิ่งต่อ
- ตาราง: `orders`, `reorders`, `stock`, `stock_adjustment_history`

Q: ตัดสินใจ “คุ้มค่า” แล้วทำไมยังไม่มีใบสั่งซื้อ

- 5A แค่ตัดสินใจ ใบสั่งซื้อเกิดตอนกดสั่งที่แท็บ “รอสั่งซื้อเติมสต๊อก” (6A) ตามเล่ม

Q: กดสั่งซ้ำแล้ว supplier ได้ 2 ใบไหม

- ไม่ ระบบถามยืนยันถ้ามีใบค้าง และส่งซ้ำด้วยรหัสคำขอเดิม

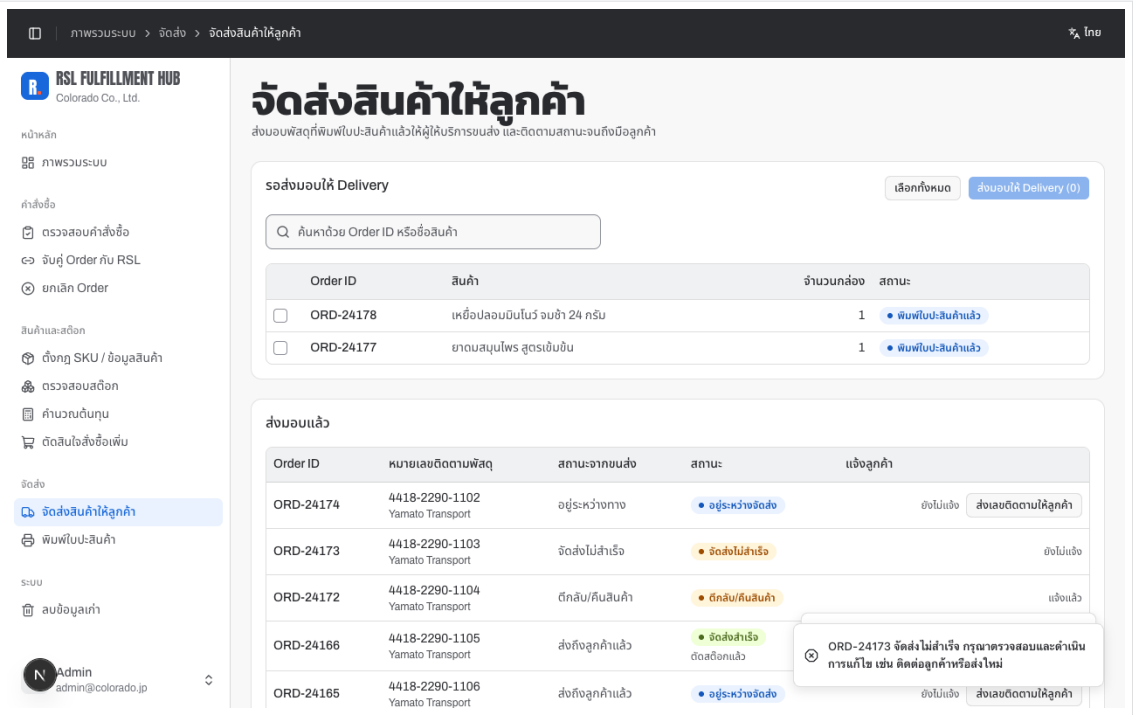
Q: รับของแล้วเข้าคลังไหน

- คลังบริษัท (`in_house_qty`) เสมอ รับได้ครั้งเดียวต่อใบ

SELECT พิสูจน์

```
SELECT order_id, order_status, reorder_decision, decided_qty, reorder_id
FROM orders WHERE order_id = 'ORD-24184';
SELECT reorder_id, sku_id, order_qty, reorder_status, expected_date,
       received_qty, received_at
FROM reorders ORDER BY reorder_id;
```

15.11 จัดส่งสินค้าให้ลูกค้า (/shipping)



ภาพ: หน้าจัดส่ง (ภาพหลังเปิดหน้าครั้งแรก มี toast แจ้ง ORD-24173)

เปิดหน้านี้

- เปิดหน้าแล้วระบบถามขนส่งทันที (`pollCarrierAction`, UC 8A Q8A.3–Q8A.5) ไม่ต้องกดอะไร
- ทดสอบหลัง `db:reset`: เปิดครั้งแรกเปลี่ยน 4 order
  - ORD-24166 แจ้งเลขติดตามแล้ว → จัดส่งสำเร็จ
  - ORD-24165 ยังเป็น อยู่ระหว่างจัดส่ง แต่ `delivery_status` เป็น delivered (รอแจ้งเลขก่อน)
  - ORD-24173 → จัดส่งไม่สำเร็จ (toast แดง)
  - ORD-24172 → ตีกลับ/คืนสินค้า และคืนสต๊อก FSH-RDS-180 คลัง RSL 1 → 2 (Q8A.7)
- เปิดครั้งที่สองไม่เปลี่ยนอะไรอีก
- ถ้าจะโชว์ dashboard “ก่อน” ให้เปิด dashboard ก่อนเข้าหน้านี้

บนจอมีอะไร

- ตารางบน “รอส่งมอบให้ Delivery”: order พิมพ์ใบปะสินค้าแล้ว มีช่องติ๊ก + ปุ่ม “ส่งมอบให้ Delivery (n)”
- ตารางล่าง “ส่งมอบแล้ว”: เลขพัสดุ + ขนส่ง, สถานะจากขนส่ง, สถานะ order, แจ้งลูกค้าแล้วหรือยัง, ปุ่ม “ส่งเลขติดตามให้ลูกค้า”

กดแล้วเกิดอะไร

- **ส่งมอบให้ Delivery** (UC 8A, `dispatchOrdersAction`): ขอเลขพัสดุ (Q8A.1) ได้เลข → บันทึก (Q8A.2) พิมพ์ใบปะสินค้าแล้ว → อยู่ระหว่างจัดส่ง; ยังไม่ได้เลข → รอส่งมอบ (Q8A.6) สต๊อก**ยังไม่ลด**
- **ส่งเลขติดตามให้ลูกค้า** (UC 5S, `notifyTrackingAction`): บันทึกความพยายาม (Q5S.1–Q5S.3) ส่งให้ Rakuten แล้วอัปเดต (Q5S.4) → แจ้งเลขติดตามแล้ว; ส่งไม่ได้ → รอแจ้งเลขติดตาม
- **7S ต่อเอง**: ตัดสต๊อก (Q7S.2–Q7S.5) ส่งยอดใหม่ไป Rakuten (Q7S.8–Q7S.11) ถ้าขนส่งยืนยันถึงแล้วด้วย → จัดส่งสำเร็จ (Q7S.7)
- ตาราง: `orders`, `order_items`, `stock`, `tracking_notification_logs`, `rakuten_stock_sync_states`

Q: ส่งมอบแล้วทำไมสต๊อกยังไม่ลด

- ตัดสต๊อกตอน 7S หลังแจ้งเลขติดตามลูกค้า (5S) ตามเล่ม

Q: จัดส่งสำเร็จเกิดเมื่อไร

- ครบ 3 ข้อ: ขนส่งยืนยันถึงแล้ว, แจ้งเลขแล้ว, ตัดสต๊อกครบ (บทที่ 7)

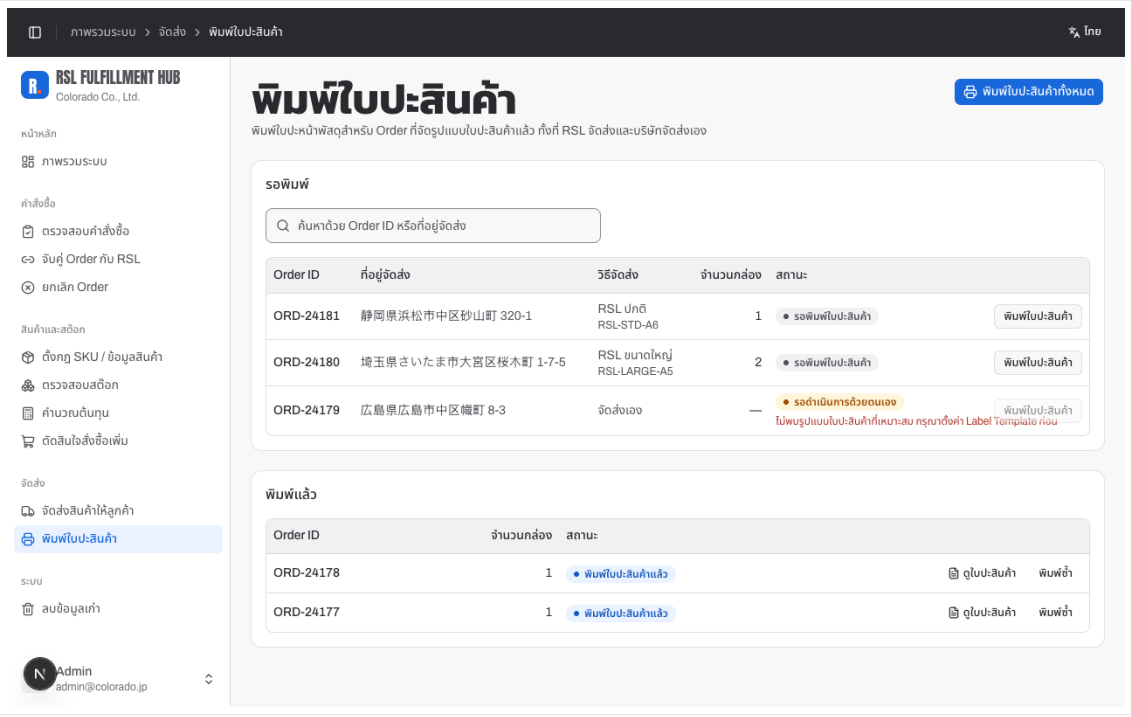
Q: ระบบติดตามพัสดุเป็นระยะไหม

- ไม่มี scheduler ใช้วิธีถามขนส่งทุกครั้งที่เปิดหน้านี้แทน

SELECT พิสูจน์

```
SELECT order_id, order_status, tracking_number, carrier_name,
       delivery_status, tracking_notified
FROM orders WHERE order_id IN ('ORD-24178', 'ORD-24174');
SELECT order_id, stock_deducted, deducted_at
FROM order_items WHERE order_id = 'ORD-24174';
SELECT order_id, outcome, sent_at FROM tracking_notification_logs
ORDER BY sent_at DESC LIMIT 3;
```

15.12 พิมพ์ใบปะสินค้า (`/shipping/label`)



ภาพ: หน้าพิมพ์ใบปะสินค้า

เปิดหน้านี้อาจสร้างใบปะ

- เปิดหน้าแล้วระบบเรียก `formatLabelsAction` (UC 6S) สร้างใบปะให้ order ที่ รอจัดรูปแบบใบปะสินค้า
- ข้อมูลหลัง reset ไม่มี order สถานะนั้น จึงไม่มีอะไรเปลี่ยน (ทดสอบแล้ว) แต่หลังเดโมบางขั้นอาจมี

บนจอมีอะไร

- ตาราง “รอพิมพ์”: ที่อยู่ (ญี่ปุ่น), วิธีจัดส่ง + แม่แบบใบปะ (เช่น RSL-STD-A6), จำนวนกล่อง, ปุ่ม “พิมพ์ใบปะสินค้า”
- ORD-24179 ขึ้นแดง “ไม่พบรูปแบบใบปะสินค้าที่เหมาะสม” = ทางเลือกของ 6S (ไม่มีแม่แบบสำหรับ “จัดส่งเอง”)
- ตาราง “พิมพ์แล้ว”: ปุ่ม “ดูใบปะสินค้า” (เปิดหน้ารายงาน) และ “พิมพ์ซ้ำ”
- ปุ่มขวาบน “พิมพ์ใบปะสินค้าทั้งหมด”

กดพิมพ์แล้วเกิดอะไร (UC 9A)

- `printLabelsAction` → `printLabelsUseCase`
- อ่านใบปะของ order (`Q9A.2`) ส่งเครื่องพิมพ์ (mock) ทีละกล่อง
- ทุกครั้งเก็บประวัติ (`Q9A.3`) ครบทุกกล่อง → `labels.label_status` = `printed` และ `รอพิมพ์ใบปะสินค้า` → `พิมพ์ใบปะสินค้าแล้ว` (`Q9A.4`)
- พิมพ์ซ้ำ: เก็บประวัติ `is_reprint` แต่ไม่เปลี่ยนสถานะ
- ตาราง: `labels`, `label_print_history`, `orders`

Q: เครื่องพิมพ์เสียระหว่างพิมพ์

- หยุดที่กล่องนั้น เก็บประวัติว่าพัง สถานะไม่เปลี่ยน ลองด้วยสวิตช์ 9A ใน `/dev`

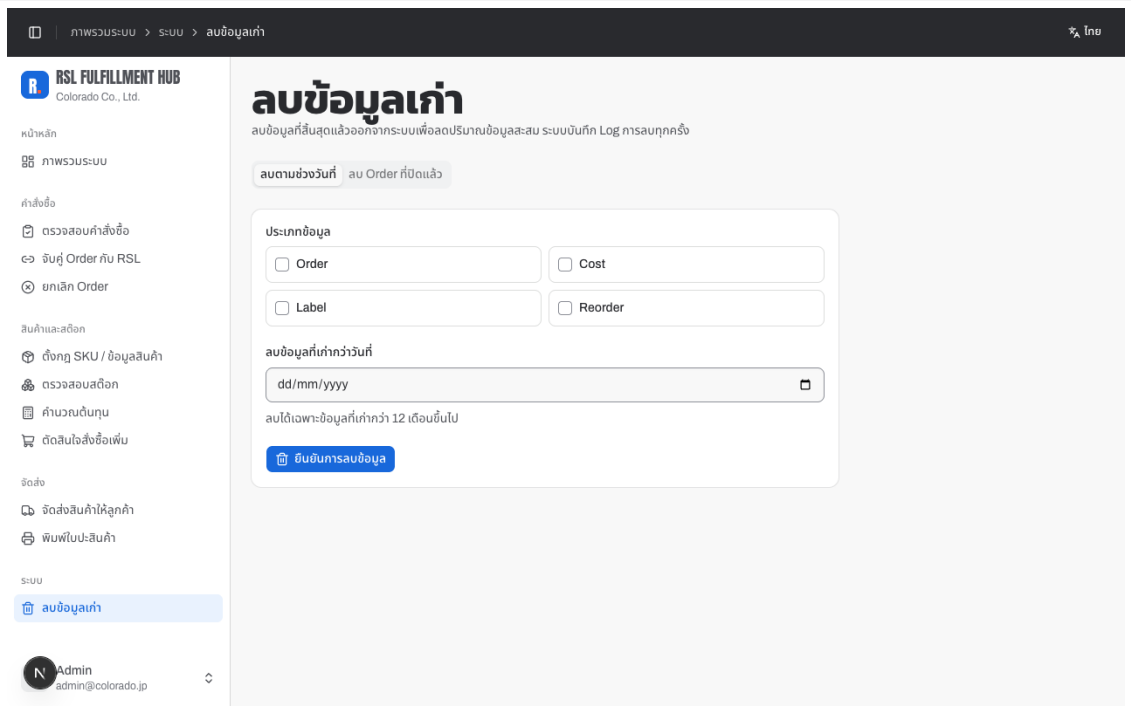
Q: order หนึ่งมีหลายใบปะได้ไหม

- ได้ 1 ใบต่อกล่อง (`parcel_seq` / `parcel_total`) เช่น ORD-24180 มี 2 กล่อง

SELECT พิสูจน์ (กดพิมพ์ ORD-24181)

```
SELECT label_id, order_id, label_status, printed_at
FROM labels WHERE order_id = 'ORD-24181';
SELECT label_id, is_reprint, outcome, attempted_at
FROM label_print_history ORDER BY attempted_at DESC LIMIT 5;
```

15.13 ลบข้อมูลเก่า (`/system/cleanup`)



ภาพ: แท็บ “ลบตามช่วงวันที่”

บนจอมีอะไร (2 แท็บ)

- **ลบตามช่วงวันที่**: ติ๊กประเภท (Order, Cost, Label, Reorder) + วันที่ ต้องเก่ากว่า 12 เดือน
- **ลบ Order ที่ปิดแล้ว**: ลบ `จัดส่งสำเร็จ` และ `ยกเลิกแล้ว` หลัง reset มี 5 รายการ
- ก่อนยืนยัน: หน้าจอแสดงรายการที่จะลบ และให้ดาวน์โหลด CSV สำรอง

กด “ยืนยันการลบข้อมูล” แล้วเกิดอะไร (UC 10A)

- `cleanupOrdersAction` → `cleanupOrdersUseCase` ใน transaction เดียว
- server ตรวจซ้ำว่าปิดงานแล้วจริง เก็บสำเนาลง `deletion_history` แล้วลบ order (`Q10A.1`) ใบปะ (`Q10A.2`) ต้นทุน (`Q10A.3`) ใบสั่งซื้อ (`Q10A.4`)
- ประวัติการลบอ่านด้วย `Q10A.5`

- พังกลางทาง → ไม่มีอะไรถูกลบ

ระวัง

- ปุ่มนี้**ลบจริง** ถ้ากดในเดโม รัน `npm run db:reset` หลังจบ

Q: ลบ order ที่ยังไม่ปิดได้ไหม

- ไม่ได้ server กรองซ้ำให้เหลือแค่ `จัดส่งสำเร็จ` / `ยกเลิกแล้ว`

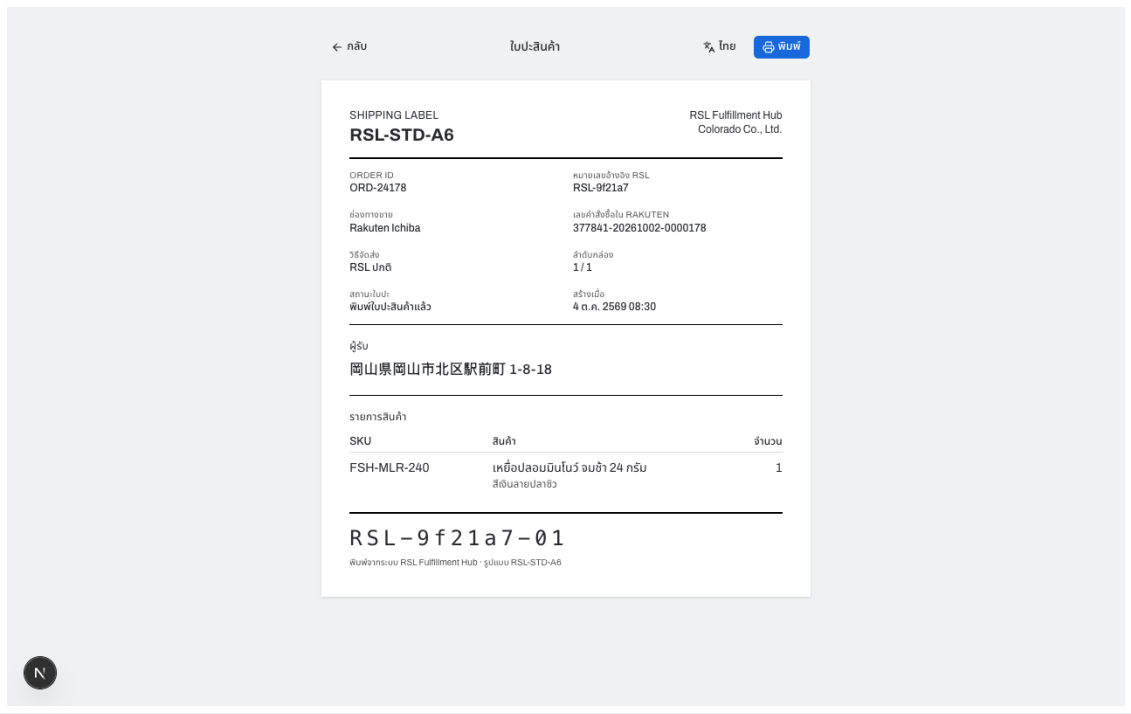
Q: ลบแล้วย้อนดูได้ไหม

- ดูได้จาก `deletion_history.order_snapshot` และไฟล์ CSV ที่ดาวน์โหลดก่อนลบ

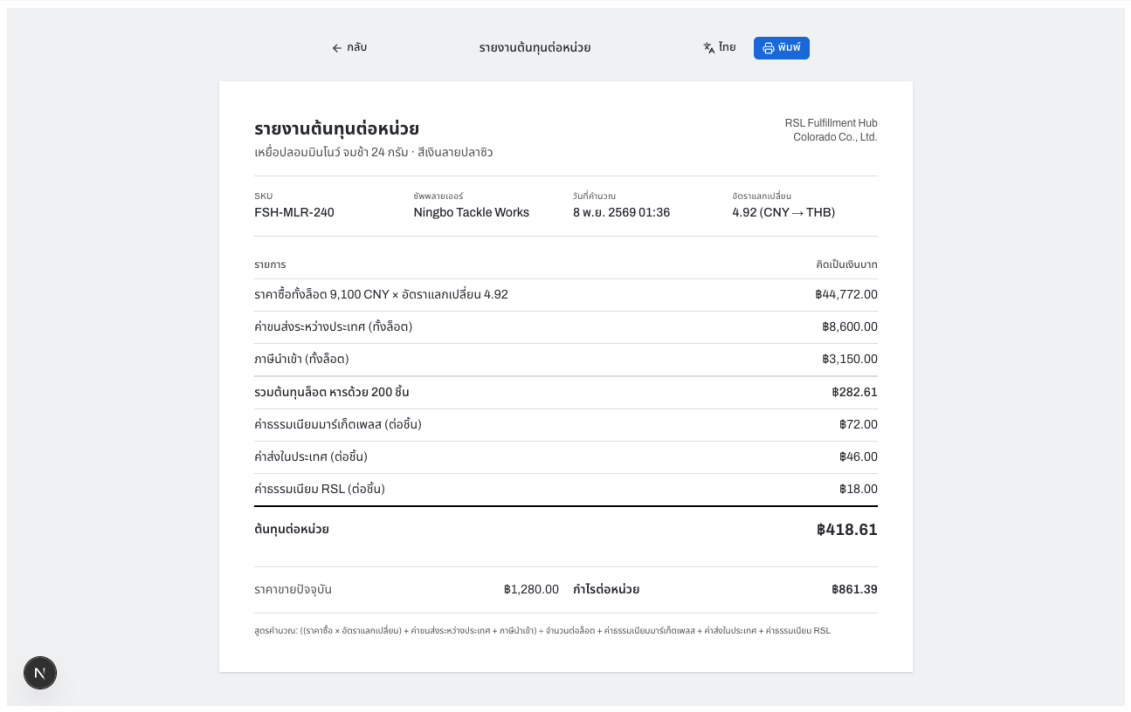
SELECT พิสูจน์

```
SELECT order_status, COUNT(*) FROM orders
WHERE order_status IN ('จัดส่งสำเร็จ', 'ยกเลิกแล้ว') GROUP BY 1;
SELECT order_reference, admin_id, deleted_at
FROM deletion_history ORDER BY deleted_at DESC;
```

15.14 หน้ารายงาน (`/reports/label/<order>`, `/reports/unit-cost/<sku>`)



ภาพ: ใบปะสินค้าของ ORD-24178 (เปิดจากปุ่ม “ดูใบปะสินค้า”)



ภาพ: รายงานต้นทุนต่อหน่วยของ FSH-MLR-240 (เปิดจากหน้าคำนวณต้นทุน)

- หน้าสำหรับพิมพ์ ไม่มีเมนูซ้าย มีปุ่ม “กลับ”, สลับภาษา, “พิมพ์”
- **อ่านอย่างเดียว** ไม่มีปุ่มที่เขียน DB
- ใบปะ: แม่แบบ, เลข order, เลข RSL, กล่องที่เท่าไร, ที่อยู่, รายการสินค้า, barcode
- รายงานต้นทุน: แยกทุกบรรทัดของสูตร ตั้งแต่ราคาซื้อ × เรต จนถึงต้นทุนต่อหน่วยและกำไร
- order ที่ยังไม่มีใบปะ → ขึ้น “Order นี้ยังไม่มีใบปะสินค้า”

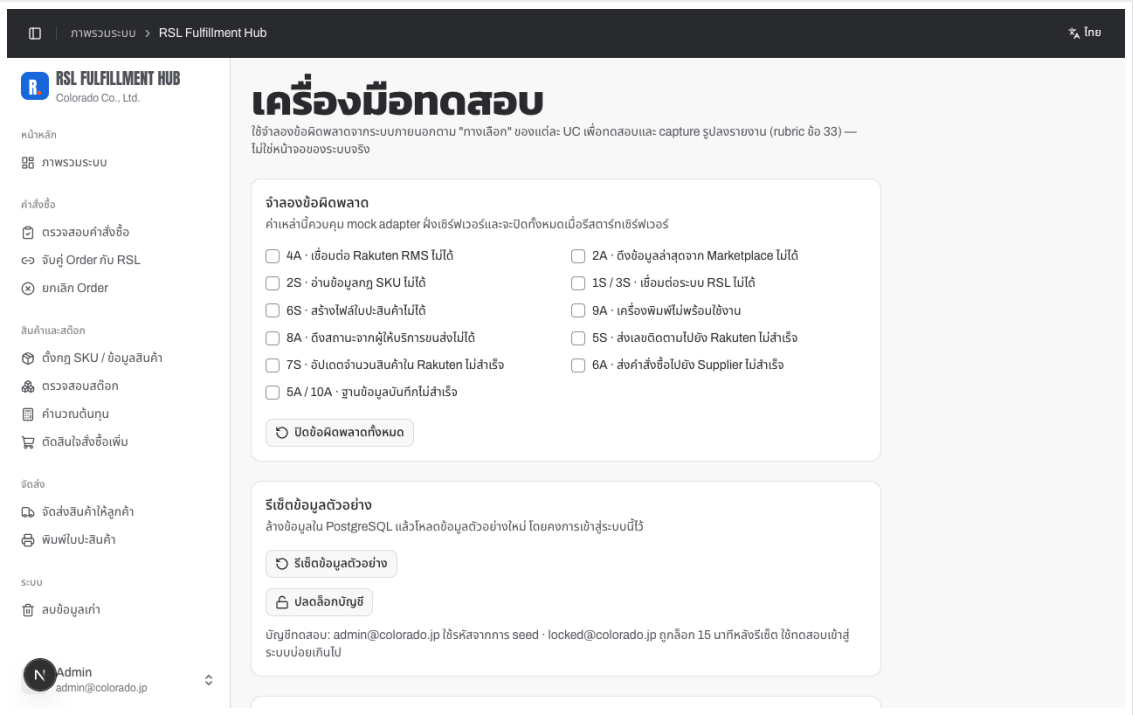
Q: รายงานในระบบมีอะไรบ้าง

- 2 ฉบับ: ใบปะสินค้า (6S/9A) และรายงานต้นทุนต่อหน่วย (4S)

Q: “วันที่คำนวณ” ในรายงานต้นทุนคือวันไหน

- `costs.calculated_at` หลัง reset จะเป็นเวลาที่รัน seed

15.15 เครื่องมือทดสอบ (/dev)



ภาพ: หน้า /dev

บนจอมีอะไร

- “จำลองข้อผิดพลาด” 11 สวิตช์ แต่ละตัวผูกกับทางเลือกของ UC เช่น “9A · เครื่องพิมพ์ไม่พร้อมใช้งาน”
- ปุ่ม “ปิดข้อผิดพลาดทั้งหมด”
- “รีเซ็ตข้อมูลตัวอย่าง” และ “ปลดล็อกบัญชี”

ระวัง

- สวิตช์เก็บในหน่วยความจำของ server ไม่ใช่ของเบราว์เซอร์ → **มีผลกับทุกคนที่เปิดเว็บ** รวมแท็บอื่น
- เปิดสวิตช์ทิ้งไว้แล้วลืม → หน้าอื่นจะ error แปลก ๆ ก่อนพรีให้กด “ปิดข้อผิดพลาดทั้งหมด”
- “รีเซ็ตข้อมูลตัวอย่าง” = `db:reset` ลบทุกตาราง แล้ว login ให้ใหม่ และปิดสวิตช์ทั้งหมด
- “ปลดล็อกบัญชี” ล้าง `fail_attempts` / `locked_until` ทุกบัญชี
- รีสตาร์ท server แล้วสวิตช์กลับเป็นปิด

Q: หน้านี้เป็นส่วนของระบบจริงไหม

- ไม่ เป็นเครื่องมือทดสอบทางเลือกของ UC (rubric ข้อ 33) ไม่อยู่ในเมนู

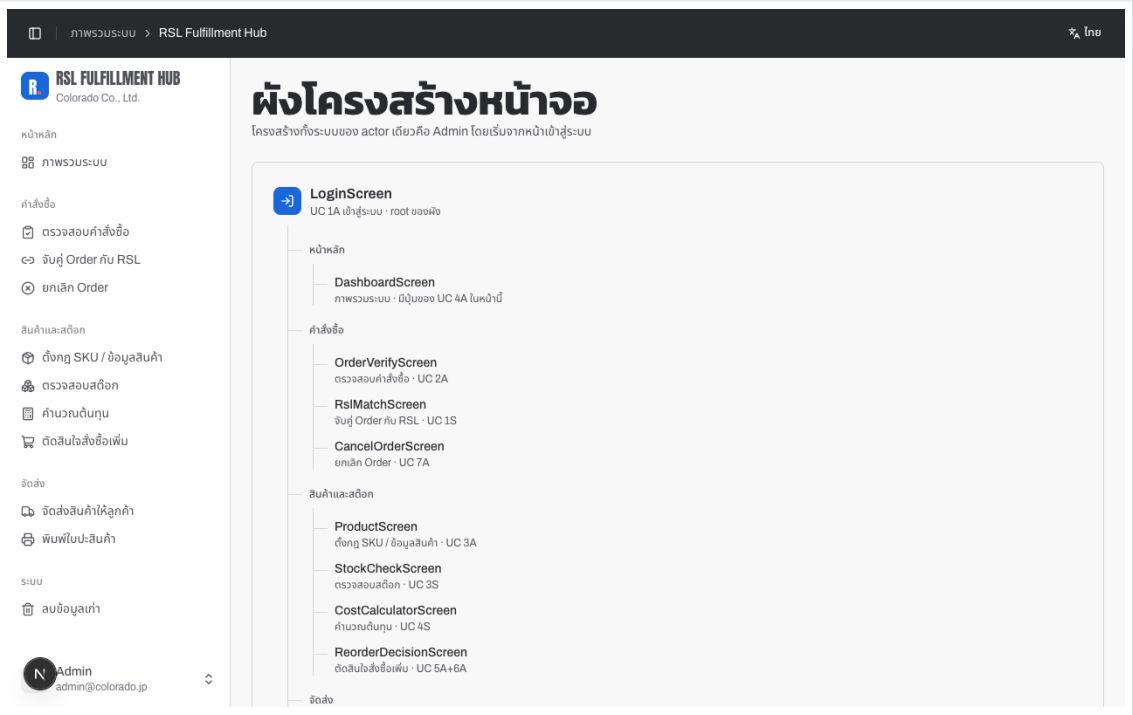
Q: ระบบภายนอกล่มแล้วข้อมูลค้างครึ่งทางไหม

- ไม่ ทั้งก้อนถูกยกเลิก และบันทึก error ลง `auto_error_logs`

SELECT พิสูจน์

```
SELECT use_case_id, error_code, safe_message, occurred_at
FROM auto_error_logs ORDER BY occurred_at DESC LIMIT 5;
SELECT user_email, fail_attempts, locked_until FROM admins;
```

15.16 ผังโครงสร้างหน้าจอ (`/sitemap`)



ภาพ: หน้า Site map

- แสดงผังหน้าจอทั้งระบบ เริ่มที่ LoginScreen แล้วแตกตามกลุ่มเมนู
- ชื่อ `...Screen` ตรงกับ object ฝั่ง View ใน sequence diagram
- ดึงจาก `lib/nav.ts` ชุดเดียวกับเมนูซ้าย ผังจึงไม่หลุดจากเมนูจริง
- อ่านอย่างเดียว ใช้จับภาพใส่รายงาน (rubric ข้อ 29)

Q: ทำไมเมนูมีแค่ 11 หน้า แต่ UC มี 17

- UC อัตโนมัติ (2S, 6S, 7S) ไม่มีหน้าของตัวเอง ส่วน 4A อยู่ใน dashboard และ 5S อยู่ในหน้าจัดส่ง

16 เดโม: เล่นโปรแกรม + SELECT ให้ดู

อธิบายแบบภาษาคน

1. ก่อนพรี: เปิด Docker, รัน `npm run db:reset` แล้ว `npm run dev`
2. เปิด terminal อีกหน้าต่างไว้ต่อ DB ตลอดเวลา
3. ทุกครั้งที่กดในเว็บ ให้ SELECT ก่อนกด 1 ครั้ง และหลังกด 1 ครั้ง ให้เห็นค่าที่เปลี่ยน
4. ถ้าพลาด กด reset ที่หน้า `/dev` แล้วเริ่มใหม่

16.1 เปิด DB

```
docker start rsl-pg
docker exec -it rsl-pg psql -U rsl_app -d rsl_fulfillment_hub
-- PostgreSQL installed locally: psql -U rsl_app -d rsl_fulfillment_hub

\dt                 -- list 17 tables (+ schema_migrations)
\d orders           -- columns of one table
\x auto             -- wide rows readable
\q                  -- quit
```

16.2 ชุด SELECT สำหรับเดโม

1) ภาพรวมสถานะ Order

```
SELECT order_status, COUNT(*) FROM orders
GROUP BY order_status ORDER BY 2 DESC;
```

2) ตรวจสอบคำสั่งซื้อ (UC 2A) แล้วงานอัตโนมัติวิ่งต่อ (2S, 3S, 6S)

```
SELECT o.order_id, o.order_status, o.manual_reason,
       i.channel_sku, i.qty, i.fulfill_source
FROM orders o JOIN order_items i USING (order_id)
WHERE o.order_id = 'ORD-24194';
```

- ก่อนกด: สถานะ `รอตรวจสอบคำสั่งซื้อ`
- กดยืนยันที่หน้า `/orders/verify` แล้ว SELECT ซ้ำ
- หลังกด: ระบบทำ 2S, 3S, 6S ต่อเองในรอบเดียว ORD-24194 มีของแค่คลังบริษัท จึงกลายเป็น `รอพิมพ์ใบปะสินค้า` และ `fulfill_source = in_house`
- ทดสอบจริงแล้ว (7 พ.ย.) ด้วยข้อมูลหลัง `db:reset`

```
SELECT o.order_id, o.order_status, o.tracking_number,
       o.delivery_status, l.label_id, l.label_status
FROM orders o LEFT JOIN labels l USING (order_id)
WHERE o.order_id = 'ORD-24194';
```

- ใช้ SELECT นี้ซ้ำหลัง UC 2A, UC 9A, UC 8A
- หลัง UC 2A: ใบปะ `ORD-24194-LABEL-1` สถานะ `ready`
- หลัง UC 9A: ใบปะเป็น `printed` และ order เป็น `พิมพ์ใบปะสินค้าแล้ว`
- หลัง UC 8A: order เป็น `อยู่ระหว่างจัดส่ง` มีเลขพัสดุ `4418-2291-4194` และ `delivery_status = in_transit`

3) สต๊อกถูกตัดจริง (UC 5S → UC 7S)

```
SELECT k.sku, s.in_house_qty, s.rsl_qty, s.updated_at
FROM stock s JOIN skus k USING (sku_id)
ORDER BY s.updated_at DESC LIMIT 5;

SELECT order_id, stock_deducted, deducted_at
FROM order_items WHERE stock_deducted ORDER BY deducted_at DESC LIMIT 5;
```

- หลัง UC 8A สต๊อก**ยังไม่ลด** เป็นเรื่องปกติ
- สต๊อกถูกตัดหลังกด “ส่งเลขติดตามให้ลูกค้า” (UC 5S) แล้วระบบทำ UC 7S ต่อ
- ถ้าจะโชว์ จัดส่งสำเร็จ ต้องให้ขนส่งยืนยันว่าส่งถึงก่อน ในเดโมทำแทนได้ด้วย SQL ข้างล่าง แล้วค่อยกดส่งเลขติดตาม

```
UPDATE orders SET delivery_status = 'delivered'
WHERE order_id = 'ORD-24194';   -- demo only: pretend carrier delivered
```

4) ตัดสินใจและใบสั่งซื้อ Supplier (UC 5A, UC 6A)

```
SELECT order_id, order_status FROM orders
WHERE order_status IN ('รอ Admin ตัดสินใจสั่งซื้อ', 'รอสั่งซื้อจาก Supplier');


SELECT reorder_id, sku_id, order_qty, reorder_status, received_qty
FROM reorders ORDER BY reorder_id;
```

- หลัง UC 5A (กด “คุ้มค่า” ที่ order ใบแรก ORD-24184): order ย้ายจาก รอ Admin ตัดสินใจสั่งซื้อ เป็น รอสั่งซื้อจาก Supplier
- ตาราง `reorders` **ยังไม่เปลี่ยน** (มีแค่ PO-0091) แถวใหม่เกิดตอนกดสั่งซื้อ (UC 6A) ในแท็บ “รอสั่งซื้อเติมสต๊อก”

5) ใบปะสินค้าและประวัติพิมพ์ (UC 6S, UC 9A)

```
SELECT label_id, order_id, label_status FROM labels ORDER BY 1 DESC LIMIT 5;
SELECT label_id, is_reprint, outcome, attempted_at
FROM label_print_history ORDER BY attempted_at DESC LIMIT 5;
```

6) Login และการล็อกบัญชี (UC 1A)

```
SELECT user_email, fail_attempts, locked_until FROM admins;
SELECT session_id, admin_id, expires_at, revoked_at
FROM sessions;                -- token stored as token_hash only
```

ระวัง

- ชื่อคอลัมน์ทุกข้อเช็กกับ DB จริงแล้ว ถ้าสงสัยดูด้วย `\d <ตาราง>` ก่อนพรี 1 รอบ
- อย่าโชว์รหัสผ่านหรือ `.env.local` บนจอ
- ถ้าอาจารย์ให้เขียน SELECT เอง: เริ่มจาก `SELECT * FROM <ตาราง> LIMIT 5;` ก่อน แล้วค่อยเติม WHERE / JOIN

16.3 ลำดับเดโมแนะนำ (ประมาณ 5 นาที)

1. Login (UC 1A) แล้ว SELECT `sessions`
2. นำเข้าคำสั่งซื้อ (UC 4A) แล้ว SELECT ภาพรวมสถานะ: รอตรวจสอบคำสั่งซื้อ 7 → 8 และ รอดำเนินการด้วยตนเอง 1 → 2 (เข้าใหม่ 2 ข้ามซ้ำ 1)
3. ตรวจสอบคำสั่งซื้อ ORD-24194 (UC 2A) แล้ว SELECT order นั้น ให้เห็นงานอัตโนมัติวิ่งต่อ
4. พิมพ์ใบปะ (UC 9A) และส่งมอบ (UC 8A) แล้ว SELECT order + `labels` ซ้ำ
5. ตัดสินใจสั่งซื้อ (UC 5A) แล้ว SELECT สถานะ order และ `reorders`
6. ถ้ามีเวลา: ตั้ง `delivery_status` เป็น delivered แล้วกดส่งเลขติดตาม (UC 5S) ให้เห็นสต๊อกลดและ จัดส่งสำเร็จ

ข้อความเตือนที่จะเห็นตอนนำเข้า (ปกติ)

- ORD-24181 ลูกค้ายกเลิก, SKU ที่ไม่ได้ลงทะเบียน และข้อมูลไม่ครบ มาจากข้อมูลตัวอย่างที่ตั้งใจใส่ไว้
- อธิบายว่าเป็นกรณีทางเลือกใน Use Case Description ไม่ใช่ bug

7. จบเดโมแล้วรัน `npm run db:reset` ให้ข้อมูลกลับเหมือนเดิม

17 เตรียมตอบคำถาม: Diagram

ตัวเลขที่ต้องจำ

- Use Case 17 ตัว (A = Admin เริ่ม, S = ระบบทำเอง) Actor 4 ตัว
- ตาราง 17 ตาราง ความสัมพันธ์ 22 เส้น
- SQL 113 ข้อ ตรงกับโค้ดทุกข้อ
- สถานะ Order 22 สถานะ
- State diagram 6 เรื่อง (Order 2 รูป + ใบสั่งซื้อ, ใบปะ, SKU, ขนส่ง, ส่งยอดสต๊อก)
- Sequence 17 UC แบบ MVC ไม่มีลูกศรกลับหา Actor

17.1 Business Process (As-Is / To-Be / Pain point)

Q: As-Is ต่างจาก To-Be อย่างไร

- As-Is: Admin ทำมือทุกขั้น ตั้งแต่ดู order, เช็กสต๊อก, พิมพ์ใบปะ ไปจนถึงแจ้งเลขพัสดุ
- To-Be: ระบบทำ 3 เรื่องอัตโนมัติ คือ พิมพ์ใบปะ, สั่งซื้อซ้ำ และคำนวณต้นทุนต่อหน่วย
- Admin เหลือแค่ตรวจและตัดสินใจ

Q: Pain point คืออะไร

- งานซ้ำและพลาดง่ายเมื่อทำมือ
- ไม่รู้สต๊อกจริง ทำให้ขายเกิน
- ต้นทุนต่อหน่วยคิดยาก

17.2 Use Case Diagram และ Use Case Description

Q: ทำไมมี UC ที่ลงท้าย S

- S = System auto ไม่มีคนกด ระบบเรียกเองต่อจาก UC อื่นใน `executeWorkflow`
- เช่น UC 2A ตรวจสอบเสร็จ ระบบเรียก UC 2S → UC 3S → UC 6S ต่อในรอบเดียว

Q: ทำไมไม่มี scheduler

- ระบบเรียกงานอัตโนมัติต่อท้ายทุก action ที่เปลี่ยนข้อมูล
- เฟส 1 Admin ใช้คนเดียว จึงไม่ต้องมี cron

17.3 Sequence และ Collaboration Diagram

Q: อ่าน Sequence อย่างไร

- ซ้ายไปขวา: Admin → `:XxxScreen` → `:XxxController` → `:XxxUseCase` / `:XxxQueries` (Model) → `:DBConnector`
- ลูกศรทึบ = เรียก, ลูกศรประ = ส่งค่ากลับ
- ไม่มีลูกศรกลับหา Actor: Screen แสดงผลเอง (self message)
- เลข Q<UC>.n บนลูกศร = SQL ข้อเดียวกับใน Use Case Description

Q: ทำไม object ติด Actor ชื่อ Screen ไม่ใช่ View

- เกณฑ์ข้อ 24 บอกว่า object ติด actor ต้องเป็น user interface และห้ามใช้ชื่อว่า view
- Screen คือชั้น View ของ MVC ตามที่อาจารย์สอน คู่กับ Controller และ Model
- ชื่อตรงกับหน้าจอในหัวข้อ UI (2.21) เช่น `:OrderVerifyScreen`

Q: ในโค้ดมี class Controller จริงไหม

- ไม่มี โค้ดเป็นฟังก์ชันธรรมดา
- `:OrderVerifyController` ในรูป = ฟังก์ชัน `verifyOrderAction` ใน `actions.ts`
- รูปใช้ชื่อตามแบบ MVC เพื่อให้อ่านง่าย แต่ทุกชื่อมีไฟล์จริงกำกับในวงเล็บ

Q: Collaboration ต่างจาก Sequence อย่างไร

- ข้อมูลเดียวกัน แต่ Collaboration เน้นว่าใครคุยกับใคร มีเลขลำดับบนเส้น
- Sequence เน้นลำดับเวลาจากบนลงล่าง

17.4 CRUD Matrix และ Query Table

Q: CRUD Matrix บอกอะไร

- แต่ละ UC ทำอะไรกับตารางไหน (Create/Read/Update/Delete)
- ตรงกับ SQL ใน Query Table ทุกข้อ

Q: ให้ลองรัน query ข้อนี้สด

- เปิด psql ตามหัวข้อ 16
- copy SQL จาก Query Table แทน `\$1` ด้วยค่าจริง เช่น `'ORD-24194'`

17.5 ER Diagram และ Table Structure

Q: ทำไมมี 17 ตาราง ไม่ใช่ 15 แบบเดิม

- เพิ่ม `sessions` สำหรับ login และ `rakuten_stock_sync_states` สำหรับส่งยอดสต๊อกให้ Rakuten
- ER ใหม่วาดจากฐานข้อมูลจริง

Q: Order กับ Order Item สัมพันธ์กันอย่างไร

- 1 Order มีได้หลาย Order Item (1:N) ผ่าน FK `order_items.order_id`
- ลบ Order แล้ว item หายตาม (`ON DELETE CASCADE`)

Q: ทำไม stock แยกจาก skus

- 1 SKU มี stock 1 แถว (1:1) เก็บ `in_house_qty` และ `rsl_qty`
- ล็อกแถว stock (`FOR UPDATE`) ตอนเช็กและตัดสต๊อก กันตัดซ้ำ

17.6 Class Diagram และ Component Diagram

Q: Class diagram มาจากไหน

- 1 class ต่อ 1 ตาราง attribute = คอลัมน์ method = UC ที่ใช้ตารางนั้น

Q: Component diagram มีอะไร

- FrontEnd (Next.js) → Server Actions → Use-case → Queries → PostgreSQL
- Adapters ต่อ Rakuten, RSL, ขนส่ง, Supplier (ตอนนี้เป็น mock)

17.7 State Diagram

Q: State diagram ของอะไรบ้าง

- 6 เรื่อง: Order (`order_status` 22 สถานะ แบ่ง 2 รูป), ใบสั่งซื้อ, ใบปะ, SKU, สถานะขนส่ง, การส่งยอดสต๊อกไป Rakuten
- เส้นแต่ละเส้นเขียนแบบ `[guard] event / <UC> (Qid)` เช่น `[RSL พอ] ตรวจสอบสต๊อก (Q3S.4)`
- ทุกเส้นตรงกับ SQL ที่ update คอลัมน์นั้นในโค้ด

Q: ใบสั่งซื้อ (`reorders.reorder_status`) เปลี่ยนอย่างไร

- เริ่มที่ `รอส่งคำสั่งซื้อ` ตอนกดสั่งซื้อ (Q6A.7)
- supplier รับ → `สั่งซื้อแล้ว` ส่งไม่ได้ → อยู่ที่เดิม (Q6A.9)
- รับของ → `ได้รับสินค้าแล้ว` (Q6A.11)
- ยกเลิก order (7A) → `ยกเลิกแล้ว` (Q7A.4)
- ลบข้อมูลเก่า (10A) → จบ (Q10A.4)

Q: ใบปะ (`labels.label_status`) เปลี่ยนอย่างไร

- 6S สร้างใบปะ → `ready` (Q6S.4)
- 9A พิมพ์ → `printed` (Q9A.4)
- ยกเลิกการจับคู่ RSL (1S) ลบใบปะที่ยัง `ready` (Q1S.4)
- 10A ลบใบปะที่พิมพ์แล้ว (Q10A.2)

Q: SKU, ขนส่ง และการส่งยอดสต๊อกล่ะ

- SKU: `active` กับ `inactive` สลับกันได้ใน 3A (Q3A.6, Q3A.9) ลบได้เมื่อไม่มีข้อมูลอื่นอ้างถึง (Q3A.11)
- ขนส่ง (`delivery_status`): ว่าง → `in_transit` ตอน 8A (Q8A.2) → `delivered`, `failed` หรือ `returned` (Q8A.4)
- ส่งยอดสต๊อก (`sync_status`): สต๊อกเปลี่ยน → `pending` (Q3S.8, Q6A.12, Q7S.6) ส่งสำเร็จ → `synced` (Q7S.10) ส่งไม่ได้ → `pending` รอส่งใหม่ (Q7S.11)

18 ภาคผนวก: แผนที่ไฟล์

ไฟล์	หน้าที่
ฐานข้อมูล	
`db/migrations/001_core_schema.sql`	ตารางหลัก 9 ตาราง + 22 สถานะ
`db/migrations/002_runtime_records.sql`	session, log, ประวัติ, สถานะส่งยอด
`db/migrations/003_order_workflow_state.sql`	เหตุผลค้าง, คำขอยกเลิก, ประวัติแก้
`db/migrations/004_product_inventory_reorders.sql`	ประวัติปรับสต๊อก, การตัดสินใจสั่งซื้อ
`db/migrations/005_shipping_labels_cleanup.sql`	สถานะขนส่ง, ประวัติพิมพ์ใบปะ
`db/migrations/006_reorder_links_and_receive.sql`	`orders.reorder_id` (ใบสั่งซื้อ 1 : N order), `reorders.received_qty`
`db/migrations/007_admin_login_lock.sql`	`admins.locked_until` (ล็อก 15 นาที)
`db/migrations/008_reorder_number.sql`	sequence `reorder_no_seq` เลขใบสั่งซื้อ PO-0091, PO-0092, … (ไม่เพิ่มตาราง)
`db/seed/cli.ts`, `index.ts`, `reset-smoke.ts`	คำสั่ง db:*, ใส่ข้อมูลตัวอย่าง, test
จุดรับคำขอจากหน้าเว็บ	
`src/app/(app)/actions.ts`	Server Action ของงานหลัก (20 ตัว)
`src/app/login/actions.ts`	login / logout
`src/app/(app)/dev/actions.ts`	สวิตช์จำลองความเสีย, reset ข้อมูล
`src/app/(app)/layout.tsx`, `src/app/reports/layout.tsx`	ตรวจ login + โหลด snapshot
กฎธุรกิจ (`src/lib/use-cases/`)	
`execute-workflow.ts`	ระบบงานอัตโนมัติ 2S/3S/6S/7S
`import-orders.ts` 4A, `verify-order.ts` 2A	นำเข้า / ตรวจสอบ order
`match-order.ts` 1S/2S, `check-stock.ts` 3S	จับคู่ RSL, SKU, ตรวจสต๊อก
`format-labels.ts` 6S, `print-label.ts` 9A	สร้าง / พิมพ์ใบปะ
`dispatch-order.ts`, `poll-carrier.ts` 8A	ส่งมอบ / ติดตามพัสดุ
`notify-tracking.ts` 5S	แจ้งเลขติดตาม
`deduct-stock.ts`, `sync-rakuten-stock.ts` 7S	ตัดสต๊อก / ส่งยอดไป Rakuten
`cancel-order.ts` 7A, `cleanup-orders.ts` 10A	ยกเลิก / ลบข้อมูลเก่า
`products.ts` 3A, `stock.ts` 3S, `costs.ts` 4S, `reorders.ts` 5A/6A	สินค้า สต๊อก ต้นทุน สั่งซื้อ
SQL (`src/lib/queries/`)	
`app-state.ts`	รวมข้อมูลทุกหน้า (snapshot)
ไฟล์อื่น 13 ไฟล์	SQL แยกตามเรื่อง ชื่อ `Q<UC>_<n>`
`README.md`	กติกาตั้งเลข Q
โครงสร้างพื้นฐาน	
`src/lib/db.ts`, `src/lib/db/errors.ts`	ต่อฐานข้อมูล, transaction, แปลง error
`src/lib/auth/*`	รหัสผ่าน, session, ตรวจ login
`src/lib/adapters/*`	ตัวกลางคุยระบบภายนอก + ของจำลอง + สวิตช์
หน้าเว็บ / ข้อมูลจำลอง	
`src/lib/store.tsx`	ส่ง snapshot ให้ทุกหน้า
`src/lib/workflow.ts`	ระบบเก่า ตอนนี้ใช้เป็นตัวช่วยของหน้าเว็บ
`src/mock/*.ts`	ข้อมูลตัวอย่าง + ข้อมูลของระบบจำลอง
`src/types/*.ts`	ชนิดข้อมูล + สูตรต้นทุน
เอกสารใน repo	
`specs/001-order-status-alignment/`	spec กฎสถานะ order (เสร็จแล้ว)
`specs/002-real-database/`	spec ต่อฐานข้อมูลจริง (T050 ค้าง)
`docs/source/*`	use case description, business flow, ER ที่ทีมใช้อ้างอิง

ไฟล์	หน้าที่
`CLAUDE.md`, `AGENTS.md`, `PRODUCT.md`, `DESIGN.md`	กติกาโค้ด, คำเตือน Next 16, ขอบเขตผลิตภัณฑ์, กฎหน้าตา